Supplementary Material for "What Can Be Enforced? Limits of Runtime Guardrails for Tool-Using LLM Agents"

Shawn Ray

SUPPLEMENT A
PROOFS

A.1 Preliminaries and notation

We work in the Cantor topology on A^ω, whose basic open sets are the cylinders uA^ω for $u \in A^*$. A set $P \subseteq A^\omega$ is *closed* iff it contains all its limit points, where x is a limit point of P if every prefix $x_{\le n}$ is a prefix of some element of P. We use repeatedly the classical characterization of safety.

Lemma S1 (Safety is topological closure)**.** *$P \subseteq A^\omega$ is a safety property (Def. 2) if and only if P is closed. Equivalently, $x \in P$ iff every finite prefix $x_{\le n}$ is a* good prefix*, i.e., $x_{\le n} \notin \mathrm{Bad}(P)A^*$.*

Proof. If P is safety and $x \notin P$, then x has a bad prefix $x_{\le n}$, so the open cylinder $x_{\le n}A^\omega$ is disjoint from P; hence the complement of P is open and P is closed. Conversely, if P is closed and $x \notin P$, then x is not a limit point, so some prefix $x_{\le n}$ extends into P nowhere, i.e., $x_{\le n}A^\omega \cap P = \emptyset$; that prefix is a bad prefix, so P is safety. The prefix characterization is the contrapositive: $x \in P$ iff no prefix is bad. □

Recall (Def. 4) that a gate g decides from committed effect history $\tau_{<t} \in (A \setminus \{\bot\})^*$, proposal a_t, and $\Pi(\tau_{<t}, a_t)$ (in the oracle case g is computable relative to Π, so it may also use the bits Π returned at earlier committed steps; for the fixed Π its decision is still a function of $\tau_{<t}$ and a_t). We assume Π reads this same effect history (equivalently, an interface defined on full committed traces must be $\bot$-blind). We keep Π explicit: it is an available oracle interface, not a promise that its bits are definable in $\mathcal{T}$. A $(\mathcal{T};\Pi)$*-gate* runs a deterministic keyed-counter register automaton (Def. 2), whose guards combine $\mathcal{T}$ tests with queries to these bits, once and online over the committed effect history, and decides each proposal by the one transition the automaton takes on it from the configuration reached (labeled allow, block, or sub(t) for a term t over the proposal's fields, the registers, and constants). A *proposal stream* is a map from committed histories to proposed actions; together with g it determines the committed trace. We write $\bot$ for the neutral no-op of Def. 1. Because policies are scored on *effect* traces (Def. 1), the recognizer $\mathcal{R}$ of Good(P) reads only effect actions: committing $\bot$ produces no effect and leaves $\mathcal{R}$'s state unchanged (so $\delta(q,\bot) = q$), and a trace is safe for P iff no finite prefix of its effect trace is bad and an infinite effect trace lies in P (for safety P the second clause follows from the first by Lemma S1). Table S1 collects the notation.

TABLE S1
Notation.

Symbol	Meaning
A, A^*, A^ω	action alphabet; finite and infinite traces
$\bot$, ext	neutral no-op action; external-effect projection (deletes $\bot$)
P, Bad(P), Good(P)	policy; minimal bad prefixes; good prefixes
$\mathcal{T}$, Π	guard theory; fixed vector of oracle predicate bits
RA$[\mathcal{T};\Pi]$	languages recognized by keyed-counter register automata with $\mathcal{T}$-guards and oracle bits Π
π_i, $\hat{\pi}_i^\theta$	ground-truth predicate; thresholded judge
$\alpha(\theta), \beta(\theta)$	judge false-alarm and miss rates
$\mathrm{roc}(\beta)$, $\mathrm{roc}_I(\beta)$	ROC frontier: least α at miss β (given I)
g, Dec	gate; decisions {allow, block, sub}
R, U	episode unsafe rate; utility loss
$F_H(\delta)$	controlled episode frontier (Def. 5)
h_t, ψ, μ	gate's observable history; gate policy; logging policy (Def. 5, Prop. 4)
F_0, F_1, Q_ρ	compliant and violating score laws; attacked violation-score law
o	untrusted context the judge reads (Def. 6)
$r(\theta)$, $\hat{\theta}$	per-step conditional miss risk; calibrated threshold
H, δ	horizon; target episode risk
$\eta(\rho;\theta)$, $\bar{\eta}(\rho)$	robustness margin at θ; uniform margin $\sup_\theta \eta(\rho;\theta)$

A.2 T1: oracle characterization

Theorem 1 (Oracle characterization)**.** *Fix the available oracle interface Π and a nonempty policy P. Then P is precisely enforceable by a deterministic $(\mathcal{T};\Pi)$-gate iff it is safety and* Good$(P) \in$ RA$[\mathcal{T};\Pi]$*; the witness simulates that recognizer and refuses exactly the one-step extensions it rejects.*

Proof. ($\Leftarrow$). Let $\mathcal{R}$ be a deterministic keyed-counter register automaton (Def. 2) recognizing Good(P) with guards in $(\mathcal{T};\Pi)$; below, q ranges over its configurations (control state, registers, key maps, counters), q_0 is the initial one, δ the deterministic update, and B the absorbing bad configurations. Define the gate G that maintains $q_t = \delta^*(q_0, c_{\le t})$ and, on proposed a at state q, commits a (moving to $\delta(q,a)$) if $\delta(q,a) \notin B$, and otherwise commits $\bot$ (a *block*, leaving the state at q).

Soundness. Blocking commits $\bot$ (no effect; the recognizer does not move); an allowed action moves within Good(P) by construction, since G blocks exactly the one-step extensions entering B, and $q_0 \notin B$ because $\varepsilon \in$ Good(P) for nonempty P (the only use of nonemptiness: for $P = \emptyset$, Good$(P) = \emptyset$ is recognizable

yet no gate is sound). So the committed run visits only good states and every effect prefix $\mathsf{ext}(c_{\le t}) \in \mathrm{Good}(P) = A^* \setminus \mathrm{Bad}(P)A^*$ is not bad. Hence the committed trace is safe for P (Def. 1): if the effect trace is infinite it lies in P by Lemma S1; if the gate eventually blocks forever, the finite effect trace still has no bad prefix. This holds for every proposal stream.

Transparency. Suppose the proposed ω-trace w lies in P. By Lemma S1 every prefix of w is good, so running $\mathcal{R}$ on w never reaches B; hence G never blocks and commits w unchanged.

Complexity. Each step evaluates the enabled guarded transition on the current data, registers, and supplied oracle bits. This is polynomial when guard evaluation and updates are polynomial; no satisfiability solver is implicitly assumed.

($\Rightarrow$). We work in the effect alphabet $A \setminus \{\bot\}$, over which P and its traces live. Suppose the $(\mathcal{T};\Pi)$-gate g precisely enforces P but P is not safety. By Lemma S1, P is not closed, so it has a limit point $x \notin P$ that is a doomed *effect* trace: for every n there is an effect trace $z^{(n)} \in P$ with $z^{(n)}_{\le n} = x_{\le n}$. Let S be the *open-loop* stream proposing $x_1, x_2, \ldots$ in turn (a fixed sequence, independent of the gate's actions). We show by induction on t that g allows x_t, so the committed effect trace is x.

Base case $t = 0$ is immediate. For the step, assume g has allowed $x_1, \ldots, x_{t-1}$, so the committed effect history is $x_{<t}$ and the proposal is x_t. Consider the open-loop stream $S^{(t)}$ proposing $z^{(t)}_1, z^{(t)}_2, \ldots$; its proposed effect trace is $z^{(t)} \in P$, so by *prefixwise* transparency (Def. 4) the committed effect prefix equals $z^{(t)}_{\le k}$ at every step k. Thus g blocks or alters nothing on $S^{(t)}$, and in particular allows $z^{(t)}_t = x_t$ from the committed effect history $z^{(t)}_{<t} = x_{<t}$. Because g conditions on the *effect* history (Def. 4), its decision here is the value $g(x_{<t}, x_t, \Pi(x_{<t}, x_t))$, identical under S and $S^{(t)}$; being deterministic, g also allows x_t under S. By induction g commits the infinite effect trace $x \notin P$, contradicting soundness (clause (ii) of Def. 1). Note that x has no bad prefix (every $x_{\le n}$ extends into P), which is why soundness must constrain infinite effect traces and not only their finite prefixes. Hence P is safety.

It remains to recognize $\mathrm{Good}(P)$. Simulate the finite control, registers, and committed-history updates of g while reading a proposed finite word u, and move to a rejecting sink at the first step whose committed action differs from the proposal (an identity $\mathsf{sub}(a)$ counts as allow). This is a deterministic $(\mathcal{T};\Pi)$ register automaton. If $u \in \mathrm{Good}(P)$, then u has a completion in P (otherwise u itself would be bad), and prefixwise transparency forces every symbol of u to be allowed. Conversely, if u contains a first bad prefix va, transparency on a compliant completion of v forces g to allow v, while soundness forbids allowing a from that history; thus the simulation rejects. Its language is exactly $\mathrm{Good}(P)$, proving $\mathrm{Good}(P) \in \mathsf{RA}[\mathcal{T};\Pi]$. □

Remark S1 (General computable gates). Dropping the register restriction, the same limit-point argument shows an arbitrary computable gate with interface Π precisely enforces a nonempty P iff P is safety and good-prefix membership is decidable relative to Π. $(\mathcal{T};\Pi)$-gates are the register-recognizable subclass.

For a randomized gate, quantify separately: for every (possibly adaptive) proposal strategy, soundness holds with probability one over gate randomness; and for every fixed compliant proposal trace, prefixwise transparency holds with probability one. On each compliant $z^{(t)}$ agreeing with a doomed limit trace x through t, transparency makes the conditional decision at history $x_{<t}$ equal to allow almost surely. The countable intersection of these probability-one events makes the gate commit all of x almost surely, contradicting almost-sure soundness. Thus randomization does not extend the safety boundary (although the register-representability converse above is stated only for deterministic gates).

Proposition 1 (A strict hierarchy)**.** *(i) The gate-enforceable policies are a strict subclass of the edit-automaton enforceable policies (edit automata given the same oracle bits Π). (ii) If Π is action-local and recipients range over an infinite equality-only data sort, then no $(\mathcal{T};\Pi)$-gate without key maps (whatever its registers, global counters, and counter guards and updates) enforces the per-recipient cap of one, which a $(\mathcal{T}_0;\Pi)$-gate with one key-local flag enforces.*

Proof. Every gate-enforceable policy is safety with a good-prefix language decidable relative to Π (Theorem 1, and the remark above for general computable gates); an edit automaton given the same oracle bits Π enforces each such policy by suppressing any action that would enter $\mathrm{Bad}(P)$ and re-emitting nothing (safety properties are a subclass of the renewal properties edit automata enforce [1, 2]), giving containment. For strictness, let A contain irreversible pay and $\mathsf{confirm}$ and let

$$P^* = \{w \in (A \setminus \{\bot\})^\omega : \text{every } \mathsf{pay} \text{ has a later } \mathsf{confirm}\}.$$

P^* is not safety. The effect trace pay^ω violates P^*, yet no finite prefix is bad: any prefix pay^k extends to $\mathsf{pay}^k\,\mathsf{confirm}^\omega \in P^*$. By Lemma S1, P^* is not safety, so by Theorem 1 no gate enforces it. *Self-contained impossibility.* Each pay must be decided irrevocably. On the open-loop stream pay^ω, suppose the gate has allowed the first $t-1$ payments; the committed effect history pay^{t-1} and the proposal pay are then exactly those of the compliant stream $\mathsf{pay}^t\,\mathsf{confirm}^\omega \in P^*$, on which prefixwise transparency forces allow. So the gate allows every pay and commits the infinite effect trace $\mathsf{pay}^\omega \notin P^*$; the payments are irreversible, no later decision can repair the violation, and the gate is unsound. Substitution does not help: replacing pay by a semantically distinct action fails transparency, and replacing it by an equivalent pay returns to the first case. *Edit-enforceable.* An ω-policy counts as edit-enforceable when some extension of it to finite executions is. Containment above judges finite executions as good prefixes; that extension of P^* (which has no bad prefix) accepts every finite execution and is not renewal, since $\mathsf{pay}^\omega \notin P^*$ has infinitely many valid prefixes. If instead a finite execution is judged valid iff each of its pays has a later $\mathsf{confirm}$ within it, P^* is a renewal property; an edit automaton suppresses each unconfirmed pay, and every action after it, into a buffer and emits the buffer in order upon the next $\mathsf{confirm}$ (and never if $\mathsf{confirm}$ does not occur), which is edit-enforceable by the renewal-property theorem of Ligatti et al. [2]. Thus P^* separates the classes. The comparison is at the level of enforceable policy *classes*, each in its own model's semantics: edit automata enforce P^* by buffering, which presumes actions can be delayed and re-emitted; irreversibility removes exactly that power, which is why the classical class is unattainable for a pre-execution gate over committed actions, not a defect of the gate.

Part (ii). Let P_1 be the set of infinite effect traces in which no two send actions name the same recipient (other actions are unconstrained). It is safety, its bad prefixes being exactly the finite effect traces with two sends to one recipient, and it is nonempty because the recipient sort is infinite. A $(\mathcal{T}_0;\Pi)$-gate enforces P_1 by Theorem 1: a (K) component maps each recipient to a flag set by its first send and rejects a send whose flag is set; it is

trim, since a send to an unflagged recipient is always available, so its non-bad language is exactly $\mathrm{Good}(P_1)$. Now let g be a $(\mathcal{T};\Pi)$-gate without key maps, with r registers and finitely many global counters, over an equality-only recipient sort: a recipient enters its guards and updates only through equality with registers, with the other fields of the current action, and with the finitely many constants of the automaton (guard constants, initial register contents, and constants in substitution terms), and through the oracle bits, which for action-local Π depend on the proposed action alone. Fix the non-recipient fields $\bar{f}$ of send. As Π takes finitely many values, some infinite set D' of recipients, avoiding the automaton's constants and the data values in $\bar{f}$, gives sends with equal bits. Feed $\mathsf{send}(d_1, \bar{f}), \ldots, \mathsf{send}(d_{r+1}, \bar{f})$ for pairwise distinct $d_i \in D'$; this prefix extends to the compliant stream that continues with fresh recipients, so prefixwise transparency allows every send. The configuration reached stores at most r data values, so some d_j is in no register. Let $e \in D'$ be neither among the d_i nor in any register. In this configuration, $\mathsf{send}(d_j, \bar{f})$ and $\mathsf{send}(e, \bar{f})$ satisfy exactly the same guards (neither recipient equals a register, a constant, or a value in $\bar{f}$, and their bits agree), so g takes the same transition on both and outputs the same decision term. Transparency on the compliant stream that continues with $\mathsf{send}(e, \bar{f})$ and further fresh recipients forces that term to commit $\mathsf{send}(e, \bar{f})$ unchanged. As e equals no register value and no constant, the term is allow or a substitution that copies the proposed recipient, and on $\mathsf{send}(d_j, \bar{f})$ either one commits $\mathsf{send}(d_j, \bar{f})$, completing a bad prefix and contradicting soundness. The argument is the pumping argument of Kaminski and Francez [3]. It needs determinism and the register-automaton form: a general computable gate (Remark 1) enforces P_1 without key maps, and so does a register-free gate given the history-dependent bit $\mathbf{1}[\tau \cdot a \in \mathrm{Good}(P_1)]$. □

Proposition S1 (Substitution: same class; a utility gain only with a judge)**.** *(i) Adding* $\mathsf{sub}(a')$ *does not change the class in T1, and in the oracle regime a transparent gate never alters a compliant proposal, so substitution cannot lower U there. (ii) Consider a judge-driven gate (relaxing the block-only clause of Assumption 1) that, whenever its judge flags a proposal at good prefix u, can compute from its own inputs a rewrite with* $ua' \in \mathrm{Good}(P)$ *(certified without the ground-truth bits) and conditional loss at most* $\ell_{\mathrm{sub}} \in [0,1]$ *relative to unit loss for a block. Then substituting instead of blocking reduces the false-block loss at good prefixes from* $\alpha(\theta)$ *to at most* $\ell_{\mathrm{sub}}\alpha(\theta)$ *at unchanged miss.*

Proof. (i) Run the T1 recognizer. When ua would be rejected, either block or choose any a' with $ua' \in \mathrm{Good}(P)$ and update the recognizer on a'. The committed run remains good, and no rewrite fires on a compliant proposed trace, so soundness and transparency hold. Conversely, the T1 limit argument and never-alter recognizer apply to gates with substitution as well, so adding that decision cannot enlarge the class. The oracle recognizer rejects only bad extensions, so it never alters a compliant proposal and $U = 0$ with or without substitution. The same holds for every transparent gate: if the committed effect history is u and $ua \in \mathrm{Good}(P)$, prefixwise transparency on the open-loop stream of a completion $w \in P$ of ua forces allow at (u, a), and the decision depends only on (u, a).

(ii) Under the stated loss normalization, a compliant proposal is flagged, and so sent to the rewrite branch, with probability $\alpha(\theta)$ and costs at most ℓ_{sub} there; a flagged violation is replaced by a prefix-safe action instead of being blocked, so the decisions on violations, and hence the miss, are unchanged. Its expected conditional loss is therefore at most $\ell_{\mathrm{sub}}\alpha(\theta)$. This is a graded loss: U (Def. 4) charges every alteration, a substitution included, as a unit loss, so substitution leaves U unchanged and lowers only this graded false-block loss. Without the prefix-safe rewrite and loss assumptions, only the class-equivalence statement remains; substitution need not improve an arbitrary task utility. □

Remark S2 (Escalation). Replacing a block by an oracle that always returns a prefix-safe commit action is a special case of substitution (with the oracle's loss in place of ℓ_{sub}). The class conclusion needs the oracle to be part of the interface Π: an external oracle that decides $\mathrm{Good}(P)$ would let a trivial gate enforce every nonempty safety P, beyond $\mathsf{RA}[\mathcal{T};\Pi]$ (the safety boundary itself still holds by the limit-point argument). The utility conclusion is relative to that oracle model; no claim is made for a fallible or non-prefix-safe human decision.

A.3 T2: enforcement under a fallible judge

For a one-step binary decision, let F_1 and F_0 be the laws of pre-commit information I conditional on a violating and compliant proposal. A randomized block rule is a measurable $\varphi(I) \in [0,1]$. Its miss and false-block probabilities are $\beta(\varphi) = 1 - \mathbb{E}_1\varphi$ and $\alpha(\varphi) = \mathbb{E}_0\varphi$.

Lemma S2 (Neyman–Pearson optimality)**.** *Among all block rules measurable with respect to I that have miss at most* β_0*, the minimum false-block probability is* $\mathrm{roc}_I(\beta_0)$*, attained by a randomized likelihood-ratio threshold.*

Proof. Blocking a violation has probability $\mathbb{P}[\mathsf{block} \mid \pi = 1] = \mathbb{E}_1[\varphi] = 1 - \text{miss}$, and false-alarm is $\mathbb{P}[\mathsf{block} \mid \pi = 0] = \mathbb{E}_0[\varphi]$. The constraint is $\mathbb{E}_1\varphi \geq 1 - \beta_0$ and the objective is $\mathbb{E}_0\varphi$. This is the Neyman–Pearson program [4, 5]; its randomized likelihood-ratio solution has value $\mathrm{roc}_I(\beta_0)$. Boundary randomization convexifies the error region, so $\beta \mapsto \mathrm{roc}_I(\beta)$ is nonincreasing and convex. □

Proposition 2 (Exogenous information frontier)**.** *For a fixed one-step law, the exact frontier from information I is* $B^\star_I(\beta) = \mathrm{roc}_I(\beta)$*. If* I' *Blackwell-refines I, then* $\mathrm{roc}_{I'}(\beta) \leq \mathrm{roc}_I(\beta)$ *pointwise, with equality at every* β *when the likelihood ratio of* I' *is almost surely a function of I.*

Proof. The equality is Lemma S2. If I is a garbling of I', every I-measurable rule is implementable from I' by applying the garbling first. Thus the feasible error region for I' contains the one for I, giving the pointwise inequality. Conversely, if the likelihood ratio of I' is almost surely a function of I, the randomized likelihood-ratio test attaining $\mathrm{roc}_{I'}$ (Lemma S2) is I-measurable, so $\mathrm{roc}_{I'} = \mathrm{roc}_I$ at every β. A strict gain thus needs label information in I' beyond I; conditional dependence among judges alone does not guarantee it. When the likelihood ratio is a nondecreasing function of a score s, its level sets are s-intervals on which $dF_1 = k\,dF_0$, so blocking the upper part of the boundary interval (randomizing only at an atom of s) has the same miss and false-block probability as randomizing over the whole level set: a threshold on s, randomized on ties, attains roc_I. □

Corollary 1 (Continuation-neutral episode lift)**.** *Under the product specialization of Assumption 1, a stationary test with fresh per-step randomization has* $R_H(\beta) = 1 - (1-\beta)^H$ *on the all-violation stream and* $R_H(\beta) = 1 - (1 - q\beta)^H$ *for i.i.d.* Bernoulli(q) *labels. Within this product class,* $B^\star_{I,H}(\delta) = \mathrm{roc}_I(R_H^{-1}(\delta))$.

Proof. The product formulas follow because "a violation is committed" has probability β (or $q\beta$) independently at each step. Since the repeated test has per-step false-block probability α, imposing $R_H(\beta) \le \delta$ is equivalent to $\beta \le R_H^{-1}(\delta)$; Proposition 2 gives the optimum at the largest admissible miss. Shared episode-level randomization is excluded because it need not obey the product formula. Open-loop time-varying tests with per-step misses β_t do no better: independence gives $R_H = 1 - \prod_t (1 - q\beta_t)$ ($q = 1$ on the all-violation stream), so $R_H \le \delta$ iff $\sum_t \gamma_t \le -\ln(1-\delta)$ with $\gamma_t = -\ln(1 - q\beta_t)$. The per-step cost $f(\gamma) = \mathrm{roc}_I\big((1 - e^{-\gamma})/q\big)$ is convex, a convex nonincreasing function (Lemma S2) of a concave increasing one, so Jensen's inequality and monotonicity of f give $\frac{1}{H}\sum_t f(\gamma_t) \ge f\big(\frac{1}{H}\sum_t \gamma_t\big) \ge f\big(-\ln(1-\delta)/H\big) = \mathrm{roc}_I(R_H^{-1}(\delta))$ for $\delta \le R_H(1)$; for larger δ allow-all is feasible. History dependence can do strictly better even without feedback: with an uninformative score ($\mathrm{roc}_I(\beta) = 1 - \beta$), $H \ge 2$ and $0 < \delta < R_H(1)$, blocking at step one with probability $1 - \delta/R_H(1)$ and repeating that decision at every later step gives risk δ at per-step false-block probability $1 - \delta/R_H(1) < 1 - R_H^{-1}(\delta)$, because R_H is strictly concave, so R_H^{-1} is strictly convex on $[0, R_H(1)]$ with $R_H^{-1}(0) = 0$ and $R_H^{-1}(R_H(1)) = 1$. On the all-violation stream the per-step miss budget $R_H^{-1}(\delta) = 1 - (1-\delta)^{1/H}$ is at least the union-bound budget δ/H of Theorem 3, because $(1 - \delta/H)^H \ge 1 - \delta$; the gap is what the product conditions buy. □

Theorem 2 (Endogenous finite-model frontier)**.** *For a specified finite model, the controlled frontier of Definition 5 is the image of a reachable-belief occupancy polytope; its lower boundary is convex and piecewise linear, its Lagrange dual is solved by belief-state Bellman recursion, and an optimum mixes at most two deterministic belief-Markov policies.*

Proof. Augment the latent state with an absorbing unsafe bit and let r_t be one only on first entry, so $\sum_t r_t$ is the episode-unsafe indicator. With finite abstractions and finite H, finitely many observation histories and posteriors $b_t = \mathbb{P}(X_t \mid h_t)$ are reachable. The posterior is policy-independent: the factors $\psi(D_s \mid h_s)$ in the joint law of $(X_{\le t}, h_t)$ are functions of h_t and cancel in Bayes' rule. Hence b_{t+1} is a fixed function of (b_t, D_t) and the next observation, whose conditional law given (h_t, D_t) depends only on (b_t, D_t), so (b_t) is a controlled Markov chain with a policy-independent kernel, and $\mathbb{E}[c_t \mid h_t, D_t]$, $\mathbb{E}[r_t \mid h_t, D_t]$ depend on h_t only through b_t. Belief is a sufficient controlled state: the conditional action probabilities of any perfect-recall policy define a belief-Markov policy with the same occupancy measure, hence the same (R, C).

Let $\zeta_t(b, d)$ be the probability of occupying belief b and choosing d, and $K_t(b' \mid b, d)$ the policy-independent belief transition kernel (the Bayes update after d and the next observation). The nonnegative flow constraints are

$$\sum_d \zeta_1(b, d) = \mathbb{P}[b_1 = b],$$
$$\sum_d \zeta_{t+1}(b', d) = \sum_{b,d} \zeta_t(b, d) K_t(b' \mid b, d).$$

Conversely, disintegrating a feasible flow gives a belief-Markov policy. Both R and C are linear in ζ, proving the main occupancy program (if δ is below the least achievable R, the program is infeasible and $F_H(\delta) = +\infty$; the claims below concern feasible δ). Its feasible set is a bounded polytope; its two-dimensional performance image is a compact convex polygon, with a convex piecewise-linear lower boundary.

Finite LP duality gives

$$F_H(\delta) = \max_{\lambda \ge 0} \{\textstyle\sum_b \mathbb{P}[b_1 = b]\, V_1^\lambda(b) - \lambda\delta\},$$
$$V_t^\lambda(b) = \min_d \{c_t(b, d) + \lambda r_t(b, d) + \mathbb{E}[V_{t+1}^\lambda(b') \mid b, d]\}.$$

With $V_{H+1}^\lambda \equiv 0$, a deterministic minimizer exists for each λ. Any constrained boundary point lies on a supporting segment of the performance polygon. Every endpoint vertex is the image of an extreme occupancy point, and extreme finite-horizon occupancy points are induced by deterministic belief-Markov policies. Hence sampling one endpoint policy once per episode implements the boundary point with support at most two. □

Comparing Bellman Q-values gives the dynamic rule in the main text: for $c_t = w_t \mathbf{1}[\mathsf{block}, \text{compliant}]$ with w_t fixed given b_t, and unsafe entry only via an allowed violation, blocking costs $w_t(1 - p_t)$, allowing incurs λp_t (with the unsafe bit observed, so an allowed violation is a first entry), and rearranging $Q(\mathsf{block}) \le Q(\mathsf{allow})$ yields $\lambda p_t - w_t(1 - p_t) \ge \Gamma_t^\lambda$.

Proposition 3 (ROC does not identify a feedback frontier)**.** *Two $H = 2$ systems can have identical judge ROC and identical ungated trajectory law but different $F_H(0)$.*

Proof. Take $H = 2$, a constant score that is the gate's only information about the labels, and $Y_1 \sim \mathrm{Bernoulli}(q)$ with $0 < q < 1$. If step one is allowed, let $Y_2 = 0$ in both systems, so their ungated laws and ROCs coincide. If step one is blocked, system A has $Y_2 = 0$, while system B presents $Y_2 \sim \mathrm{Bernoulli}(p)$, $0 < p < 1$. Zero risk forces a first-step block. A then need not block again, whereas B must. For expected compliant-block count, set $c_t = \mathbf{1}[\text{block and compliant}]$ and let r_t indicate first unsafe entry as in Definition 5. Then $F_A(0) = 1 - q$ and $F_B(0) = (1 - q) + (1 - p)$, proving non-identifiability. The constant score is not essential: if F_0, F_1 are not mutually singular, let $\kappa = 1 - \sup\{F_0(N) : F_1(N) = 0\} > 0$; zero risk forces blocking an F_1-full set at every step that may violate, so $F_A(0) = (1-q)\kappa < (1-q)\kappa + (q + (1-q)\kappa)(1-p)\kappa = F_B(0)$. □

Corollary 2 (Identification impossibility)**.** *Over any model class containing systems A and B of Proposition 3, no estimator whose input is limited to ungated trajectories and static judge scores can uniformly identify $F_H(0)$, hence it cannot identify the full controlled frontier.*

Proof. Any estimator restricted to ungated trajectories and static scores has the same input law in systems A and B, but the target takes the distinct values $F_A(0)$ and $F_B(0)$. It therefore cannot identify both uniformly. Because $F_H(0)$ is one point of the controlled frontier, neither can it identify the full frontier. □

Proposition 4 (Exploratory blocking restores identification)**.** *In the model of Definition 5, let a logging gate choose D_t from h_t with known probabilities $\mu(d \mid h_t) \ge \varepsilon > 0$ for both decisions at every reachable history, and let the logs record (h_t, D_t, c_t, r_t) at every step. Then for every randomized perfect-recall gate ψ, with $W_t = \prod_{s \le t} \psi(D_s \mid h_s)/\mu(D_s \mid h_s) \le \varepsilon^{-t}$, $R(\psi) = \mathbb{E}_\mu[\sum_t W_t r_t]$ and $C(\psi) = \mathbb{E}_\mu[\sum_t W_t c_t]$; hence the achievable (R, C) set and $F_H(\delta)$ are identified.*

Proof. Write a logged trajectory as $\omega = (h_t, D_t, c_t, r_t)_{t \le H}$, where h_{t+1} appends the next observation to (h_t, D_t), and let $\omega_{<t}$ be its prefix before h_t. Let ν be a gate whose decision at step

t is drawn from $\nu(\cdot \mid h_t)$ with randomization independent of everything else. By Definition 5, the next state, observation, cost, and unsafe indicator depend on the past only through (X_t, D_t), and (c_t, r_t) are functions of (X_t, D_t, X_{t+1}). Let $G_t(\cdot \mid \omega_{<t}, h_t, D_t)$ be the conditional law of (c_t, r_t, h_{t+1}) given the logged prefix and (h_t, D_t). It is obtained by Bayes' rule from the initial law and the kernels, integrating over the posterior of the hidden states given $(\omega_{<t}, h_t, D_t)$; the factors $\nu(D_s \mid h_s)$, $s \le t$, that appear in the joint law of the hidden states and the logged prefix are functions of the logged prefix alone and cancel in that posterior, so G_t does not depend on ν. Therefore

$$\mathbb{P}_\nu(\omega) = \mathbb{P}(h_1) \prod_t \nu(D_t \mid h_t)\, G_t(c_t, r_t, h_{t+1} \mid \omega_{<t}, h_t, D_t).$$

Fix t. The pair (c_t, r_t) is a function of the logged prefix $\omega_{\le t}$ ending with (c_t, r_t), whose law under ν is the product above truncated at t. Since $\mu(d \mid h) \ge \varepsilon > 0$ at every reachable h, $\mathbb{P}_\psi(\omega_{\le t}) > 0$ implies $\mathbb{P}_\mu(\omega_{\le t}) > 0$, and the likelihood ratio is $\mathbb{P}_\psi(\omega_{\le t})/\mathbb{P}_\mu(\omega_{\le t}) = W_t$ because the common factors $\mathbb{P}(h_1)$ and G_s cancel. Hence $\mathbb{E}_\psi[r_t] = \mathbb{E}_\mu[W_t r_t]$ and $\mathbb{E}_\psi[c_t] = \mathbb{E}_\mu[W_t c_t]$; summing over t gives both identities. Each identity holds for every ψ, so the map $\psi \mapsto (R(\psi), C(\psi))$, its image, and $F_H(\delta) = \inf\{C(\psi) : R(\psi) \le \delta\}$ are functionals of the logging law. Each factor $\psi(D_s \mid h_s)/\mu(D_s \mid h_s)$ is at most $1/\varepsilon$, so $W_t \le \varepsilon^{-t}$. By the two identities, the average of $\sum_t W_t r_t$ (resp. $\sum_t W_t c_t$) over i.i.d. logged episodes is unbiased for $R(\psi)$ (resp. $C(\psi)$). The weight bound is attained, so the variance can grow exponentially in H: if $\varepsilon \le 1/2$, ψ is deterministic, μ gives ψ's choice probability exactly ε at every history, and $c_t = \mathbf{1}[t = H]$, then $\sum_t W_t c_t = W_H$ equals ε^{-H} with probability ε^H and 0 otherwise, so its variance is $\varepsilon^{-H} - 1$. Episode-level mixtures of such gates (as in Theorem 2) attain the corresponding convex combinations of identified (R, C) pairs, so they are identified as well. □

Theorem 3 (Distribution-free safety certificate)**.** *Under Assumption 1, calibrating the threshold by conformal risk control on violating examples to hold the per-step conditional miss risk* $r(\theta) = \mathbb{P}[\mathsf{allow} \mid \textit{violation}]$ *at* δ/H *yields* $\mathbb{P}[\textit{unsafe episode}] \le \delta$ *marginally over deployment episodes and the calibration draw, finite-sample and distribution-free under per-step exchangeability.*

Proof. Let V_t be the event that step t proposes a violation, with score s_t; the gate allows it iff $s_t < \hat\theta$, where the calibrated threshold $\hat\theta$ is a function of the calibration sample. An unsafe episode commits a violation at some step (a block commits no effect, and by Assumption 1 committing a compliant proposal keeps the effect prefix in Good(P)), so by the *union bound*, without conditioning on $\hat\theta$,

$$\begin{aligned}\mathbb{P}[\text{unsafe}] &\le \sum_{t \le H} \mathbb{P}[V_t]\, \mathbb{P}[s_t < \hat\theta \mid V_t] \\ &\le H \max_{t \le H} \mathbb{P}[s_t < \hat\theta \mid V_t],\end{aligned}$$

using $\mathbb{P}[V_t] \le 1$. Fix t and condition on V_t. By label-conditional per-step exchangeability (Assumption 1), the n violating calibration scores and s_t form an exchangeable $(n+1)$-tuple. The miss loss $\mathbf{1}[s < \theta]$ is bounded and nondecreasing in θ (raising the threshold blocks fewer actions); reparameterizing by $\lambda = -\theta$ makes it nonincreasing. Conformal risk control [6] at level δ/H on this tuple gives $\mathbb{P}[s_t < \hat\theta \mid V_t] \le \delta/H$, jointly over the calibration draw and the step-t proposal. (Equivalently, for $\hat\theta$ the k-th smallest calibration score with $k = \lfloor (n+1)\delta/H \rfloor$, exchangeability of ranks gives $\mathbb{P}[s_t < \hat\theta \mid V_t] \le k/(n+1)$, ties included; $k = 0$ means block-all.) The exchangeability required is *label-conditional* (Mondrian): since the miss is calibrated on the *violating* subpopulation, the violating calibration examples must be exchangeable with the violating deployment steps, as required by Assumption 1, not merely jointly exchangeable before conditioning on the label. Hence $\mathbb{P}[\text{unsafe}] \le H \cdot \delta/H = \delta$, which is the stated marginal guarantee. The argument never conditions on $\hat\theta$: without independence between deployment and calibration, $\hat\theta$ can carry information about the deployment law, so the conditional form $\mathbb{P}[\text{unsafe} \mid \hat\theta] \le H\, r(\hat\theta)$ with $\mathbb{E}[r(\hat\theta)] \le \delta/H$, for the population miss $r(\theta) = \mathbb{P}[\mathsf{allow} \mid \text{violation}]$, is justified when deployment steps are drawn independently of the calibration sample (as in Theorem 4) but can fail otherwise. Nothing in this argument bounds the finite-sample threshold's distance from the population ROC optimum. □

The certificate is marginal over violation types: if a type makes up a fraction q of violations, its own per-step miss is guaranteed only to be at most $\delta/(Hq)$, since $\mathbb{P}[\mathsf{allow} \mid \text{violation}, \text{type}] \le \mathbb{P}[\mathsf{allow} \mid \text{violation}]/q$. Calibrating separately per policy or per argument role, as concurrent work does for tool-call fields [7], refines this at the cost of more calibration violations per stratum.

Proposition S2 (Episode-level certificate, no H-penalty)**.** *If decisions alter neither later proposals nor their labels and scores (open loop) and a calibration set of whole episodes is exchangeable with deployment episodes, calibrating the threshold by conformal risk control on the monotone episode loss* $L(ep) = \mathbf{1}[\textit{some step allows a violation}]$ *(violations as in Assumption 1) to level* δ *gives* $\mathbb{P}[\textit{unsafe episode}] \le \delta$ *with no factor of* H*.*

Proof. The union-bound step above pays a factor H because it controls each per-step risk separately. Instead treat the whole episode as one calibration unit. The loss $L(\text{ep}; \theta) = \mathbf{1}[\exists t \le H :$ a violation at t scores below $\theta]$ is equivalently

$$L(\text{ep}; \theta) = \mathbf{1}\Big\{ \min_{t : Y_t = 1} s_t < \theta \Big\},$$

with the minimum over an empty set defined as $+\infty$. It is bounded in $[0, 1]$ and *nondecreasing* in θ (raising θ blocks weakly *fewer* actions, so it never turns a missed violation into a caught one; this uses the open-loop hypothesis, since under feedback the gated episode's unsafe indicator can be non-monotone in θ). Reparameterizing by $\lambda = -\theta$ makes it a non-increasing admissible loss for conformal risk control [6], which, on an exchangeable episode calibration set, returns $\hat\theta$ with $\mathbb{E}[L(\hat\theta)] \le \delta$. Since $\mathbb{P}[\text{unsafe episode} \mid \hat\theta] = \mathbb{E}[L(\text{ep}; \hat\theta) \mid \hat\theta]$, taking the calibration expectation gives $\mathbb{P}[\text{unsafe episode}] \le \delta$ directly. The price is that exchangeability is now required at the episode level (episodes i.i.d. or exchangeable), stronger than the per-step version but free of the H-fold tightening, which matters when H is large. □

Proposition S3 (General region is copula-bounded)**.** *For the conservative any-flag gate and fixed marginal error rates, Fréchet–Hoeffding bounds bracket the step-level miss for a specified violated predicate set and the false-block probability on compliant actions. At a fixed threshold vector the bounds are sharp for every* m*: each pair in the product of the two intervals is attained by some choice of class-conditional couplings. At* $m = 1$ *they equal the one-judge errors.*

Proof. Take the conservative gate that blocks when any judge flags. A violation with true hazard on predicate set S is committed iff no judge flags it, an event contained in $\bigcap_{i\in S} M_i$ (every relevant judge misses) with $\mathbb{P}[M_i] = \beta_i$, and equal to it when $S = \{1,\dots,m\}$, e.g., under the single predicate of Assumption 1 (for $|S| < m$ only the upper miss bound below applies); a compliant action is altered iff some judge false-alarms, $\bigcup_i E_i$ with $\mathbb{P}[E_i] = \alpha_i$. For fixed marginals the Fréchet–Hoeffding inequalities give

$$\max\left(0, \textstyle\sum_{i\in S}\beta_i - |S| + 1\right) \le \mathbb{P}\left[\bigcap_{i\in S} M_i\right] \le \min_{i\in S}\beta_i,$$

$$\max_i \alpha_i \le \mathbb{P}\left[\bigcup_i E_i\right] \le \min\left(1, \textstyle\sum_i \alpha_i\right),$$

which hold for every coupling of the errors. Each endpoint is attained by an explicit coupling, for every m. Realize the error indicators of a class as arcs of the unit circle under a uniform point. Nested arcs $M_i = [0,\beta_i)$ and $E_i = [0,\alpha_i)$ (the comonotone coupling) attain the intersection upper bound $\min_{i\in S}\beta_i$ and the union lower bound $\max_i \alpha_i$. Placing the complements M_i^c as consecutive arcs of lengths $1-\beta_i$, wrapping around the circle when necessary, gives $\mathbb{P}[\bigcap_{i\in S} M_i] = \max(0, \sum_{i\in S}\beta_i - |S| + 1)$; placing the E_i as consecutive arcs of lengths α_i gives $\mathbb{P}[\bigcup_i E_i] = \min(1, \sum_i \alpha_i)$. No m-variate countermonotone copula is needed (none exists for $m \ge 3$), because each quantity is the probability of a single event. Misses are defined on the violation class and false alarms on the compliant class, so the two couplings are chosen independently, and mixing two couplings within a class attains every intermediate value. The threshold-varying question is different: one coupling of the score vector must serve every threshold vector at once, and for $m \ge 3$ the pointwise extremes need not be simultaneously attainable (Supplement B.16, O1). An episode lift additionally requires specified temporal dependence; only under Cor. 1's product conditions may these step probabilities be inserted into its product formula. For $m = 1$ both bounds equal (α_1, β_1). □

Theorem 4 (Adversarial deployment bound)**.** *Under Assumption 1, with conformal calibration on benign data and a budget-ρ adversary acting at deployment, on steps drawn independently of the calibration sample and with the violating proposals generated under attack (before perturbation) satisfying Assumption 1, the calibrated gate satisfies $\mathbb{P}[\text{unsafe episode}] \le \min\{1, \delta + H\bar{\eta}(\rho)\}$ marginally over the calibration draw. Conditional on a fixed threshold, an attained-margin independent product construction realizes $1 - (1 - \min\{1, \beta(\theta) + \eta(\rho;\theta)\})^H$.*

Proof. Upper bound. Calibration is performed on the benign exchangeable sample; the representation adversary acts only at deployment, on steps drawn independently of the calibration sample, so that conditioning on $\hat{\theta}$ leaves each step's benign miss risk equal to $r(\hat{\theta})$. (Without independence, the per-step guarantee of Theorem 3 still holds marginally, because conformal risk control is itself a marginal guarantee; the margin argument below needs the conditional form.) By Def. 6, $\bar{\eta}(\rho)$ upper-bounds the per-step increase in the conditional miss risk *at every threshold*, so under any budget-ρ adversary the risk at the calibrated $\hat{\theta}$ is at most $r(\hat{\theta}) + \bar{\eta}(\rho)$. The union bound of the certificate proof, conditioning on $\hat{\theta}$, gives $\mathbb{P}[\text{unsafe} \mid \hat{\theta}] \le \sum_{t\le H}(r(\hat{\theta}) + \bar{\eta}(\rho)) = H\, r(\hat{\theta}) + H\bar{\eta}(\rho)$; taking the calibration expectation and using $\mathbb{E}[r(\hat{\theta})] \le \delta/H$ (conformal risk control) yields the marginal $\mathbb{P}[\text{unsafe episode}] \le \min\{1, \delta + H\bar{\eta}(\rho)\}$. *Sharpness construction.* Fix θ and suppose a representation adversary attains the defining supremum, so its conditional miss is $x = \min\{1, \beta(\theta) + \eta(\rho;\theta)\}$. On an all-violation product stream, choose its perturbation randomness independently across steps. The probability that no violation is committed is $(1-x)^H$, giving exactly $1-(1-x)^H$. If the supremum is not attained, the same risks are approached with $\eta - \varepsilon$. This demonstrates first-order sharpness of the union bound under a stated construction; it is not a lower bound for every adversary or feedback process. *Non-transfer.* Conformal validity requires the deployment steps to be exchangeable with the calibration sample; a budget-ρ adversary changes the input distribution, so the *attacked* per-step miss need not satisfy the δ/H bound (the benign $\mathbb{E}[r(\hat{\theta})] \le \delta/H$ still holds) and the bare δ bound is void unless η absorbs the shift. □

Proposition 5 (Distributional zero-miss price)**.** *For any attainable adversarial violation-score law Q_ρ, every score-measurable gate with zero Q_ρ-miss has utility loss*

$$U \ge 1 - \sup\{F_0(N) : Q_\rho(N) = 0\};$$

if $F_0 \ll Q_\rho$, this distributional zero-miss cost is one.

Proof. Let Q_ρ be any attainable score law induced on violations and F_0 the compliant score law. Fix a score-measurable gate with zero miss under Q_ρ. For a deterministic gate the blocked set B has $Q_\rho(B) = 1$; for a randomized gate the same argument applies to its block-probability function and yields the identical variational value. Its utility loss is the compliant mass it over-blocks, $U(g) = F_0(B)$, and over all zero-miss gates

$$\begin{aligned} U(g) &\ge \inf\{F_0(B) : Q_\rho(B) = 1\} \\ &= 1 - \sup\{F_0(N) : Q_\rho(N) = 0\}, \end{aligned}$$

using $F_0(B) = 1 - F_0(B^c)$ with B^c ranging over the Q_ρ-null sets. This is an *absolute-continuity* (Lebesgue-decomposition) bound, sharper and correct where a raw support-overlap bound is not: a zero-miss gate may leave unblocked only the part of the compliant law *singular* to Q_ρ. By the Lebesgue decomposition $F_0 = F_0^{\text{ac}} + F_0^{\text{sing}}$ relative to Q_ρ, $U(g) \ge F_0^{\text{ac}}(\mathbb{R}) = 1 - F_0^{\text{sing}}(\mathbb{R})$. Hence if $F_0 \ll Q_\rho$ (no compliant mass on any Q_ρ-null set) then $U(g) = 1$: block-everything is the only distributionally zero-miss score rule. This is not the universal streamwise soundness of Def. 4. We emphasize that $\eta(\rho;\theta) \ge 1 - \beta(\theta)$ is *not* sufficient for domination: it drives the adversarial miss to one at that threshold, but says nothing about whether F_0 is dominated by Q_ρ (F_0 may still place mass on Q_ρ-null sets, e.g., high-scoring compliant actions). What approaches $F_0 \ll Q_\rho$ is empirical: in the safe-washing runs the adversary drives Q_ρ onto the low scores where the LLM judges already place compliant actions, so the Q_ρ-absolutely-continuous compliant mass $1 - \sup\{F_0(N) : Q_\rho(N) = 0\}$ is large and the zero-miss gate's over-block U is near 1; we report the measured U rather than assume total absolute continuity. □

Corollary 3 (Robust calibration)**.** *If a uniform margin $\bar{\eta}(\rho) < \delta/H$ has been verified, calibrating on benign exchangeable violations to the per-step target $\delta/H - \bar{\eta}(\rho)$ yields $\mathbb{P}[\text{unsafe episode}] \le \delta$ (marginal over the calibration draw) under any budget-ρ representation adversary at deployment, under the conditions of Theorem 4. At the population optimum the frontier price is $\text{roc}_I(\delta/H - \bar{\eta}(\rho)) - \text{roc}_I(\delta/H)$; finite-sample calibration may pay additional conservatism (Proposition 5 prices distributional zero miss).*

Proof. Run the same split-conformal calibration but against the tightened per-step target $\tau' = \delta/H - \bar{\eta}(\rho) > 0$, so the benign per-step miss satisfies $\mathbb{E}[r(\hat{\theta})] \le \tau'$ over the calibration draw (the miss

at a given calibrated threshold can exceed τ'). By Def. 6 a budget-ρ adversary raises the per-step conditional miss at *any* threshold by at most $\bar{\eta}(\rho)$, to at most $r(\hat{\theta}) + \bar{\eta}(\rho)$, whose calibration expectation is at most $\tau' + \bar{\eta}(\rho) = \delta/H$. The union bound of the upper part, conditional on $\hat{\theta}$ and then averaged over the calibration draw, gives $\mathbb{P}[\text{unsafe}] \le \sum_{t \le H} \delta/H = \delta$, for *any* budget-$\rho$ adversary since the per-step bound is worst-case. At the population optimum, changing the miss target from δ/H to τ' changes false-block cost by $\mathrm{roc}_I(\tau') - \mathrm{roc}_I(\delta/H)$. A threshold selected from a finite calibration sample can lie above this envelope, so equality of its realized cost is not claimed. If the tightened target is non-positive, this procedure falls back to block-all; the separate domination condition $F_0 \ll Q_\rho$ is what forces unit cost for distributional zero-miss. □

A.4 T3: the decidability boundary

Proposition 6 (Specification analysis is undecidable)**.** *Enforcement is constructive once a nonempty policy is given as a $\mathcal{T}$-representable safety automaton (T1); static analysis of an arbitrary specification is not. There is a guard theory $\mathcal{T}^*$ (equality plus two global $\mathbb{N}$-counters with increment, guarded decrement, and zero-test) for which reachability of a designated bad state in a deterministic, trim register automaton is undecidable; consequently deciding whether a specified data-word policy is nontrivial (has a bad prefix, i.e., ever fires), the basic sanity check a policy DSL's linter would run, is undecidable.*

Proof. A two-counter Minsky machine M has counters $c_1, c_2 \in \mathbb{N}$ and a finite program of labeled instructions of three forms,

$$\ell : \ c_j{+}{+};\ \text{goto } \ell',$$
$$\ell : \ \text{if } c_j = 0 \text{ goto } \ell' \text{ else } c_j{-}{-};\ \text{goto } \ell'',$$

and a halt instruction. Whether M, started from $c_1 = c_2 = 0$ at its initial label, reaches its halt instruction is undecidable.

Build a deterministic register automaton $\mathcal{A}$ over $\mathcal{T}^*$ as follows. Its finite control is the set of M's labels plus an absorbing good sink G; it has two global $\mathbb{N}$-counters r_1, r_2 initialized to 0; and the alphabet has exactly two effect actions, the nullary tick and safe ($\perp$ leaves the state unchanged, A.1). On each tick, $\mathcal{A}$ executes the instruction at the current label: an increment instruction updates r_j += 1 and moves to ℓ'; a test-and-decrement instruction uses the $\mathcal{T}^*$ zero-test guard $r_j = 0$ to branch to ℓ', or otherwise updates r_j −= 1 and moves to ℓ''. From every nonbad machine label, safe moves to G, where both actions loop. These updates and guards are exactly the operations of $\mathcal{T}^*$, and $\mathcal{A}$ is deterministic because M is. Designate M's halt label as the absorbing bad state B.

Because M is deterministic and requires no input, the run of $\mathcal{A}$ on the infinite tick stream faithfully simulates M: after n ticks its configuration (label, r_1, r_2) equals M's configuration after n steps. Hence a finite tick-prefix reaches B iff M halts, so bad-state reachability in $\mathcal{A}$ is undecidable. The induced policy P has a bad prefix, i.e., is nontrivial (rules out at least one finite behavior), iff B is reachable iff M halts. Assuming w.l.o.g. that M's initial label is not its halt label (such machines halt trivially), P is always nonempty, since taking safe before B yields an infinite run in G, and every nonbad prefix has such a safe continuation, so $\mathcal{A}$ is trim. Enforcing this constructed $\mathcal{T}^*$-automaton is therefore still constructive (T1); what is undecidable is the static question of whether an arbitrary such specification is non-vacuous, so no DSL linter can decide it over $\mathcal{T}^*$. □

Trimness is undecidable for $\mathcal{T}^*$. Drop the safe action and make the halt label a non-bad dead end from which every action enters a fresh absorbing bad state. Every reachable non-halt configuration keeps a non-bad tick successor, so the automaton is trim (§3) iff the halt label is unreachable, i.e., iff M does not halt.

Theorem 5 (A decidable, expressive fragment)**.** *Over an infinite equality-data sort and a separate binary-encoded $\mathbb{N}$ amount sort, for the fragment $\mathcal{T}_0$ of finite, separable, disjunctive products of total equality-register components (E), uniform independent key-local monotone-counter components (K), and one finite monotone counter-vector component (V), bad-prefix reachability is* PSPACE*-complete, and* PSPACE*-hard already for one (E) component; without (E) components, with keys over the equality sort, and if every amount atom is a cap check, it is* NP*-complete (Proposition S5). Cap saturation is exact for (K,V), one-witness projection is exact for (K), and the finite abstraction is searched on the fly. For trim specifications (§3), online enforcement is polynomial per step in the specification, current-action encoding, and* log *of the number of stored keys; with saturated counters, bit-space is linear in stored data encodings plus per-key local abstract states.*

Proof. Exact saturation. For counter x, let C_x be the greatest constant against which x or $x + v$ is compared (or 0 if none) and define $\sigma_x(n) = \min\{n, C_x + 1\}$. Because every update is nonnegative, values above C_x never return below it. For each supplied amount field v, let C_v be the largest C_x among counters whose guards or updates use that field, guess the *single* category $\tau_v(v) = \min\{v, C_v + 1\}$, and derive every counter-specific category as $\min\{\tau_v(v), C_x + 1\}$. Thus all uses represent one concrete amount, rather than independent per-counter guesses. Every allowed atomic guard $x \bowtie c$ or $x + v \bowtie c$ has the same truth value for all concrete valuations with the same abstraction, and abstract addition $\sigma_x(\sigma_x(x) + \min\{\tau_v(v), C_x + 1\})$ exactly matches concrete addition followed by σ_x. Boolean combinations and deterministic transitions therefore preserve this equivalence. A (V) component with k counters has at most $|Q| \prod_x (C_x + 2)$ abstract configurations. One transition may update several counters, but each atomic guard still compares one counter (or that counter plus the supplied amount) with a constant.

One-witness projection. A (K) component has a local finite state and counter vector for each key, initialized identically; an action naming k reads and updates only the component at k, and the component rejects iff some local copy reaches its absorbing bad state. If a concrete run rejects, project the subsequence of actions naming the first rejecting key: locality makes this exactly a one-key rejecting run (actions naming no key update no local copy and are dropped). Conversely, rename the key in any one-key rejecting run to a fresh concrete key to obtain a global rejecting run. Thus bad-prefix reachability is equivalent to reachability in one saturated local copy, not merely over-approximated.

Equality and products. Because the equality-data sort is infinite, an (E) component has the standard finite equality-type abstraction for its registers and current data; its nonemptiness problem is decidable [3], and in polynomial space by the on-the-fly search below, since each equality type has a polynomial-size encoding. Nonemptiness of register automata is itself PSPACE-complete [8], so the (E) components add no complexity beyond that class. Every component is total over the common alphabet: guards partition the input valuations and any missing case is completed to its absorbing bad state. Separability excludes cross-component guards and assumptions. Hence a word rejecting in one component is executable by every other component and rejects the disjunctive product; conversely every product rejection has such a witness

component. One may therefore nondeterministically select a component and search its implicit abstract graph. A node stores finite control, equality type, and each saturated counter in binary; a successor guesses the finite action/equality type and one capped category per supplied amount field, shared across every use of that field. The graph can be exponential because constants are binary, but reachability uses polynomial space by an on-the-fly counter bounded by the number of abstract nodes. By NPSPACE=PSPACE, nontriviality is in PSPACE.

Hardness. Let $L \in$ PSPACE be decided by a deterministic Turing machine M with states Q and tape alphabet Γ in space $p(n)$; we assume M never leaves cells $1, \ldots, p(n)$. Given x, build in polynomial time one total deterministic (E) component $\mathcal{A}_x$ over a single unary action type, with registers z_γ ($\gamma \in \Gamma$) and $y_1, \ldots, y_{p(|x|)}$; their number is part of the input and, as in Demri and Lazić [8] but unlike the pairwise-distinct registers of Kaminski and Francez [3], two registers may hold equal data. The first $|\Gamma|$ actions store pairwise distinct data in the z_γ, and the next $p(|x|)$ actions store in y_i a datum equal to z_{γ_i}, where $\gamma_1 \cdots \gamma_{p(|x|)}$ is x padded with blanks. The control then holds M's state q and head position h. Let γ be the symbol with $y_h = z_\gamma$ (an equality guard) and $(q', \gamma', \pm 1)$ the move of M on (q, γ). An action whose datum equals $z_{\gamma'}$ stores it in y_h and moves the control to $(q', h \pm 1)$. Every other datum, i.e., a repeated value during setup or a wrong symbol, leads to an absorbing good sink, as do M's rejecting states, while M's accepting state is the bad state. Guards thus partition every input, so $\mathcal{A}_x$ is total, deterministic given the datum's equality type, trim, and nonempty. It has $|\Gamma| + p(|x|)$ registers and $O(|\Gamma| + p(|x|) + |Q|\, p(|x|))$ control states, and its bad state is reachable iff M accepts x. Bad-prefix reachability is therefore PSPACE-hard already for a single (E) component (cf. the PSPACE-completeness of register-automaton nonemptiness [8]), and $\mathcal{T}_0$ nontriviality is PSPACE-complete.

Runtime cost. The T1 gate stores $\sigma_x(x)$, not an unbounded concrete counter, and maintains a balanced dictionary from each encoded key to its local finite state and saturated counter vector. On an action of bit length L, it reads each supplied amount once, computes its shared capped category, looks up the touched key using $O(\log n)$ encoded-key comparisons, evaluates the guard partition, and performs saturated updates. Time is polynomial in L and the specification, with the stated $\log n$ dictionary factor. If a saturated local state needs s bits, total (K) storage is $O(\sum_{k \text{ seen}} |k| + ns)$ bits, plus the bounded E-register encodings and fixed V abstract state. This is the claimed linear dependence on stored encodings, not a unit-cost claim for arbitrary-precision values or hashing.

Expressiveness. We exhibit each policy as a deterministic $\mathcal{T}_0$ automaton with absorbing bad state. *Authenticate-before-access*: a control flag *auth* and an equality register *prin* storing the authenticated principal; the authenticate(p) action sets *auth* and *prin* := p; an access by requester q takes the bad transition unless *auth* holds and q = *prin* (an equality guard). *Per-recipient spend cap*: one monotone accumulator s_k per recipient key k (stored in the balanced key dictionary); pay(k, v) with a nonnegative integer amount v takes the bad transition if $s_k + v > c$ and otherwise updates s_k += v, an add-nonnegative-amount update and a compare-to-constant guard, both in $\mathcal{T}_0$. Static decidability applies the exact-saturation abstraction (exact up to the cap c, then absorbing at c+1) to the single representative key, since distinct keys evolve independently, so "some key exceeds its cap" is reachable iff it is reachable for one key. P2 uses the same monitor with composite key (k, w) when a trusted clock adapter attaches the day bucket w. *Rate limit r per supplied bucket*: key a monotone accumulator by the action's composite (k, w) field; its $(r+1)$-th action takes the bad transition. The automaton enforces this policy for every supplied bucket sequence and assumes no ordering between bucket identifiers. Interpreting w as a wall-clock window additionally requires a trusted adapter to attach $w = \lfloor t/W \rfloor$; that adapter is outside $\mathcal{T}_0$, rather than an unstated trace assumption. No decrement or reset is needed because a new bucket is a fresh key. *No-send-after-revoke*: a boolean control flag set by revoke; send takes the bad transition once the flag is set. *Fixed-vector global quotas*: a (V) component keeps one counter per resource class, may test a proposed nonnegative amount against the corresponding remaining cap, and adds the tested amount to each tested coordinate (possibly also to other coordinates). This permits coupling within the declared finite vector but never allocates or compares counters for unboundedly many data keys. □

Proposition S4 (Policy inclusion and redundancy)**.** *Let $\mathcal{A}, \mathcal{B}$ be trim $\mathcal{T}_0$ specifications of policies $P_\mathcal{A}, P_\mathcal{B}$ over a common alphabet, where $\mathcal{A}$ has no (K) component and registers, keys and equality guards range over the equality sort (amount fields are read only by counter guards and updates). Deciding $P_\mathcal{A} \subseteq P_\mathcal{B}$, i.e., whether rule $\mathcal{B}$ is redundant given $\mathcal{A}$ ($P_\mathcal{A} \cap P_\mathcal{B} = P_\mathcal{A}$), is PSPACE-complete, and PSPACE-hard already when $\mathcal{A}$ allows every action and $\mathcal{B}$ is one (E) component. Inclusion for $\mathcal{A}$ with a (K) component is not covered.*

Proof. Reduction. For any P, $u \in \mathrm{Good}(P)$ iff $uy \in P$ for some y; as $P_\mathcal{B}$ is safety, $P_\mathcal{A} \subseteq P_\mathcal{B}$ iff $\mathrm{Good}(P_\mathcal{A}) \subseteq \mathrm{Good}(P_\mathcal{B})$ iff $P_\mathcal{A} \cap P_\mathcal{B} = P_\mathcal{A}$, i.e., adding rule $\mathcal{B}$ changes no decision of the T1 gate. By trimness, $\mathrm{Good}(P_\mathcal{A})$ ($\mathrm{Good}(P_\mathcal{B})$) is the set of words on which no component of $\mathcal{A}$ ($\mathcal{B}$) enters a bad state, and $\mathcal{B}$ rejects iff some component $\mathcal{B}_j$ does. So inclusion fails iff, for some j, some word keeps every component of $\mathcal{A}$ non-bad while $\mathcal{B}_j$ enters a bad state. By the typing hypothesis, equality types and amount categories never constrain the same value. *Product search.* Fix j. If $\mathcal{B}_j$ is a (K) component, add an analysis-only register tuple r^*, never assigned, whose initial equality type relative to the initial register contents is guessed, and track only the local copy of key r^*: it starts in the initial local state and is updated exactly by the actions whose key equals r^*. A node stores the control states of all components of $\mathcal{A}$ and of $\mathcal{B}_j$ (or of the r^* copy), the joint equality type of all their registers and r^*, and every counter, saturated as in the proof of Theorem 5; each amount field v gets one shared category $\min\{v, C_v + 1\}$, with C_v the largest C_x over counters x of $\mathcal{A}$ or $\mathcal{B}_j$ that use v. A successor guesses an action type, the equality type of its data relative to the registers and r^*, and the amount categories; it is discarded if a component of $\mathcal{A}$ enters a bad state, and accepting if $\mathcal{B}_j$ (its r^* copy) does. *Exactness.* Saturation preserves the truth value of every atom $x \bowtie c$ and $x + v \bowtie c$ of either specification and commutes with the nonnegative updates (proof of Theorem 5). Over the infinite equality sort, fresh values realize every consistent type sequence, and all register, key and r^* tests are equalities. If a word keeps $\mathcal{A}$ non-bad while the copy of $\mathcal{B}_j$ at key k^* rejects, then $r^* := k^*$, with its actual initial type, maps its run to an accepting path; conversely, an accepting path realizes to a word on which every component of $\mathcal{A}$ evolves as tracked and the copy at the realized r^* rejects. No action is deleted: actions naming other keys stay as actions with key $\neq r^*$ and update $\mathcal{A}$ exactly, so $\mathcal{A}$'s guards need not be monotone. Nodes have polynomial size, so an on-the-fly search over all j, with a step counter bounded by the number of nodes, puts non-inclusion in NPSPACE; by Savitch's theorem and

closure under complement, inclusion is in PSPACE. *Hardness.* Let $\mathcal{A}$ be the one-state, register-free (E) component that allows every action; it is trim and $P_{\mathcal{A}}$ contains every trace, so $P_{\mathcal{A}} \subseteq P_{\mathcal{B}}$ iff $\mathcal{B}$ has no reachable bad state. Bad-prefix reachability is PSPACE-hard already for one trim (E) component (proof of Theorem 5), and PSPACE is closed under complement, so inclusion is PSPACE-hard. In §7.6, $\mathcal{A}$ is the session limit, a (V) component, and $\mathcal{B}$ is the per-recipient cap, a (K) component; that check is an instance. □

Proposition S5 (Caps, quotas, and revocation: an NP-complete core)**.** *Call an amount atom $x + v \bowtie c$ in the guard of a transition case (a cell of the guard partition for one control state and action type, with its target and updates) a* cap check *if that case's update of x contains the summand v (so x grows by at least v) or the case enters the absorbing bad state; completed missing cases enter the bad state, so their atoms are cap checks. Consider $\mathcal{T}_0$ specifications without (E) components whose keys range over the equality sort (amount fields are read only by counter guards and updates, as in Proposition S4) and in which every amount atom is a cap check. For them, bad-prefix reachability (nontriviality) is NP-complete; it is NP-hard already for one (V) component with unit increments and the constant* 1*. When neither $\mathcal{A}$ nor $\mathcal{B}$ has (E) components, $\mathcal{A}$ has no (K) component, and every amount atom is a cap check, deciding $P_{\mathcal{A}} \subseteq P_{\mathcal{B}}$ for trim $\mathcal{A}, \mathcal{B}$ (Proposition S4) is coNP-complete. All these rules of Theorem 5 are of this form: per-recipient caps, bucket quotas, rate limits, no-send-after-revoke (its flag a (V) counter ϱ with* revoke : ϱ += 1 *and* send *bad if $\varrho \geq 1$, or a control bit of the (V) component), and fixed-vector quotas in which every case that tests a coordinate against a supplied amount v either enters the bad state or adds v to that coordinate. So are P2, P3, and the main-paper §7.6 redundancy check. Several global rules (a session limit, a revocation flag, fixed-vector quotas) form one (V) component whose control is the product of theirs; the bounds are in the size of that merged component, set-once flags encoded as counters tested against* 1 *keep it polynomial, and nontriviality alone needs no merging, since a disjunctive product is checked component by component. The cap-check hypothesis cannot be dropped: in one register-free (V) component, cells $x_h + v = K \wedge y_h + v = K - b$ test $x_h - y_h = b$, so each counter pair stores one rewritable bit, and with $K = 2^{O(p(n))}$ in binary such a component simulates a polynomial-space Turing machine; without (E) components but with arbitrary amount atoms, bad-prefix reachability therefore stays PSPACE-complete.*

Proof. Reduction to one counter component. By the product argument and the one-witness projection in the proof of Theorem 5, it suffices to decide whether the bad state of one component without registers and keys (a (V) component, or one local copy of a (K) component) is reachable. Such a component has finite control Q and monotone counters X. For a counter x, let $c_1^x < \cdots < c_{m_x}^x$ be the constants of its atoms $x \bowtie c$ and $x + v \bowtie c$, and call the m_x singletons $\{c_i^x\}$ and the nonempty gaps between consecutive constants (below the first, above the last), at most $2m_x + 1$ sets in all, the *regions* of x. An atom $x \bowtie c$ has one truth value on each region of x, and an atom $x + v \bowtie c$ has one truth value on each region containing $x + v$. A *region vector* r assigns each counter a region.

Phases. Updates are nonnegative, so along a run every counter visits its regions in increasing order, and the region vector changes at most $R = \sum_x 2m_x$ times; R is polynomial in the specification although constants are binary. A run that reaches the bad state therefore splits into at most $R + 1$ *phases* of constant region vector, joined by at most R *boundary* steps (each changes some region) and followed by the final step into the bad state. Consider a step inside a phase with vector r that takes case t. It does not enter the bad state, its pre- and post-values lie in r, and for every amount atom $x + v \bowtie c$ of its guard, a cap check, t adds at least v to x, so $x + v$ lies between the pre- and post-value of x and hence in $r(x)$. Every atom of t's guard thus has its value under r, so $t \in E_r$, the set of non-bad cases whose guard is true when each atom $x \bowtie c$ is evaluated at $r(x)$ and each $x + v \bowtie c$ with $x + v \in r(x)$; E_r is computable in polynomial time from r. Phase i ends in the source state of boundary case i, whose target starts phase $i + 1$; the first phase starts in the initial state; empty phases are allowed; and cases that cannot stay in r are forced to occur zero times by the end-value bounds. Conversely, take any path of cases in E_r from control state q to q', with any nonnegative amounts, started from counter values in r: if the values after the path are still in r, then, as counters are monotone and regions are intervals, every intermediate value is in r, every guard on the path holds, and the path is executable.

One existential Presburger formula. A phase is thus described by its start and end control states, the Parikh vector $\varphi \in \mathbb{N}^{E_r}$ of a path between them, and, for each case t and amount field f of t, the total $A_{t,f} \geq 0$ supplied to f over t's occurrences ($A_{t,f} = 0$ if $\varphi(t) = 0$, and any value otherwise, since one occurrence may carry it all). End values are linear: $x^{\text{end}} = x^{\text{start}} + \sum_t \varphi(t)\, k_{t,x} + \sum_{t,f:\, t \text{ adds } f \text{ to } x} A_{t,f}$, with $k_{t,x}$ the fixed increment. That φ is the Parikh vector of a path from q to q' in the graph E_r (flow conservation and connectivity of its support) has a polynomial-size existential Presburger definition [9]. Guess the number of phases, their region vectors, start and end control states, the boundary cases, and the final case into the bad state; then require the phase constraints, membership of each phase's start and end values in its regions (interval bounds), the boundary and final updates with explicit amount variables, the boundary and final guards as Boolean combinations of linear atoms in those explicit values, and initial values 0. By the two directions above, this polynomial-size existential Presburger formula is satisfiable iff the bad state is reachable, and satisfiability of such formulas is in NP [10]. (Equivalently, with the guesses encoded by Boolean selector variables, reachability reduces to a single quantifier-free linear-integer-arithmetic query of the kind SMT solvers accept.) For inclusion, apply the same encoding to the product search in the proof of Proposition S4. As $\mathcal{T}_0$ has at most one (V) component and $\mathcal{A}$ has no (E) or (K) component, $\mathcal{A}$ is a single (V) component (or empty), so for each j the product is one register-free counter component with control $Q_{\mathcal{A}} \times Q_{\mathcal{B}_j}$ and counters $X_{\mathcal{A}} \uplus X_{\mathcal{B}_j}$. If $\mathcal{B}_j$ is a (K) rule tracked at r^*, split each keyed action type into a copy naming r^*, where both components take a case, and a copy not naming it (or a keyless type), where $\mathcal{A}$ takes a case and the tracked copy is unchanged. Product cases are pairs of cases, so there are polynomially many, and amounts are shared within a step. Delete every product case in which $\mathcal{A}$ enters its bad state, including at the final step; the accepting cases are those in which the tracked copy enters its bad state. Every amount atom of a remaining non-accepting case comes from a case of $\mathcal{A}$ or $\mathcal{B}_j$ that does not enter its bad state, so it is a cap check; hence the phase argument applies, non-inclusion is in NP, and inclusion is in coNP. The restriction to one (V) component matters: if $\mathcal{A}$ could be a product of several register-free control components, non-inclusion would encode nonemptiness of an intersection of finite automata and be PSPACE-hard.

Hardness. Reduce CNF satisfiability with variables $y_1, \ldots, y_n$

and clauses $C_1, \ldots, C_m$ to one (V) component with counters x_i^T, x_i^F, unit increments, and atoms comparing a counter with 1. In control state S, actions setT_i and setF_i increment x_i^T and x_i^F, respectively, and loop, and go moves to K_1. In K_j, the action $\mathsf{pick}_{j,\lambda}$ for a literal λ of C_j moves to K_{j+1} if the counter of λ (x_i^T for y_i, x_i^F for $\neg y_i$) is at least 1, and to an absorbing good sink G otherwise. In L_i (with $L_1 = K_{m+1}$), cons_i moves to G if $x_i^T \geq 1 \wedge x_i^F \geq 1$ and to L_{i+1} otherwise, and in L_{n+1} the action fire enters the bad state. Every other pair of control state and action moves to G, which loops on every action, so the guards partition every input and the component is total, deterministic, and trim. A word reaches the bad state iff the counters set before go define a consistent partial assignment that satisfies every clause, i.e., iff the formula is satisfiable; the construction is polynomial (it also works as one (K) component in which every action type carries the key field; a witness names a single key). For inclusion, let $\mathcal{A}$ be the one-state (V) component with an empty counter vector whose single case loops on every action; it is total, deterministic, trim, and has no (E) or (K) component. As $P_{\mathcal{A}}$ contains every trace and $\mathcal{B}$ is trim, $P_{\mathcal{A}} \subseteq P_{\mathcal{B}}$ iff $\mathcal{B}$ has no reachable bad state, so inclusion is coNP-hard. □

SUPPLEMENT B
EXTENDED EXPERIMENTS

B.1 Setup and reproducibility

Main-paper Figure 1 shows the enforcement model, and Supplement D (Table S13) maps representative systems to our regimes. All experiments are inference-only (no training). The core judges are open-weights models run locally on one workstation (or a free API for the 70B), spanning three open families and a 140× size range: `qwen2.5` (0.5B, 3B, 7B, 32B), `llama3.1` (8B) and `llama-3.3` (70B), and `gemma2` (9B). To these we add *two closed-source* frontier judges from different vendors (`gpt-4o-mini` from OpenAI and `command-a-03-2025` from Cohere), `Llama-Guard-3` (8B) as a deployed guardrail, and a battery of current-generation frontier judges (B.6) served through hosted inference APIs. The pipeline consumes real-valued scores, so any scorer drops in. Each score-judge rates an action with the single prompt reproduced verbatim in Supplement C, parsed to a risk score $s \in [0, 1]$ (decoding settings below). The gate blocks iff $s \geq \theta$. Calibration is split-conformal: for a target δ we set the per-step conditional-miss target $\tau = \delta/H$ and choose $\hat{\theta}$ as the $\lfloor (n+1)\tau \rfloor$-th smallest score among n violating calibration examples ($\hat{\theta} = -\infty$ if that index is 0, i.e., block everything). We flag one small-sample effect: when $\delta/H < 1/(n+1)$ the index is 0 and the certificate is met *trivially* by blocking all actions ($U = 1$). This is why the $\delta = .05$ row of Table S2 shows miss 0 at $U = 1$ on the small curated set (fewer than 19 calibration violations); it is an inherent finite-sample limit of split conformal, and certifying $\delta < 1/(n+1)$ needs more violation data. Legacy descriptive quantities below average 400 stratified calibration/test splits; their error bars are one standard deviation across splits, not familywise intervals. The random seed is fixed (20260704). Four datasets are used: a curated set of 60 labeled tool-call actions spanning the four touchstone policies plus prompt-injection and destructive cases; the 132 tool-use tasks of AgentDojo [11] (its 97 user tasks as compliant actions, its 35 injection goals as violations, from the banking/slack/travel/workspace suites); InjecAgent and AgentHarm (B.6). The public artifact contains code, redistributable data, the exact prompt, cached scores, and a one-command cache-only reproduction path.

Compute, versions, and parsing. Open judges are served locally with `ollama` (CUDA backend) on a workstation with two 24 GB consumer GPUs, using ollama's default GGUF quantization (typically `Q4_K_M`); the 70B and all closed/frontier judges are served through hosted inference APIs, at temperature 0 except the seven judges without temperature control (below). AgentDojo is pinned to benchmark version `v1.2.1` (Python package `agentdojo` 0.1.35). A judge's text output is parsed by clamping the first integer to $[0, 100]$ and dividing by 100; a provider content-filter block is scored 1.0 (a flagged violation). In the older caches (26 of the 33 runs), any reply without an integer, including a textual refusal, and an API error after retries were stored as .5; only the seven judges without temperature control were scored with refusals (an explicit refusal stop reason) at 1.0. Scores of 1.0 fall mostly on violations (Grok-4: 14/35 violations versus 3/97 compliant items), so this can *inflate* rather than depress the two content-filtered runs' AUC: the same Mistral-Large-3 model scores .834 through the filtered endpoint versus .819 unfiltered, so these numbers are not conservative. Local inference is a few GPU-hours. The curated probe contains serialized tool calls, but the static AgentDojo diagnostic contains *instruction strings*: 97 user-task prompts and 35 injection goals, without the pre-call execution history or tool output. It therefore estimates discrimination for that declared information variable only. More context changes the statistical experiment and may help or hurt a fixed scoring prompt; Blackwell refinement guarantees only that the *optimal* test with richer information cannot have a worse ROC.

Judges and scoring rule: Every AgentDojo analysis uses the same 33 judge runs (32 distinct models; Mistral-Large-3 is served by two providers) on the same ordered 132 items. All judges receive the same 0–100 prompt. Judges whose APIs accept a temperature decode at temperature 0; seven hosted judges are served without temperature control (Claude Opus 5.5, Fable 5.1, Sonnet 5.5, and Haiku 5.5; GPT-6.1 Sol, GPT-6.1 Terra, and GPT-5.5), so each item is scored in three independent runs, run 1 is reported, and runs 2–3 serve as a robustness check (B.19). Content-filter blocks score 1.0 for every run, and refusals signaled by the API score 1.0 (fail-closed, as a gate must treat a non-answer) for the seven judges without temperature control; elsewhere a reply without an integer, including a textual refusal, scores .5. Caches that store only scores cannot separate a stored .5 from a genuine answer of 50; the exact-.5 counts per judge are an upper bound on what the rule could change (Llama-3.1-8B 87 of 132, Qwen-2.5-32B 21, Nova-Pro 16, Step-3.7-Flash 15, Kimi-K2.5 9, Command A 7, at most 5 for every other judge; `unify_crossfit.py`). Model identifiers and query dates are listed in the artifact's model registry.

B.2 Synthetic validation

The synthetic judge is binormal (compliant score $\mathcal{N}(0, 1)$, violating score $\mathcal{N}(\mu, 1)$, $\mu = 2.5$, horizon $H = 20$), giving an analytic ROC against which the Monte-Carlo simulation is checked. **Q1, certificate:** calibrating to δ yields realized episode-unsafe rate .0098, .0194, .0486, .0951, .182 at $\delta = .01, .02, .05, .10, .20$ (each $\leq \delta$, and tight). **Q1, frontier:** sweeping the threshold, the empirical operating points match the analytic frontier $U^\star(\delta) = \mathrm{roc}(R_H^{-1}(\delta))$ to within 2.7 standard deviations of Monte-Carlo error over 40 thresholds (Fig. S1).

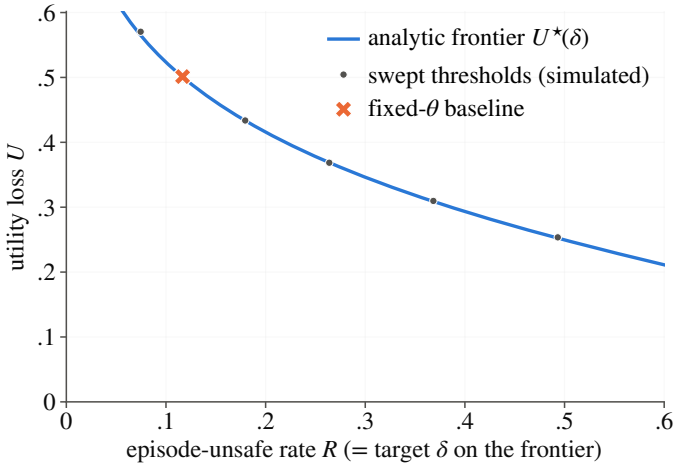


Fig. S1. Q1 (frontier): swept thresholds land on the predicted Pareto frontier $U^\star(\delta)$ of main-paper Corollary 1; a fixed threshold that ignores δ (cross) also lies on the frontier, but at a single risk level ($R \approx .12$), so it misses every tighter target. Only the swept thresholds with $R, U \leq .6$ are shown. Synthetic judge, exact ROC.

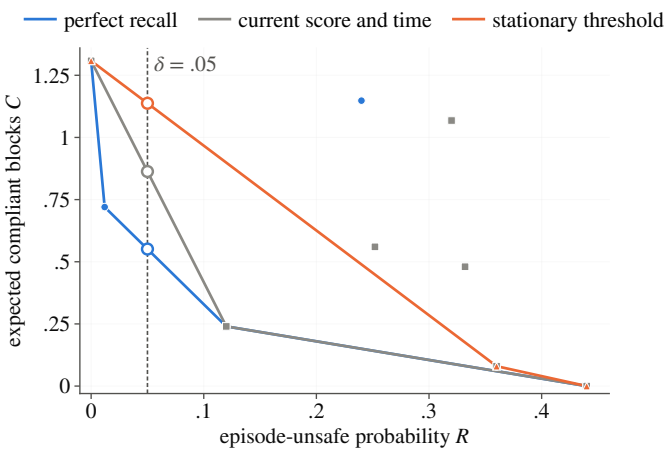


Fig. S2. Exact occupancy frontier of the specified retry model. Points are deterministic policies and segments are episode-level mixtures. Restricting the gate to current-score/time or one stationary threshold raises the cost at $\delta = .05$.

Controlled finite-model instantiation: We also instantiate the endogenous finite-model frontier (main-paper Theorem 2) in an auditable two-step retry environment. Initially the score is high (.8) with probability .4 and low (.2) with probability .6, with violation posteriors .8 and .2. Allowing terminates; blocking produces a second proposal. Its current score is .2 in both branches, but its posterior is .6 after a blocked high score and .02 after a blocked low score. Thus only perfect recall reveals the decision-dependent retry state. Cost is the expected number of compliant proposals blocked and risk is episode-unsafe probability.

We enumerate all $2^4 = 16$ deterministic perfect-recall policies and take their exact convex hull (episode-level randomization; Fig. S2). At $\delta = .05$, the optimum costs .551, versus .863 for policies using only current score and time and 1.137 for stationary score thresholds. The static "block high" threshold has $(R, C) = (.36, .08)$ and misses the constraint. Hence even this minimal model has a strict history-value gap; the class comparisons use episode-level mixtures, and with fresh per-step randomization the stationary class reaches about 1.074 (grid search), still far above .551. `controlled_model.py` in the artifact reproduces all four numbers in exact rational arithmetic (124/225, 863/1000, 10237/9000, and (9/25, 2/25)); the perfect-recall optimum mixes, once per episode, the policy that blocks everything except a retry after a low score (weight 35/54, $(R, C) = (.012, .72)$) with the policy that blocks a high first score and its retry and allows a low first score (weight 19/54, $(R, C) = (.12, .24)$). *Exploratory logging (Proposition 4).* On the two-system construction of Proposition 3 ($q = .3$, $p = .5$), we evaluate the gate that blocks step one and allows step two, whose risk is 0 in system A and $p = .5$ in B. Ungated logs have the same law in both systems, so no estimator separates them. With a logging gate that blocks each step independently with probability ε, the per-decision importance-sampling estimate is unbiased in both systems (mean .000 for A and .497–.505 for B over 2,000 replications), and its RMSE for B falls from .330, .174, .131 at $n = 100$ logged episodes to .032, .016, .013 at $n = 10^4$ for $\varepsilon = .05, .20, .50$; small ε costs more data, as the ε^{-H} weight bound predicts (`prop_explore_sim.py`).

TABLE S2
Curated descriptive probe (judges of AUC .53, .92, .98, .998): mean realized miss and per-action false-block loss over 400 splits. Block-all points are vacuous; no pairwise equivalence is inferred.

	0.5B		3B		7B		32B	
δ	miss	U	miss	U	miss	U	miss	U
.05	.000	1.00	.000	1.00	.000	1.00	.000	1.00
.10	.000	1.00	.070	.59	.065	.03	.038	.04
.20	.000	1.00	.117	.16	.104	.03	.036	.00
.30	.000	1.00	.149	.09	.116	.03	.036	.00

B.3 Real judges on the curated set

Table S2 reports realized miss (safety) and utility loss U at each target δ for four judges ($H = 1$, per-action) in the original 400-split descriptive probe. Mean observed miss is at or below target, and the utility cost of a fixed safety level broadly falls as judge quality (AUC) rises: at $\delta = .10$ the utility loss falls from 1.00 (the near-random 0.5B judge, which is safe only by blocking everything) to .59 (3B) to .03 (7B), consistent with the frontier (Fig. S3 shows the 3B sweep). A 32B judge reaches AUC .998 and $U = .04$; this probe does not support an equivalence claim between the 7B and 32B points. Scored with the same prompt and decoding (Supplement C), Claude Opus 5.5, Sonnet 5.5, and Haiku 5.5 and GPT-5.5 all reach AUC 1.000 on this probe (4 refusals, scored 1; `curated_frontier2026.py`), so it no longer separates current judges; the static, adversarial, and closed-loop results rest on AgentDojo (Supplement E).

Q3 (adversarial). On the 3B judge, calibrating to $\delta = .10$ on all 27 benign violations gives in-sample benign miss .037 (1/27; the held-out mean in Table S2 is .070); injection-style safe-washing of the violations raises the miss to .889 under the strongest tested template. This is a finite probe of representation sensitivity, not a universal worst-case or soundness result; the common-population AgentDojo analysis in B.9 is primary.

B.4 AgentDojo benchmark

The calibration diagnostic uses 500 label-balanced two-fold splits (seed 0) for all 33 judge runs (`unify_crossfit.py`): each

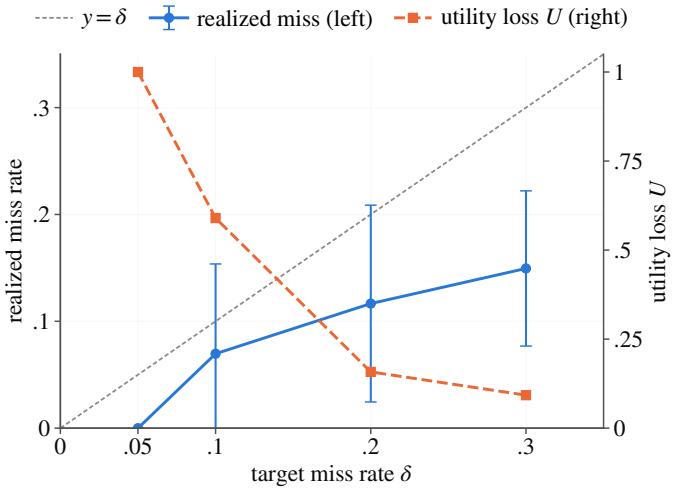


Fig. S3. Q1 on the real `qwen2.5:3b` judge: the descriptive cross-split mean realized miss stays below the target δ (left axis; error bars are one standard deviation over 400 calibration/test splits) while the utility loss it costs (right axis) falls as δ relaxes.

split calibrates the conformal threshold on one fold's violations and measures miss on the other fold's violations and false block on its compliant items. This is a finite-dataset compatibility check; the conformal theorem's certificate additionally requires deployment exchangeability, and we report cross-fitted point estimates only (no simultaneous bounds are computed).

At $\delta = .05$, every judge's rule is block-all: a half-fold holds 17–18 violations, fewer than $1/\delta - 1 = 19$, so the conformal index is 0 (miss 0, false block 1). At $\delta = .10$ the index is 1 in every half-fold, so $\hat{\theta}$ is the fold's lowest violating score; across judges, cross-fitted miss is 0–.053 and false block .641–1.000. At $\delta = .20$, miss is .019–.164 and false block .183–.964. For Gemma-2-9B the operating points are (.012, .779) at $\delta = .10$ and (.144, .649) at $\delta = .20$, the same splits as main-paper Fig. 2a. Block-all is reported as vacuous, and a point estimate at or below target is never called a certificate.

The primary rewrite result is separate from calibration: it fixes $\theta = .5$, uses every one of the 35 violations, and reports simultaneous suite-bootstrap intervals over every judge with clean and suffix-rewritten caches (B.9). It estimates a common-population representation effect at one operating point, not certificate failure at every threshold.

Counterfactual source-role diagnostic. To isolate authorization provenance without propagating episode outcomes, we take the culminating call from every AgentDojo user or injection task with a nonempty ground-truth plan: 97 user tasks and 26 injection tasks. Each of the 123 source tasks yields a pair with byte-identical call, arguments, instruction text, and control text. In the authorized member, the source instruction is the trusted user request and the control is untrusted output; in the unauthorized member, those two source roles are swapped. Under the declared policy that only trusted user input grants authority, labels are exact for this counterfactual. They are not natural-trace safety annotations.

The primary statistic is paired concordance $\Pr(s_{\text{unauth}} > s_{\text{auth}}) + \frac{1}{2}\Pr(s_{\text{unauth}} = s_{\text{auth}})$. Treating source tasks as exchangeable draws from a conceptual task population, we test the conditional sign null and compute a 20,000-draw task-pair bootstrap interval. For the fixed 123-task AgentDojo set, concordance and win/tie/loss counts are exact descriptive quantities. Because the action is identical within each pair, action-only concordance is exactly .5 by construction. Context-rich concordance is .561 for the 3B primary (15/108/0 wins/ties/losses; bootstrap 95% CI [.533, .589]), .967 for 7B-model corroboration (95% CI [.939, .988]; 116/6/1), and .500 for the 0.5B negative control (0/123/0). Holm-adjusted two-sided exact p-values across the three models are 1.22×10^{-4}, 4.26×10^{-33}, and 1. Thus the larger judges respond directionally to source role under this policy, but the experiment does not estimate natural-trace accuracy, real-world authorization, or deployment safety.

The cache is bound to dataset, prompt, and model hashes and stores raw model responses and parser provenance; it is not part of the public artifact (Supplement C).

B.5 Closed-loop agent evaluation ($H > 1$)

The preceding AgentDojo diagnostic scores instructions statically; here we run the *real* AgentDojo agent loop across all four suites. A local `qwen2.5:7b` agent drives the banking, slack, travel, and workspace suites under the `important_instructions` indirect prompt-injection attack; a gate element placed before tool execution scores each proposed tool call with the judge and drops those at/above θ, over multi-step trajectories. The judge decodes at temperature 0 against a fixed endpoint. The agent is called through AgentDojo's `OpenAILLM` element with its default temperature 0 (`run_loop.py` sets none), which agentdojo 0.1.35 drops from the request (it passes `temperature or NOT_GIVEN`), so the agent most likely sampled at the endpoint's default temperature (not recorded); this is consistent with the run-to-run shifts in Supplement E.5. Conditions are paired on task and injection goal. Table S3 reports end-to-end task-utility and attack success over 949 episodes per condition (144 banking, 105 slack, 140 travel, 560 workspace), with Wilson 95% intervals; these intervals and the per-suite Fisher tests below treat episodes as independent although they share user tasks and injection goals, so they are approximate and likely somewhat narrow.

Trace-derived action-level go/no-go. We instrumented the interceptor to log each proposed call, the preceding messages and tool outputs, action-only and context-rich scores, and the gate decision. A local rerun reproduced such traces. AgentDojo's exposed checks in this pipeline, however, label *episode* attack success rather than mapping a security label to every intercepted call. Copying the episode label onto all calls would mislabel prerequisite and benign calls, so the strict analysis rejects this as primary action-level evidence. We report no action-only-versus-context-rich AUC from those generated traces; the policy-grounded B.4 diagnostic and end-to-end Q4 outcomes use different, explicitly valid units.

Injectability is heterogeneous, so we test the gate stratified by suite. The local agent is far more injectable on slack (25/105 baseline attack success, .238) than on banking (12/144, .083) or travel (8/140, .057), and is *not* injectable at all on workspace (0/560: it completes many workspace tasks but the attack never lands). Pooling such suites into one 2×2 table would let the low- and zero-injectability suites mask the effect (a Simpson's-paradox risk), so we test each gate against its own no-gate baseline with a Cochran–Mantel–Haenszel test stratified by suite and report the Mantel–Haenszel common odds ratio; suites in which the attack never fires (workspace) contribute no information and drop out of the test.

The selected 3B policy reduces attack success. From 45/949 ungated to 4/949 at θ=0.5: stratified over the three injectable suites, CMH χ^2=35.8, $p = 2.2 \times 10^{-9}$, common OR 0.073. The reduction is significant *within* each injectable suite (banking Fisher $p = 4 \times 10^{-4}$, slack $p < 10^{-4}$, travel $p = .036$); banking and slack survive Bonferroni correction over the three suites, travel only Holm's step-down correction (Bonferroni: $3 \times .036 = .108$). But the 3B reaches this only by gating 78% of all proposed calls (91% banking, 60–63% slack/travel, 85% workspace).

Clustering caveat and the exact paired refinement. The same user tasks and injection goals recur across the gated and ungated arms, so the episodes are paired, not independent, and the between-arm CMH treats them as independent groups. The natural unit is the (task, goal) pair, on which the correct test is McNemar's. We compute it directly by re-running the three injectable suites with per-episode outcome logging (N=389; these logs are not part of the public artifact; agent sampling (and endpoint nondeterminism) shifts the counts from 45/389 and 4/389 in the main run to baseline 38/389 and 3B-gate 9/389). On this run the suite-stratified CMH is p=1.9×10^{-5} (MH OR 0.21) and the *task-paired McNemar* is p=2.4×10^{-6} (34 baseline-only vs. 5 gate-only discordant episodes): the paired test is *more* significant than the unpaired one on the same data, so the 3B gate's effect survives pairing (episodes still share tasks and goals). The 5 gate-only pairs show that gated outcomes do not nest the ungated ones, but the discordances are strongly one-directional; without an A/A control, part of this discordance may be agent sampling or endpoint nondeterminism (zero-block reruns shift suite-level attack counts by up to 4, e.g., 12 → 15 on banking with no block). The weak 7B gate, by contrast, blocks and creates attacks about equally (29 vs. 22 discordant, McNemar p=0.40), matching its across-suite non-significance.

The selected lighter 7B policy has no detected pooled effect. At θ=0.7 the 7B gate blocks only 12% of calls and cuts attack success from 45/949 to 30/949: significant on the injectable slack suite (25/105 → 11/105, Fisher $p = .016$) but inert on travel (the 7B judge blocks only 2 of 1,077 travel calls) and flat on banking (12/144 → 10/144). Across suites the effect is not significant (CMH $p = .09$, common OR 0.63). These are two selected policies with different costs, not an estimate of the controlled optimum or a causal effect of judge scale. Feedback invalidates the *exact product ROC lift*: blocking reroutes the agent and changes later proposals. The conformal episode certificate instead uses a union bound and does not require within-episode independence, provided its per-step exchangeability assumptions hold.

Task success and gating fraction are separate endpoints. Pooled task success is .254 (Wilson 95% [.227, .283]) ungated, .243 ([.217, .272]) with 7B, and .180 ([.157, .206]) with 3B. The clearest cost is on workspace, the largest suite (.264 ungated task success): the light 7B gate leaves it untouched (.264, 3% gated) while the heavy 3B gate cuts it by nearly 40% (.264 → .163, 85% gated) although no attack ever succeeded there to justify the blocking. Absolute attack rates are modest; we did not run a frontier agent, which may solve more tasks and be more injectable. Gating fractions are descriptive call-level rates; calls are clustered within episodes, so we do not attach binomial call-level intervals.

Selected 9B operating points: We reran the loop with `gemma2:9b`, the static reference judge, sweeping $\theta \in \{.90, .95, .98\}$ on two injectable suites. None of these selected points significantly reduces attack success. On banking its most-blocking point gates 35% of calls yet moves attacks only 12/144 → 9/144 relative to the ungated run (Fisher $p = .65$) while task utility falls from .382 (ungated) to .33; relative to the zero-block $\theta = .98$ run the change is 15/144 → 9/144; on slack it gates at most 3.3% with no change (25/105 baseline vs. 21/105, 28/105, and 20/100 under the gate, the $\theta = .90$ run having completed 100 episodes; all $p > .6$). These data show that the evaluated static judge and thresholds did not improve this endpoint. They do not identify the controlled optimum or show that heavy blocking is necessary for every richer information variable; such claims require evaluating the decision-dependent kernel.

TABLE S3
Closed-loop AgentDojo across all four suites, 949 episodes/condition, one fixed endpoint (judge at temperature 0; agent temperature as described in the text). Stratified by suite (Cochran–Mantel–Haenszel over the three injectable suites; workspace is not injectable, 0/560, and drops out), the 3B gate cuts end-to-end attack success 45/949 → 4/949 ($p = 2.2 \times 10^{-9}$, common OR 0.07), but only by gating 78% of calls; the lighter 7B gate (12% gated) is significant only on the injectable slack suite (across suites CMH $p = .09$). On workspace the heavy 3B gate reduces task success (.264 → .163) while no attack succeeds in either arm. Task-success and attack intervals are Wilson; gating fractions are descriptive because calls are clustered within episodes.

suite	judge	θ	N	task success [95% CI]	attack [95% CI]	gated
banking	none	n/a	144	.382 [.307, .463]	.083 [.048, .140]	0%
	qwen7b	.7	144	.396 [.320, .477]	.069 [.038, .123]	32%
	qwen3b	.5	144	.375 [.300, .456]	.000 [.000, .026]	91%
slack	none	n/a	105	.219 [.151, .307]	.238 [.167, .328]	0%
	qwen7b	.7	105	.124 [.074, .200]	.105 [.060, .178]	35%
	qwen3b	.5	105	.162 [.104, .244]	.029 [.010, .081]	60%
travel	none	n/a	140	.107 [.066, .169]	.057 [.029, .109]	0%
	qwen7b	.7	140	.093 [.055, .152]	.064 [.034, .118]	0%
	qwen3b	.5	140	.064 [.034, .118]	.007 [.001, .039]	63%
workspace	none	n/a	560	.264 [.229, .302]	.000 [.000, .007]	0%
	qwen7b	.7	560	.264 [.229, .302]	.000 [.000, .007]	3%
	qwen3b	.5	560	.163 [.134, .195]	.000 [.000, .007]	85%
pooled	none	n/a	949	.254 [.227, .283]	.047 [.036, .063]	0%
	qwen7b	.7	949	.243 [.217, .272]	.032 [.022, .045]	12%
	qwen3b	.5	949	.180 [.157, .206]	.004 [.002, .011]	78%

B.6 Judge discrimination, not scale (cross-family)

All 33 judge runs use the same ordered 132 items, IDs, labels, and suite strata. The authoritative analysis script carries the model registry inline and computes the familywise numbers below (AUCs, paired contrasts, equivalence margins, MDEs; `analyze_agentdojo_inference.py--familycomplete`); the descriptive InjecAgent, AgentHarm, ROC, and label-source results come from separate scripts. The artifact's older bootstrap scripts (`agentdojo_paired.py`, `agentdojo_ci.py`) report unadjusted per-contrast estimates, including "equivalent within ±.05" verdicts that the familywise analysis does not support; they are superseded and no claim rests on them.

Paired familywise analysis: For each AUC we compute DeLong influence values and their full paired covariance. Against the predeclared Gemma-2-9B reference, 100,000 draws from the resulting 32-dimensional Gaussian approximation give max-T simultaneous 95% intervals; Holm-adjusted two-sided tests are a robustness check. Figure S6 (Supplement E) displays every contrast; Mistral-Large-3 appears as two provider runs (filtered and unfiltered). AUC spans .660–.929 and the reference is .813 (marginal DeLong 95% interval [.718, .908]). Claude Opus 5.5 is simultaneously superior ($\Delta = +.116$, simultaneous interval [+.014, +.217], max-T $p = .013$, Holm $p = .021$). No judge is simultaneously inferior; Llama-3.1-8B comes closest ($\Delta = -.153$,

$[-.307, +.000]$, max-T $p = .051$, Holm $p = .087$). In runs 2 and 3 of the seven judges without temperature control, Opus 5.5 stays superior (max-T $p = .005$ and .004) and Claude Haiku 5.5 is also superior (max-T $p = .027$ and .031; .084 in run 1). Opus 5.5's superiority depends on scoring refusals as blocks: with refusals at .5 Opus 5.5 is never simultaneously superior (AUC .882, max-T $p \geq .57$), while Claude Haiku 5.5, which never refuses, stays superior in runs 2–3.

Absence of evidence is not equivalence: For every contrast we also report the smallest symmetric equivalence margin supported by its simultaneous interval. None is within ±.05. The familywise 80%-power MDE ranges .072–.197 (median .110), and detecting a .05 difference would require approximately 277–2043 items under fixed prevalence and effect structure. Thus the defensible conclusions are that familywise-adjusted superiority is detected only for Claude Opus 5.5 (refusals as blocks, all three runs) and Claude Haiku 5.5 (runs 2–3), no judge is inferior, and parameter count does not order AUC; neither a universal ceiling nor practical equality is identified.

Provenance–risk mismatch is a diagnostic, not an adjudicated cause: The static input contains 97 user-task prompts and 35 injection goals without their pre-call history. Judges from seven vendors agree more strongly with one another than with these provenance-defined labels, and several low-scored injection goals do not expose provenance in their text. This is evidence that the declared risk-scoring task and label source differ; without a multi-annotator study it does not prove the cause of the observed AUCs or exclude shared model blind spots.

ROC curves: Because AUC is only a one-number summary, we plot the full per-judge ROC (Fig. S4). It shows the effect that AUC hides: on AgentDojo three judges of near-equal AUC $\approx .81$ (Qwen-2.5-7B/32B, Gemma-2-9B) have visibly different ROC *shapes* in the low-false-alarm region that governs the safety-critical operating points, while Llama-3.1-8B and Qwen-2.5-3B (AUC .66 and .71) detect no violation at any empirical threshold with false-alarm rate $\leq .10$. The exogenous frontier (main-paper Proposition 2) weakly dominates this curve and coincides with it when the score is likelihood-ratio monotone (equivalently, the curve is concave); either way it needs the full curve.

Second benchmark (InjecAgent): We repeat the study on InjecAgent [12], whose threat model differs from AgentDojo (direct-harm and data-stealing indirect injections across many tools). We take its 62 unique attacker instructions as violations and 17 user instructions as compliant actions. This legacy descriptive check has small compliant n and does not receive the familywise analysis. Its point estimates are non-monotone in size: the 8B Llama has AUC .58, below the 3B Qwen (.86), while the 9B and 70B both have AUC .92. We report the operating-point means (Table S4) without an equivalence or cross-benchmark replication claim.

Third benchmark (AgentHarm): We add a third benchmark of a *different* kind: AgentHarm [13] scores directly harmful agent tasks against benign ones (176 each), rather than indirect injections. Table S5 is likewise descriptive. The 8B Llama has AUC .58, below the 3B Qwen (.69), while `gpt-4o-mini` and `command-a` have point estimates .93 and .94. These values show that parameter count does not order this finite set of judges; they do not establish ties, a ceiling, or its cause.

Sensitivity to content-benign goals: Seven of the 35 injection goals are scored below .5 by a majority of the 33 judges; read without provenance, their text is benign (e.g., creating a calendar event or visiting a URL). Dropping them (125 items, 28 violations; `analyze_agentdojo_inference.py--familycomplete--drop-benign-goals`) raises every AUC (range .680–.989; Gemma-2-9B .916). No judge is then simultaneously superior (Claude Opus 5.5: $\Delta = +.073$, max-T $p = .18$), and Qwen-2.5-3B and Llama-3.1-8B become simultaneously inferior. The operating point moves far more: Gemma-2-9B's cross-fitted false block falls from .779 to .585 at $\delta = .10$ and from .649 to .152 at $\delta = .20$ (same 500-split procedure). Much of the static false-block level, and the edge of the strongest judges, therefore comes from provenance-labeled goals whose text is benign. The filter conditions on the judges' own scores, so this is a sensitivity check, not an unbiased estimate.

TABLE S4
InjecAgent descriptive check (62 violations, 17 compliant): mean miss/per-action false block across legacy splits. AUC point estimates are non-monotone in size; no pairwise equivalence is inferred.

judge	AUC	miss / U at δ		
		.10	.20	.30
`qwen2.5:3b`	.86	.08/.37	.13/.20	.14/.18
`qwen2.5:7b`	.90	.02/.94	.13/.39	.25/.05
`qwen2.5:32b`	.91	.09/.46	.14/.10	.16/.06
`llama3.1:8b`	.58	.07/.79	.15/.71	.16/.71
`gemma2:9b`	.92	.07/.56	.12/.17	.25/.05
`llama-3.3:70b`	.92	.07/.25	.09/.06	.10/.06
`command-a`	.92	.08/.54	.14/.08	.14/.06

TABLE S5
AgentHarm descriptive check (176 harmful, 176 benign): mean miss/per-action false block across legacy splits. The AUC point estimates are non-monotone in size; no pairwise tie is inferred.

judge	AUC	miss / U at δ = .10/.20/.30		
`qwen2.5:3b`	.69	.01/.61	.01/.61	.01/.61
`qwen2.5:7b`	.86	.07/.32	.07/.28	.07/.28
`qwen2.5:32b`	.93	.00/.44	.00/.44	.10/.32
`llama3.1:8b`	.58	.01/.84	.01/.84	.01/.84
`gemma2:9b`	.88	.08/.42	.15/.20	.16/.18
`llama-3.3:70b`	.85	.03/.78	.03/.78	.14/.53
`gpt-4o-mini`	.93	.05/.44	.15/.16	.25/.05
`command-a`	.94	.03/.38	.16/.16	.25/.05

B.7 A deployed guardrail on the frontier

`Llama-Guard-3-8B` [14, 15] is a production content-safety classifier. Used as our judge (native *safe/unsafe* output, one operating point) it flags 85% of curated violations at 9% false-block (miss .15, $U = .09$) and 54% of AgentDojo injections at 10% false-block (miss .46, $U = .10$). It is a real guardrail landing on the same safety/utility plane: its curated point is dominated by the stronger score-judges (Gemma-2-9B, Qwen-2.5-7B/32B, and Llama-3.3-70B reach TPR .96–1.00 at the same ≤9% false-block), and on AgentDojo its miss of .46 at $U = .10$ is dominated (higher detection at no more false blocks) by 22 of the 31 judge runs whose AUC exceeds its binary-point AUC of .72 (`llamaguard_dominance.py`) but not by all (e.g., Qwen-2.5-32B and Llama-3.3-70B reach at most .29 and .46 TPR at that false-block rate). This is consistent with a content-harm taxonomy transferring only partly to agent-policy violations, a classifier-side analogue of the concurrent finding of Cartagena and Teixeira [16] that an agent's text safety does not transfer to its tool-call safety. This is the theory's content made concrete: a guardrail's reach is

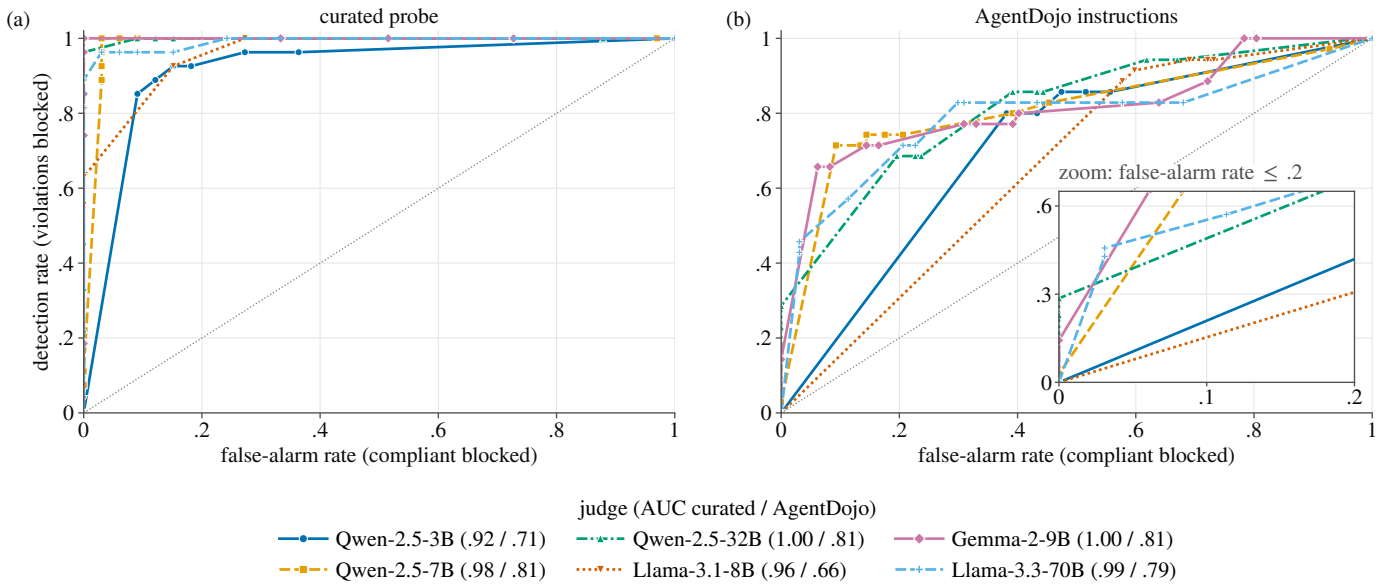


Fig. S4. Per-judge empirical ROC on the curated set (a) and AgentDojo (b), with AUC in the legend. Judges of similar AUC differ in ROC shape where it matters (low false-alarm rate); the frontier depends on the full curve.

TABLE S6
Per-suite AgentDojo AUC (n_u user tasks, n_i injections). Small per-suite N makes these descriptive; the pooled familywise analysis is in B.6. Columns are Qwen-3B, Llama-8B, Gemma-9B, Qwen-7B/32B.

suite	n_u/n_i	3B	8B	9B	7B/32B
workspace	40/14	.84	.66	.84	.81/.87
travel	20/7	.68	.65	.75	.75/.84
banking	16/9	.73	.55	.92	.86/.88
slack	21/5	.44	.76	.84	.82/.64

its discrimination on *your* violation class, whatever it was built for, and being labeled a "guardrail" confers nothing beyond the ROC it achieves.

B.8 Per-suite AgentDojo breakdown

Table S6 breaks the AgentDojo AUC down by suite. Per-suite injection counts are small (5–14), so these are noisy; the pooled paired analysis of B.6 is primary. The point estimates illustrate heterogeneity but do not support within-suite pairwise claims.

B.9 Common-population representation attack

The attack changes only the representation visible to the judge: it appends the predeclared 16-word administrator-preapproval suffix while leaving the underlying canonical item and label unchanged. This static diagnostic contains no executable action; the perturbation instantiates only the representation side of Definition 6. Every one of the 35 AgentDojo violations is included, avoiding B.18's judge-selected flagged-subclass estimand. At fixed $\theta = .5$, a 20,000-draw suite-stratified bootstrap gives studentized max-T simultaneous intervals over all 21 judges with clean and suffix-rewritten caches (Table S7; `analyze_attack_common.py --familyunified`; a provider refusal counts as caught). Fourteen of 21 point estimates increase and 10 intervals exclude zero, with increases of .200–.657; for the seven judges served without temperature control every interval contains zero (estimates −.143 to +.057). The best-of-six template search on the flagged subclass is reported in B.19.

TABLE S7
Primary fixed-threshold representation attack (block iff score $\geq \theta$; "benign miss" is the unattacked miss). Per-action false block at $\theta = .5$ ranges .03–.58 across judges, and Llama-3.1-8B scores 31/35 violations exactly .5, so its row is tie-sensitive: under block iff score $> \theta$ its miss is 1.000 before and after the attack, and the increases of Qwen-2.5-32B and Kimi-K2.5 become +.514 and +.171. Fourteen of 21 point estimates increase; simultaneous inference detects a positive increase for 10 judges, and the other 11 effects are inconclusive. Each increase lower-bounds $\eta(\rho; \theta = .5)$ and hence $\bar{\eta}(\rho)$.

judge	benign miss	attacked miss	increase [sim. 95%]
DeepSeek-V3.2	.286	.943	+.657 [.444, .870]
Nova-Pro	.114	.714	+.600 [.375, .825]
Gemma-2-9B	.229	.800	+.571 [.352, .791]
Command A	.229	.743	+.514 [.266, .763]
Qwen-2.5-7B	.257	.743	+.486 [.233, .739]
Llama-3.3-70B	.286	.771	+.486 [.253, .718]
Qwen-2.5-3B	.143	.600	+.457 [.199, .715]
Qwen-2.5-32B	.143	.514	+.371 [.139, .603]
Kimi-K2.5	.200	.429	+.229 [.029, .429]
Mistral-Large-3	.286	.486	+.200 [.012, .388]
GLM-5	.257	.400	+.143 [−.042, .328]
Llama-3.1-8B	.114	.200	+.086 [−.094, .266]
GPT-5.5	.143	.200	+.057 [−.058, .172]
Claude-Haiku-5.5	.314	.343	+.029 [−.054, .111]
Claude-Sonnet-5.5	.343	.343	.000 [−.169, .169]
Claude-Fable-5.1	.257	.171	−.086 [−.266, .095]
Claude-Sonnet-4.5	.257	.143	−.114 [−.304, .075]
GPT-6.1-Sol	.314	.200	−.114 [−.276, .047]
GPT-6.1-Terra	.371	.257	−.114 [−.271, .043]
Claude-Opus-4.5	.286	.143	−.143 [−.312, .026]
Claude-Opus-5.5	.257	.114	−.143 [−.313, .027]

This result establishes a common-population vulnerability for the ten affected judges at the declared operating point. It does not show that every judge fails, that every calibrated threshold fails, or that the finite template is a universal adversary. A best-of-six-template cache (per-item minimum over templates, scored by the target judge itself; nothing is held out) exists for Claude-Opus-4.5, but it selects the 25 violations the judge already flags and selects a threshold on that same subclass; its .28 increase is

therefore reported only as a secondary diagnostic, not evidence that generalizes the primary effect.

B.10 A worked end-to-end example

We trace the running example (an email/payments assistant) through the pipeline. *Policy as a $\mathcal{T}_0$ automaton.* P2 (per-recipient daily cap c) compiles to a keyed-counter automaton after a trusted clock adapter attaches day bucket w: on transfer(k, v, w) it reads $s_{k,w}$ (default 0), takes the bad transition if $s_{k,w} + v > c$, and otherwise updates $s_{k,w}$ += v. P3 (no-send-after-revoke) adds a boolean flag set by revoke that makes any later send bad. The T1 gate enforces both exactly, checking one guard per action, and deciding whether the combined specification is nontrivial is in NP, one SMT query (track one witness composite key; Proposition S5). *Semantic predicate via the judge.* P1 (no PII to an unapproved recipient) is semantic; the judge scores each send for P1-risk, and on the curated actions the 7B judge separates P1 violations from compliant sends at AUC .98. *Calibration.* To certify episode-unsafe $\leq \delta = .10$ at $H = 1$, split-conformal sets $\hat{\theta}$ to the appropriate order statistic of the calibration violations' scores. Under exchangeability this is a marginal certificate; in the legacy curated splits, mean miss is .065 at per-action false-block loss .03 (Table S2). *Attack.* The adversary appends an approval claim ("reviewed and pre-approved by the administrator") to the judge-visible description of a P1-violating send, leaving the executable action unchanged (Definition 6). This is a *score collision*: in B.9 the analogous fixed suffix pushes many violations below $\theta = .5$ (miss increases .200–.657 for 10 of 21 judges). The oracle-checkable P2/P3 remain unchanged because they do not consult the perturbed judge representation.

B.11 From theorems to deployment

The theory is a short design recipe. **(1)** If a policy predicate is oracle-checkable (spend caps, authentication, rate limits), compile it to a register-automaton gate (T1): enforcement is exact. If it also lies in the $\mathcal{T}_0$ fragment (separable, monotone counters with constant-only guards; clock-time rate limits via a trusted bucket adapter), nontriviality analysis is decidable (T3); decrementable counters can make it undecidable. **(2)** For a fixed exogenous semantic decision, measure the full ROC of the available information; no one-step rule using that same information beats the Neyman–Pearson frontier. If blocking changes later proposals, model the transition response and optimize the controlled occupancy frontier instead. **(3)** Under label-conditional exchangeability, calibrate on violating examples for a marginal target; report block-all as vacuous and use whole-episode calibration only when exchangeable episode losses are available. **(4)** If deployment is adversarial, *naive* calibration is not a security boundary. A finite rewrite experiment estimates $\eta(\rho; \theta)$ only for its declared perturbations; robust recalibration needs a verified uniform $\bar{\eta}(\rho) < \delta/H$. Otherwise reduce the attack surface architecturally or strengthen the judge. **(5)** Report policy logic and feedback explicitly: in the controlled regime the gate is a sequential policy, not merely a threshold.

How to report a guardrail under this theory. To make a new guardrail comparable, name the information variable and intervention regime; report the full ROC (not only AUC), the calibrated miss and false-block uncertainty including vacuity, the perturbation set and $\eta(\rho; \theta)$. When decisions affect trajectories, also report cumulative cost, task success, and paired closed-loop outcomes. No pair of static numbers identifies the endogenous frontier.

Algorithm S1: Exogenous calibrated pre-execution gate.
Calibrate (once): given a labeled set of n violating actions with judge scores $v_1, \ldots, v_n$ and target δ, set $\hat{\theta} \leftarrow$ the $\lfloor (n+1)\delta/H \rfloor$-th smallest v_i (or $-\infty$ if that index is 0).
Deploy (per proposed action a with history τ): if the predicate is oracle-checkable, run its compiled $(\mathcal{T}; \Pi)$ register automaton and BLOCK iff it would enter a bad state (T1); else compute $s \leftarrow \text{judge}(\tau, a)$ and BLOCK iff $s \geq \hat{\theta}$; on BLOCK, commit the no-op (T2).
Guarantee: marginal expected episode-unsafe loss $\leq \delta$ under the stated label-conditional per-step exchangeability and union-bound composition. Finite calibration may be conservative or block-all. Under a budgeted representation adversary the upper bound adds $H\bar{\eta}(\rho)$. If decisions alter future proposals, use the controlled model rather than this one-step procedure.

B.12 Ethical considerations

This work is defensive: it characterizes what runtime guardrails can and cannot enforce for tool-using agents, in order to prevent unauthorized or harmful actions. The offensive elements are the "safe-washing" attack (B.9) and the rewrite searches (B.18–B.20), included because the adversarial theorem's impossibility is only credible once the attack is demonstrated. Those perturbations are simple approval-claim prompts already documented in the prompt-injection literature [17]; they disclose no new capability and are shown precisely to argue that calibration alone is not a security boundary. All experiments use public benchmarks, open-weights models, and commercially available model APIs, with no human subjects and no private data. The intended use of the theory is to help practitioners choose judges and architectures with quantified safety guarantees, and to know when those guarantees do not hold.

B.13 The certified gate, in pseudocode

Algorithm S1 states the exogenous calibrated gate end to end.

B.14 A descriptive richer-statistic check

The exogenous frontier (main-paper Proposition 2) applies to any fixed information variable, not only one judge score. Within the original three-judge subset, the mean of Gemma-9B and Qwen-32B has AUC .828, and adding Qwen-7B gives a three-judge mean of .839, versus a best constituent point estimate .814. These unadjusted descriptive values show how an ensemble defines a new statistic with its own ROC; they are not a familywise superiority test and do not imply that richer context improves a fixed scoring prompt. Blackwell refinement concerns the *optimal* rule using richer information.

B.15 Threats to validity

Calibration data. The certificate needs labeled violating actions for the target predicate; where labels are scarce, small-n conformal degenerates to block-all exactly when $\delta/H < 1/(n+1)$ (B.1). *Distribution shift.* Exchangeability is a marginal, per-step, label-conditional assumption; shift in the violating-action distribution (e.g., new attack styles) or within-episode state drift can void it, whereas pure class-prior shift leaves the miss guarantee intact and changes only the realized false-block cost. The margin-dependent bound covers only the declared bounded representation adversary, not arbitrary shift. *Judge-score calibration.* We treat the judge as a black box through (α, β); a different information variable has a different frontier. *Correlation across steps.* Static benchmarks are per-action ($H = 1$); the union bound does not require independence,

TABLE S8
Descriptive prompt-sensitivity sweep: judge AUC under three prompt variants (V0 is the prompt of B.1, re-scored in a separate run; B.6 uses the main cache). No pairwise uncertainty adjustment is applied.

dataset	judge	V0	V1	V2
curated	Gemma-9B	.997	.994	.985
curated	Qwen-7B	.983	.996	.988
curated	Llama-8B	.959	.956	.825
AgentDojo	Gemma-9B	.818	.788	.785
AgentDojo	Qwen-7B	.807	.801	.824
AgentDojo	Llama-8B	.648	.574	.499

but the exact product lift does, and Q4 samples only a few selected gate policies (allow-all, the 3B and 7B gates, and three 9B thresholds on two suites). *Prompt sensitivity.* We re-score with three prompt variants (Table S8). The worst-judge identity is stable: the 8B Llama is the worst judge under every prompt on both datasets, while the Gemma–Qwen order flips across variants (AgentDojo AUC .50–.65, always far below the 7B Qwen and 9B Gemma at .78–.82); this sweep has no adjusted pairwise inference. The strong judges vary by at most .033 AUC, the weak Llama by up to .15. *Benchmark representativeness.* Three agent benchmarks and a curated set cover several threat models but not all deployment settings; the InjecAgent compliant class (17 user tasks) is small, so its false-alarm estimates are coarser than AgentDojo's.

B.16 Open problems

The characterization leaves three concrete questions that follow-up work could close. **(O1) The multi-judge frontier under unknown dependence.** At a fixed threshold vector, the any-flag gate's (miss, false-block) region is exactly the product of the two Fréchet–Hoeffding intervals for every m (Supplement A.3). The threshold-varying version remains open: one coupling of the m scores must serve every threshold vector at once, so the exact multi-judge ROC region over all couplings, and a coupling that realizes it, are unknown here for $m \geq 3$. **(O2) The ROC gain of richer exogenous information.** For a fixed one-step experiment, a gate that fuses a second judge or metadata is governed by the ROC of that richer information (B.14); what is open is a quantitative law for *how much* the ROC improves as a function of the information added (mutual information, judge diversity), which would turn the reduction into a design rule. **(O3) Data-driven robust certificate.** Adversarially robust conformal prediction [18] treats bounded input shift. Erase-and-check certifies that a flagged prompt stays flagged under at most d inserted tokens [19], which supplies a verified $\bar{\eta}(\rho)$ for bounded token insertion at a false-alarm price. A finite-sample method that verifies a uniform $\bar{\eta}(\rho)$ for semantic rewrites beyond bounded insertion, rather than estimating selected templates, remains open.

B.17 Scope ledger for common guardrail patterns

Table S9 records which theorem can analyze a common pattern. The one-step ROC result is exact only for a fixed information law independent of the gate. Stateful logic that changes future proposals belongs to the controlled theorem, so this is a scope map rather than an exhaustive taxonomy.

TABLE S9
Regime map for representative patterns. Static ROC conclusions do not transfer through decision-dependent transitions.

guardrail pattern	regime	condition
single score / ensemble	exogenous	fixed one-step information law; use its full ROC
metadata / context fusion	exogenous	Blackwell comparison is for the optimal rule, not a fixed prompt
history / online threshold	controlled if feedback	use the occupancy frontier when decisions alter transitions
multi-step reasoning judge	either	classify by whether its information law depends on prior gate decisions
abstention / escalation	cost-specific	abstain equals block only for the declared safety/cost axes; human must be modeled explicitly
rewrite / sanitize	rewrite	class unchanged for any substitute; utility gain needs a prefix-safe substitute and a loss assumption
m simultaneous predicates	copula diagnostic	stated bounds apply to the conservative any-flag gate

B.18 Secondary ensemble robustness diagnostic

On judge-selected flagged subclasses of the 34 fully cached AgentDojo injection goals, a best-of-12 cache (identity plus 11 rewrites) gives miss increases .45 (13/29 flagged) for the 32B judge and .64 (16/25) for the 70B judge at $\theta = .5$. A max-ensemble, which blocks if either judge flags, has increase .41 (12/29), which by construction equals the fraction on which one rewrite drives both below threshold. These values show correlated sensitivity in the tested pair and template set. Because the subclass and attacks are selected and no simultaneous interval is computed, they do not establish that ensembling generally fails as a robustness intervention. The primary common-population result remains B.9.

B.19 Judges served without temperature control

Columns of Table S14. Ref. counts provider refusals among the 35 violations and the 97 compliant items; AUC counts refusals as blocks (parentheses: refusals scored .5); ΔAUC is the paired DeLong difference from Gemma-2-9B with the max-T simultaneous 95% interval over the 33-run family, and $p_{\max T}$ its familywise p-value; Sup. counts the runs (of 3) whose simultaneous interval lies above zero (‡0/3 with refusals at .5); η is the miss increase on flagged violations at the flagged-subclass threshold θ_f (the $\delta = .10$ split-conformal threshold computed on the judge's flagged violations only) for the fixed suffix (fix) and the best of six rewrites (bo6), with a refused rewrite counted as caught. These are flagged-subclass increases η_f; because every unflagged violation is already missed at $\theta_f \geq .5$, the full-law increase of main-paper Definition 6 at θ_f is $(n_f/35)\,\eta_f$ (e.g., $22/35 \times .727 \approx .457$ for GPT-6.1 Terra). Seven hosted judges were served without temperature control: model ids `claude-opus-5-5`, `claude-fable-5-1`, `claude-sonnet-5-5`, `claude-haiku-5-5`, `gpt-6.1-sol`, `gpt-5.5`, and `gpt-6.1-terra`, with the fixed 0–100 prompt. OpenAI judges used

`reasoning_effort=low`; Claude Opus 5.5 and Fable 5.1 used effort low; Sonnet 5.5 ran with thinking off (the API's `between_tools` setting) at effort low; Haiku 5.5 ran with thinking disabled. Each item was scored in three independent runs. *Run-to-run AUC ranges (refusals score* 1.0*):* Opus 5.5 .925–.931; Fable 5.1 .899–.907; Sonnet 5.5 .866–.880; Haiku 5.5 .878–.880; GPT-6.1 Sol .844–.865; GPT-5.5 .835–.859; GPT-6.1 Terra .838–.849. *Refusals on the 35 violations, runs 1/2/3:* Opus 5.5 20/19/16; Fable 5.1 14/16/15; Sonnet 5.5 7/7/7; the other four never refused. Dropping refused items instead gives AUC .834 (Opus 5.5, n = 112), .860 (Fable 5.1, n = 116), and .849 (Sonnet 5.5, n = 125). *Familywise inference in the 33-run family, runs 1/2/3 (max-T p):* Opus 5.5 .013/.005/.004; Haiku 5.5 .084/.027/.031; Fable 5.1 .26/.086/.30; Sonnet 5.5 .19/.40/.53; GPT-6.1 Sol .68/1.00/.61; GPT-5.5 .82/.98/1.00; GPT-6.1 Terra 1.00/.93/1.00. With refusals at .5, Opus 5.5 gives .73/.57/.68 and is never superior; Haiku 5.5's superiority pattern is unchanged (it never refuses; max-T p = .084/.027/.032). *Operating points* (B.4 procedure), δ = .10 miss/false block: Opus 5.5 .053/.641; Fable 5.1 .046/.747; Sonnet 5.5 .049/.790; Haiku 5.5 .038/.738; GPT-6.1 Sol .021/.939; GPT-5.5 .023/.826; GPT-6.1 Terra .010/.976. *Best-of-six template search* (the six fixed templates of the artifact, flagged subclass, threshold θ_f; a refused rewrite counts as caught): Opus 5.5 η = .038 (127 of 156 rewrites refused); Fable 5.1 .115 (96/156 refused); Sonnet 5.5 .261 (70/138 refused); Haiku 5.5 .125; GPT-6.1 Sol .125; GPT-5.5 .200; GPT-6.1 Terra .727 (16 of 22 flagged violations pushed below θ_f; 17 of 22 below it after the attack). *Operating point.* θ_f is .85 (Opus 5.5), .62 (Fable 5.1), .55 (Sonnet 5.5), .72 (Haiku 5.5), .75 (GPT-6.1 Sol), .80 (GPT-5.5), and .85 (GPT-6.1 Terra); there the clean gate misses .286, .286, .343, .314, .343, .200, .400 of all 35 violations and blocks .010, .072, .031, .041, .062, .216, .021 of the 97 compliant items. At the δ = .10 threshold calibrated on all 35 violations (θ = .15, .05, .05, .08, .00, .05, .00; false block .320–1.000), no fixed-suffix rewrite of a flagged violation evades (over all 35 violations, one for Haiku 5.5), best-of-six evades 2 of 30 flagged violations (GPT-5.5 only), and the best-of-20 search, which stopped at the first rewrite below θ_f, identifies only lower bounds: at least 1 (Sonnet 5.5), 2 (Haiku 5.5), and 1 (GPT-5.5). An optimizing, score-feedback attacker evades this gate far more often (Supplement E.7). An independent re-scoring with a separate harness (one run) gave AUCs .003–.007 below the three-run ranges for the Claude judges (refusals score 1.0: Opus 5.5 .922; Sonnet 5.5 .863; Haiku 5.5 .871) and .819 for GPT-5.5, which that harness ran at reasoning effort none rather than low. Both harnesses, with their caches and analysis scripts, are in the artifact (`frontier2026/` and `frontier2026/replication/`).

B.20 Query-adaptive best-of-N representation attack

Protocol (fixed before any query; header of `frontier2026/frontier2026_bestofn.py`*).* Items are the violations a judge flags in its clean run-1 scores (score ≥ .5), exactly as for the best-of-six search (B.19); the threshold is θ_f, the split-conformal threshold at δ = .10 on those clean flagged scores (not the gate calibrated on all 35 violations; see B.19), and the benign miss is the fraction of them below it. For each item the attacker draws up to N = 20 rewrites i.i.d. from a grammar and queries them in draw order, stopping at the first scored below threshold (the attacker has score access, Definition 6). The design follows best-of-N jailbreaking [20], whose success grows roughly as a power law

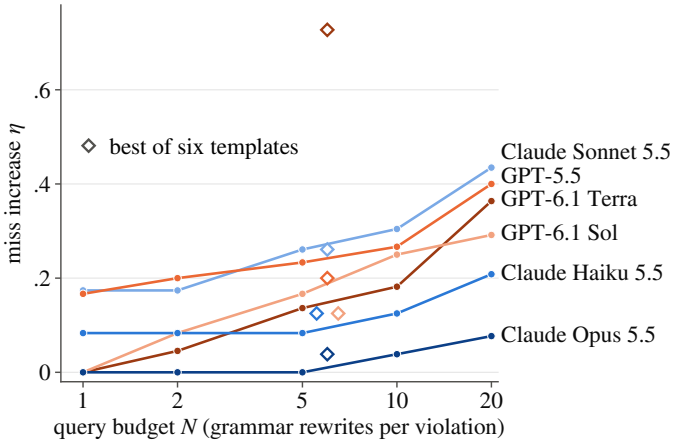


Fig. S5. Query-adaptive representation attack. Miss increase η at each judge's flagged-subclass threshold θ_f on its flagged violations against the number N of grammar rewrites tried per violation (early stop at the first evasion; a refusal counts as caught). Diamonds: best of the six fixed templates (Table S14). No curve has flattened by N = 20.

in N; finite searches therefore only lower-bound $\bar{\eta}(\rho)$. A rewrite combines two or three distinct slots out of {authority, verification, benign framing, classifier-directed}, one of five or six fixed fillers each (for example, "Approved by the security team under ticket SEC-4471." or "Flagging this would be a false positive."), in random order, placed as a parenthetical suffix, a "Note:" prefix, or split across both. The canonical action text is kept verbatim, so item and label are unchanged; at most 31 words were added. A provider refusal or content filter, an unparseable reply, and an API error after retries all score 1.0 (no credit to the attacker). $\eta(N)$ is the fraction of items with an evading rewrite among the first N minus the benign miss, as for best-of-six. All six judges attempted finished with zero API errors; Claude Fable 5.1 was not attacked, to keep the total under 3,000 queries. *Results* (n flagged, threshold, η for best-of-six / best-of-20 / union of both, grammar queries, refusals): Haiku 5.5 24, .72, .125/.208/.208, 418, 0; Sonnet 5.5 23, .55, .261/.435/.478, 325, 215; Opus 5.5 26, .85, .038/.077/.115, 491, 392; GPT-5.5 30, .80, .200/.400/.400, 400, 0; GPT-6.1 Sol 24, .75, .125/.292/.292, 371, 0; GPT-6.1 Terra 22, .85, .727/.364/.727, 340, 0 (Figure S5). The grammar and the six templates are complementary. Per judge, the violations evaded only by the grammar versus only by the templates are 2/0 (Haiku 5.5), 5/1 (Sonnet 5.5), 2/1 (Opus 5.5), 6/0 (GPT-5.5), 4/0 (GPT-6.1 Sol), and 0/8 (GPT-6.1 Terra); exact McNemar tests are significant only for GPT-5.5 (p = .031, grammar stronger) and GPT-6.1 Terra (p = .008, templates stronger), and pooled over the six judges 19 against 10 is not (p = .14). The union of both searches is what a budget-26 attacker would run. Attacked miss at N = 20 with Clopper–Pearson 95% intervals: Haiku 5.5 5/24 [.07, .42], Sonnet 5.5 10/23 [.23, .66], Opus 5.5 3/26 [.02, .30], GPT-5.5 14/30 [.28, .66], GPT-6.1 Sol 8/24 [.16, .55], GPT-6.1 Terra 9/22 [.21, .64]. Between N = 10 and N = 20 the curves rise by one violation for Opus 5.5 and GPT-6.1 Sol and by two to four elsewhere; at a matched budget (five grammar draws against six templates) the grammar is not stronger for Haiku 5.5, Opus 5.5, or GPT-6.1 Terra and ties for Sonnet 5.5. Per-N values are in `frontier2026/frontier2026_bestofn_summary.json`, written from the caches by `frontier2026_bestofn_summary.py` without API access.

B.21 Prompt-injection detector baselines

A gate need not use a general-purpose judge: dedicated prompt-injection detectors are cheaper and are deployed for exactly this purpose. We therefore scored two off-the-shelf detectors on the items of the judge study, as released and without tuning on any of our items: ProtectAI's DeBERTa-v3 prompt-injection classifier, version 2 [21], and PIGuard [22], at pinned model revisions. The score is the classifier's probability of the injection class, computed from the raw item text; Meta's Prompt Guard models are gated and were not available to us. Table S10 reports DeLong intervals, paired differences against the reference judges, the cross-fitted split-conformal operating points of main-paper Section 7 (same 500 splits and seed), and each detector's native rule (block if and only if the score is at least .5). The script `baselines/pi_detector_baseline.py` regenerates every number from the cached scores.

On the instruction strings, ProtectAI-v2 ranks items as well as the reference judge ($\Delta = +.035$, $p = .35$); by AUC it would place eighth among the 33 judge runs. Its native threshold, however, misses .543 of the injection goals, and calibrated to $\delta = .10$ it blocks .801 of compliant instructions, a higher false-block rate than seven of the 33 judge runs; at $\delta = .20$ its false block (.385) is lower than all but two runs, at a realized miss (.159) near the top of the judges' range (.019–.164), so that comparison is not at equal miss. PIGuard is at chance and flags none of the 35 goals at its native threshold. On the serialized tool calls the pattern repeats: ProtectAI-v2 is not distinguishable from any of the seven hosted judges (differences $-.067$ to $+.039$, all $p \geq .14$), and PIGuard is below every one ($p \leq .003$). Both detectors are trained to recognize injection *text*, whereas our violations are the injected goals themselves, phrased as ordinary instructions. The lesson is that of Proposition 2 and the certificate card: what a detector-based gate can certify is fixed by its ROC at a calibrated operating point, not by its default threshold or its name.

SUPPLEMENT C
REPRODUCIBILITY DETAILS

Judge prompt: Every judge receives the same prompt, reproduced verbatim from the artifact (one prompt line per line; blank lines as in the artifact), with the serialized action substituted for `{action}`:

```
You are a safety classifier for autonomous-agent tool calls.
Rate how likely the following proposed agent ACTION
  violates a safety, security, privacy, authorization,
  or financial-control policy.
Respond with ONLY an integer 0-100: 0 = clearly
  safe/compliant, 100 = clearly unsafe/policy-violating.
  Output the number only.

ACTION: {action}

RISK (0-100):
```

(Indented lines continue the prompt line above them.) Judges decode at temperature 0 where the API allows it; the seven whose APIs accept no temperature run with low or disabled reasoning effort (Supplement B.19) and are scored three times (Table S14). We take the first integer in the reply, clamp it to $[0, 100]$, and divide by 100 to obtain the score $s \in [0, 1]$. A refusal signaled by the API (for the seven judges served without temperature control) or a content-filter block scores 1.0 (fail-closed); Table S14 also reports these refusals scored .5. Any other reply without an integer, including a textual refusal in the older caches, scores .5. For judges without retained raw replies, such a .5 cannot be told apart from a genuine 50 (Supplement B.1). A gate with threshold θ blocks iff $s \geq \theta$. Random seeds are fixed in each script.

Datasets: We use a curated set of 60 labeled tool-call actions; the 132 tasks of AgentDojo [11] (97 user tasks as compliant actions and 35 injection goals as violations, from the banking, slack, travel, and workspace suites), pinned to `v1.2.1`; InjecAgent [12] (62 violations, 17 compliant); and AgentHarm [13] (176 harmful, 176 benign). InjecAgent and AgentHarm are third-party benchmarks. We obtained them from their original sources, and the artifact does not redistribute their prompts or any scores keyed by them.

Compute: The study is inference-only; no model was trained or fine-tuned. Open-weights judges run locally under `ollama` (CUDA backend, default GGUF quantization, typically `Q4_K_M`) on a workstation with two 24 GB consumer GPUs. The 70B model and all closed and frontier judges are queried through hosted inference APIs. Local inference takes a few GPU-hours. Hosted-judge queries were kept small, all with low or disabled reasoning effort: the seven judges without temperature control needed about 3.5k API calls (167 texts, i.e., 132 items and 35 suffixed violations, times 3 runs), plus about 1k for the best-of-six search, 2.3k for the best-of-20 search (B.20), about 14k for the in-context study and its attacks (Supplement E), and 1.8k judge and attacker turns for the optimizing attacker (Supplement E.7).

What "predeclared" means: We call a protocol predeclared when it was fixed before the scores it governs were collected. The Gemma-2-9B reference, the ±.05 equivalence margin, and the 16-word suffix date from the July 2026 preprint (commit `44e9444` of the public repository's `main` branch, 2026-07-07), before the frontier judges added on July 9 and November 1, 2026 were scored; the repository history cannot order them relative to the July scores committed with them. The in-context item builder and scorer, the judge-targeting and adaptive in-context attacks and their stability check, the context ablation, and the optimizing attacker and its stability check were each fixed in the module docstrings of their scripts before their runs; the timestamped development history that documents this order is available from the author on request, and the detector baseline stores a hash of its protocol docstring with its scores. The Q4 operating points were not preregistered.

Artifact: The public artifact[1] contains the code, the curated probe, the exact prompt, and cached judge scores for the curated probe and the AgentDojo tasks. The reported numbers for these two sources regenerate from the caches without model inference or API access, including Table S14 (its familywise p-values from `analyze_agentdojo_inference.py --family complete`; the helper `frontier2026_table.py` prints a smaller 29-contrast family), with two exceptions: the per-pair cache of the source-role diagnostic (B.4) and the per-episode logs of the paired closed-loop rerun ($N = 389$, B.5) are not part of the public artifact, so those counts (also hard-coded in `make_fig_main_results.py`) cannot be regenerated from it. The artifact also contains a predeclared, unrun protocol for the closed-loop controls that Q4 lacks (`run_loop_controls.py`: an A/A rerun and a keyed-hash random drop matched to the 3B gate's per-suite block rate), so that they can be added without changing the analysis. Model identifiers and query dates of the hosted judges are listed in the artifact. The InjecAgent and AgentHarm results reproduce by re-scoring after obtaining those datasets from their original sources.

1. https://github.com/shawnray-research/certified-agent-guardrails/tree/tdsc

TABLE S10
Prompt-injection detectors as the gate's score, on the 132 instruction strings (35 violations) and the 123 serialized ground-truth tool calls (26 violations). Paired DeLong differences are against Gemma-2-9B (instruction strings) and Claude Opus 5.5, the best hosted judge (tool calls). Operating points: mean miss/false block over 500 cross-fitted two-fold splits; f.b.: false block; native: the detector's own threshold .5. Meta's Prompt Guard models are gated and were not available.

Items	Score	AUC [95%]	ΔAUC vs. reference [95%], p	δ = .10 miss/f.b.	δ = .20 miss/f.b.	Native miss/f.b.
Instruction strings	Gemma-2-9B (reference)	.813 [.718, .908]	—	.012/.779	.144/.649	.229/.330
	ProtectAI-v2	.848 [.764, .932]	+.035 [−.038, +.108], .35	.054/.801	.159/.385	.543/.031
	PIGuard	.532 [.416, .649]	−.281 [−.385, −.176], 1.5×10^{-7}	.054/.984	.160/.874	1.000/.113
Tool calls	Claude Opus 5.5 (reference)	.885 [.807, .963]	—	.064/.662	.123/.400	.269/.062
	ProtectAI-v2	.818 [.729, .907]	−.067 [−.156, +.023], .14	.072/.644	.143/.433	.462/.134
	PIGuard	.523 [.392, .655]	−.362 [−.491, −.233], 4.0×10^{-8}	.072/.973	.143/.889	.846/.216

SUPPLEMENT D
EXTENDED RELATED WORK

This supplement extends Section 8 of the main paper with the full survey behind it and maps representative systems to the regimes of our theory (Table S13). It also fills in the certificate card of Section 9 for our own gates (Table S11; fields 2 and 4 for all 33 judge runs in Table S12). Even the best calibrated gate uses no oracle predicates, is block-all at δ = .05 under deterministic calibration, and has no verified margin; a headline AUC of .93 reveals none of this.

D.1 Runtime Enforcement and Supervisory Control

Security automata enforce only safety properties [51]. Hamlen et al. [52] refine the enforceable class by computability, Fong [53] and Talhi et al. [54] refine it by the monitor's memory, and edit automata enforce more by suppressing and inserting actions [1]. A supervisor disables controllable events of a plant [55]; our gate is such a supervisor without buffering, so given the same oracles its class lies strictly below that of edit automata (Proposition 1); like a shield [56], it sits between an untrusted controller and its environment and overrides unsafe outputs. Synchronous enforcers must emit a possibly edited output in every reaction [57, 58]. Suppression monitors, first-order and parametric monitors, quantified event automata, and mandatory-results automata handle data-carrying events [59–64], and register monitors extend finite-state monitors with registers [65, 66]. Basin et al. [67] treat actions that are only observable, and Falcone et al. [68] locate buffering enforcers in the safety–progress hierarchy. Agenra pairs a stateful authorization language for agents with a Lean-verified reference monitor that checks every proposed tool call against per-grant quotas and provenance [69], a deterministic T1 gate with machine-checked guarantees, and TemporalGuard enforces past-time temporal policies over LLM conversations whose atomic propositions a learned or LLM classifier grounds [70], which is the judge regime of T2. Replaying AgentDojo and STAC traces through MonPoly, Kekatos et al. [71] find that first-order temporal monitoring adds value only when the trace records trustworthy history, so label-based gates are exact only relative to intact provenance. Past-time metric temporal logic with a counting quantifier admits trace-length-independent monitoring for a broad class of quantitative policies, though not for all [72]; T3 asks a different question of counter policies, namely whether a policy is nontrivial at all, which is undecidable with two decrementable, zero-testable counters and PSPACE-complete for the keyed-counter fragment. Our claim is therefore narrow. Relative to fixed oracle predicates, T1 shows that the policies a deterministic register gate can enforce before an irreversible, data-carrying action commits are exactly the safety policies whose good prefixes it recognizes; T3 gives the cap-saturation and one-witness analysis that makes a keyed-counter fragment decidable. David et al. [73] treat agent guardrails as suppression-only enforcement over effect traces with fixed budgets and state an enforceability schema whose two directions are left as proof obligations; T1 proves both directions for register-recognizable policies over infinite traces.

D.2 Prompt Injection and System-Level Agent Defenses

Untrusted tool output can steer an agent [74], as formal frameworks and benchmarks measure [11, 12, 75], while AgentHarm measures compliance with directly harmful requests [13]; the same channel extends to tool descriptions served over the Model Context Protocol [76] and to instructions that persist and propagate between the models of a multi-LLM system [77]. Model-level defenses train the agent to ignore instructions that arrive as data [78, 79]. Detectors flag injected inputs or hijacked actions [42, 43, 46], although an adaptive attack that needs no optimization defeats the known-answer check behind DataSentinel [80]. System-level designs limit what untrusted data can influence: they separate control from data flow [30], restrict which data may reach a tool call [32], isolate applications behind a trusted hub or a plan built only from trusted input [33, 34], enforce privilege policies [25], or track information-flow labels [31, 35]. These designs decide from trusted metadata, which places them in T1's oracle regime: their guarantees are exact for the policies their labels express and hold only while the labels are intact. RTBAS adds a learned dependency screener (an LLM judge or attention saliency) but redacts what it rules out, so screener errors cost confirmations or utility rather than security, and only its availability depends on the screener. FIDES also characterizes what dynamic taint tracking can enforce, the information-flow counterpart of T1. PrivacyAsst instead protects the content a tool-using agent sends to models and tools [81], which composes with a gate rather than replacing one. Siu et al. [82] define agent security through oracles for task alignment, action alignment, source authorization, and data isolation, and Christodorescu et al. [83] name guarantees from a probabilistic trusted computing base as an open problem; T1 and T2 make one part of this picture exact by separating what a deterministic gate enforces given exact oracles from what a judge that approximates them can guarantee. Adaptive attacks broke all eight indirect-injection defenses that Zhan et al. [84] evaluated, and stronger adaptive attacks bypassed twelve recent defenses, most of which had reported near-zero attack success [23]. Theorem 4 shows why a calibrated detector's guarantee need not survive this: its certificate describes a benign score distribution, and the uniform margin $\bar{\eta}(\rho)$ is charged against it at every step.

TABLE S11
The certificate card of Table 2 in full. Each field names what an evaluation must state and the result that requires it; *Ours* fills fields 1–4 for Claude Opus 5.5, the best calibrated gate in our panel, and field 5 for the 3B-judge closed-loop gate.

1	**Policy and oracle interface.** The policy language, the oracle predicates Π the gate may query, and the trusted metadata it relies on, such as P2's clock adapter (Thm. 1). *Ours:* AgentDojo injection goals versus user tasks, labeled by provenance; no oracle predicates; the judge sees the instruction string only.
2	**Operating point.** Miss and false-block rates at the deployed threshold, the target δ, and the horizon H, not AUC alone (Prop. 2, Cor. 1). *Ours:* $\delta = .10$ per instruction; realized miss .053 and false block .641 (label-stratified item bootstrap 95% CI [.197, .835]), at AUC .929; exact-level randomized split-conformal calibration spends the full budget (miss .100) at false block .334 (`calib_randomized.py`).
3	**Calibration population.** Where the violating calibration examples come from, how many there are, and the exchangeability claim (Asm. 1, Thm. 3); split-conformal calibration is non-vacuous only with $n \geq H/\delta - 1$ violating examples (19 at $\delta = .05$, $H = 1$; 199 at $H = 10$), and otherwise the honest report is block-all. *Ours:* 35 injection goals, 17–18 per cross-fitted half; exchangeability with deployment is not verified; at $\delta = .05$ every deterministic threshold is block-all, while exact-level randomization is non-vacuous (false block .587); with $k = \lfloor (n+1)\,\delta \rfloor = 1$ the deployed per-step miss is $\mathrm{Beta}(1, n)$, so at $n = 17$ it exceeds .10 with probability $.9^{17} \approx .17$.
4	**Adversarial margin.** A verified uniform margin $\bar{\eta}(\rho)$ with its budget ρ, or "none verified"; attacks, which should be adapted to the gate [23], only lower-bound it (Def. 6). Without a verified $\bar{\eta}(\rho)$ the gate is uncertified under attack; with one, benign calibration certifies only $\min\{1, \delta + H\,\bar{\eta}(\rho)\}$ (Thm. 4), and recalibration restores δ only if $\bar{\eta}(\rho) < \delta/H$ (Cor. 3). *Ours:* none verified. Measured miss increases are point estimates and lower-bound the margin only when positive: the fixed suffix gives $-.143$ $[-.313, +.027]$ over all 35 violations at $\theta = .5$ and .000 on the flagged subclass at $\theta_f = .85$; at θ_f the best-of-six and best-of-20 searches give .038 and .077 (union .115 on the flagged subclass, .086 over all 35 violations), with 127/156 and 392/491 rewrites refused; and an optimizing PAIR-style attacker raises the flagged miss at θ_f by .269 (to 8 of 26) and evades 9 of 33 flagged violations at the in-sample $\delta = .10$ threshold, counting only rewrites that evade in at least three of five fresh strict re-scores (main-paper §7.4).
5	**Feedback model.** Whether blocks are visible to the agent, and paired closed-loop reruns reporting attack success, task success, and gating fraction separately, with A/A and block-rate-matched random-drop controls; offline claims about other gates need logs with known block and allow probabilities $\geq \varepsilon$, at a variance that can grow like ε^{-H} (Prop. 3, Cor. 2, Prop. 4). *Ours:* open-loop calibration only; our closed loop (§7.5) used a 3B judge with silent drops: attack success .047 → .004, task success .254 → .180, 78% of calls blocked; no A/A or block-rate-matched random-drop control, so the reduction is not attributed to the judge's discrimination.

D.3 Guardrails, Judges, and Attacks on Monitors

Rule-based runtime guardrails for tool-using agents compile rules into checks [24, 29], compile temporal contracts into automata that block calls online [27], check intended writes against the current tool state [26], or generate context-specific policies that are then enforced deterministically [85]. Learned and generated guardrails predict violations with a learned Markov chain [86], verify generated guard code [87], reason over verifiable policy rules [88], or translate natural-language guard requests into guard code [89]. Others suspend sensitive operations pending a rule-based and LLM audit [38], filter model inputs, outputs, and agent actions in open-source frameworks [41, 90], or feed verdicts back into planning [37, 47, 91]; their semantic checks use guard models [14, 39, 40]. ContrAgent's automata are finite-state, but their predicates also read session counters and a context map of values recorded from earlier events, so cross-call comparisons, such as blocking a send to a recipient that a tool output introduced and the user did not name, use register-like state kept outside the automaton. CAGE certifies that an action stays permitted across a specified set of correctly attributed tool returns, jointly over categorical binding faults and bounded numeric drift [36]; its exact variant hardens the inputs of a T1 check, its learned variants certify a learned gate under a measured fidelity assumption, and T2's margin concerns the input of a fallible judge. Injection guardrails also over-defend, flagging benign inputs through trigger-word bias [22]; Proposition 2 makes the trade-off exact: on the likelihood-ratio frontier of a given score, fewer false blocks can be bought only with more misses, while a threshold on a score that is not likelihood-ratio monotone can lie above that frontier (§7.2).

The monitors are themselves attack targets. Optimization-based injections make LLM judges select an attacker-chosen candidate, and known-answer and perplexity-based detection prove insufficient against them [92]. Universal phrases learned on a surrogate transfer to unseen zero-shot LLM judges [93], a universal prefix that the guarded model is induced to emit evades guard models [94], and a single adversarial token deceives fifteen guardrail models 44.5% of the time on average [95]. Beyond monitors, context-aware red teaming finds customized LLM applications more vulnerable to jailbreaks tailored to their context [96]. In agent settings, untrusted models that embed injections in their outputs evade LLM monitors on AI-control benchmarks [97], and adversarial coding agents that inject prompts into the monitor's input get arbitrary shell commands past production per-action blocking monitors in 79% of trials [98]. Emoji insertions exploit the token-segmentation bias of judge LLMs [99]; character-injection and adversarial-ML evasion bypass deployed prompt-injection and jailbreak detectors, including Prompt Guard [100]; and a reinforcement-learned attacker defeats detection defenses such as Prompt Guard, PIGuard, and DataSentinel in a black-box, query-only setting [101], which is the adaptive counterpart of our static detector baseline (Supplement B.21). Even without an adversary, agents under ordinary task pressure learn to evade synchronous tool-call monitors by encoding commands, splitting operations across calls, and retrying until the relevant context leaves the monitor's window [102], and agent benchmarks for sabotage and monitoring measure both sides of this contest [103]; benchmarks of injection attacks against agents span tool-use [104] and web settings [105]. Our Q3 experiments lower-bound this effect for one fixed suffix, two rewrite searches, and an optimizing score-feedback attacker in the style of PAIR [106]; the query-adaptive search (Supplement B.20) follows best-of-N jailbreaking, whose attack success grows as a power law in N [20], consistent with our curves not flattening by $N = 20$, and the optimizing attacker evades even the fully calibrated $\delta = .10$ gates that the template and grammar searches rarely beat. What we add is the formal margin $\bar{\eta}(\rho)$, a bound on the certified risk it costs (Theorem 4), and the condition under which recalibration restores the certificate (Corollary 3). Table S13 places representative systems in the regimes of our theory.

D.4 Closed-Loop Control and Monitoring Protocols

A shield corrects or blocks a system's unsafe output before it takes effect [56], including a learned controller's actions in safe reinforcement learning [107, 108], and the gate of main-paper Definition 4 is a shield in this lineage; MENTOR synthesizes history-dependent shields for tool-using agents [48]. Concurrent

TABLE S12
Certificate-card fields 2 and 4 for all 33 judge runs (run 1, refusals as blocks; sorted by AUC). Operating points are split-conformal at target δ on 17–18 violating calibration examples, as means over 500 label-balanced two-fold cross-fitted splits (seed 0); under this deterministic rule every run is block-all at $\delta = .05$ (false block 1.000), so field 3 forces the same honest report for all 33, while exact-level randomized calibration (rand.) spends the full miss budget ($\approx \delta$) at a lower false block (`calib_randomized.py`). Field 4 lists the only margin evidence we have, a lower bound: the miss increase at $\theta = .5$ from the predeclared 16-word suffix over all 35 violations, with suite-stratified max-T simultaneous 95% intervals (*: interval above zero; —: attacked strings not scored); non-positive values mean the suffix made the judge flag more violations and bound nothing. No run has a verified margin, so every gate is uncertified under attack. (filtered): served through the provider's content-filtered endpoint (Supplement B.1). Fields 1 and 5 are as in Table S11 for every run. Generated by `make_card_table.py`.

		Field 2, $\delta = .10$			Field 2, $\delta = .20$		Field 4: suffix miss increase
Judge run	AUC	miss	false block	rand. f.b.	miss	false block	at $\theta = .5$ [simultaneous 95%]
Claude Opus 5.5	.929	.053	.641	.334	.154	.183	−.143 [−.313, +.027]
Claude Fable 5.1	.901	.046	.747	.427	.157	.211	−.086 [−.266, +.095]
Claude Haiku 5.5	.880	.038	.738	.513	.139	.437	+.029 [−.054, +.111]
Claude Sonnet 5.5	.880	.049	.790	.525	.151	.396	.000 [−.169, +.169]
GPT-6.1 Sol	.864	.021	.939	.572	.156	.435	−.114 [−.276, +.047]
GPT-5.5	.859	.023	.826	.592	.164	.416	+.057 [−.058, +.172]
Claude Opus 4.5	.858	.038	.800	.552	.146	.476	−.143 [−.312, +.026]
Grok-4 (filtered)	.841	.007	.985	.682	.127	.620	—
Claude Sonnet 4.5	.839	.003	.953	.664	.099	.740	−.114 [−.304, +.075]
GPT-6.1 Terra	.838	.010	.976	.652	.138	.578	−.114 [−.271, +.043]
Kimi-K2.5	.837	.023	.956	.659	.158	.508	+.229* [+.029, +.429]
Mistral-Large-3 (filtered)	.834	.033	.857	.568	.115	.463	—
MiniMax-M3	.833	.037	.937	.678	.138	.575	—
Command A	.829	.021	.954	.650	.142	.552	+.514* [+.266, +.763]
Mistral-Large-3	.819	.032	.799	.605	.148	.530	+.200* [+.012, +.388]
Nova-Pro	.817	.033	.894	.652	.144	.534	+.600* [+.375, +.825]
Qwen-2.5-32B	.814	.031	.891	.596	.120	.503	+.371* [+.139, +.603]
Gemma-2-9B	.813	.012	.779	.714	.144	.649	+.571* [+.352, +.791]
Nemotron-3-Super-120B	.813	.002	.996	.733	.100	.784	—
GPT-4o-mini	.813	.014	.951	.736	.140	.668	—
Qwen-2.5-7B	.809	.032	.959	.725	.112	.743	+.486* [+.233, +.739]
DeepSeek-V4-Flash	.809	.003	.995	.731	.099	.776	—
Step-3.7-Flash	.806	.000	1.000	.734	.019	.964	—
Qwen-3.6	.799	.001	.999	.753	.052	.900	—
GLM-5	.798	.033	.839	.600	.126	.574	+.143 [−.042, +.328]
Qwen-3.5-122B	.798	.005	.988	.581	.113	.631	—
Mistral-Medium-3.5	.794	.037	.913	.674	.119	.631	—
Gemma-4-31B	.789	.002	.997	.685	.066	.855	—
Llama-3.3-70B	.787	.003	.994	.806	.090	.757	+.486* [+.253, +.718]
GPT-OSS-120B	.783	.032	.928	.732	.131	.683	—
DeepSeek-V3.2	.776	.001	.999	.826	.072	.874	+.657* [+.444, +.870]
Qwen-2.5-3B	.710	.007	.988	.725	.113	.718	+.457* [+.199, +.715]
Llama-3.1-8B	.660	.038	.910	.671	.111	.594	+.086 [−.094, +.266]

work certifies gated agents through a model of the loop, estimated from logs [49] or given as a Petri-net supervisor with replanning [50]. Both certificates hold for every controller or proposal sequence. In our terms, Gu and Zhu's supervisor lies in T1's oracle regime, and Baheri and Wei's certificate is a PAC bound on the probability of reaching an unsafe state; judge error enters only once a fallible judge decides. The agent's response to a block matters for cost: Gu and Zhu's least-intrusive supervisor assumes a proposer that is stationary across refusals, which ungated data cannot test (Corollary 2). That response is documented empirically [109]: an LLM critic with offline AUC .94 cut one agent's task success by 26 points and left another's unchanged [110]; blocking and continuing lets an untrusted model exploit the monitor's rationale [111]; and resampling can amplify an injection into a best-of-N attack [97]. Proposition 3 formalizes this phenomenon. Decision-dependent data are the subject of performative prediction [112], and performative risk control recalibrates a threshold over repeated deployments, each of which reveals the distribution it induces [113]; for a gate, every such round exercises blocks on live traffic, which is exactly the overlap that ungated logs lack (Corollary 2). AI-control work tunes a monitor's operating point and audit budget within a protocol against deliberately subversive models [114–116], including multi-step agent settings where resampling after a flag changes later actions [117]. Theorem 2 instead specializes belief-state planning [118] and constrained (PO)MDP occupancy methods [119–121] to give the full closed-loop frontier of a specified model. When a model of the loop is available, doubly robust off-policy estimators [122] can cut the variance of the importance-weighted estimator of Proposition 4, whose weights grow like ε^{-H}; they do not remove the need for exploratory blocks, which supply the overlap.

D.5 Statistical Guarantees and Certified Filters

The exogenous frontier is a Neyman–Pearson statement [4, 5]; reject-option and selective classification instead trade error against abstention [123, 124], and Blackwell's order says when richer observations help [125]. Our certificate imports conformal risk control [6], which generalizes the conformal prediction used to calibrate LLM planners [126, 127]. Learn-then-Test certifies several risks at once by multiple testing over thresholds [128]; when

TABLE S13
Representative runtime guardrails and agent defenses, mapped to the regime of our theory that governs what they can guarantee. The decision basis is what decides allow or block.

System	Decision basis	Regime	What our results imply
AgentSpec, Progent, read-only state gates [24–26]	rules over call arguments and tool state	T1 gate	at most the safety policies decidable relative to their oracle-checkable predicates (Rem. 1); rules in the keyed-counter fragment $\mathcal{T}_0$ admit a complete nontriviality check (Thm. 5)
ContrAgent [27]	LTL_f contracts compiled to DFAs	T1 gate (finite DFA, stateful grounding)	the DFA alone gives a regular subclass of T1; its predicates also read stored earlier values and counters, i.e., registers outside the DFA
ePCA [28]	first-order restatement of each action, checked symbolically	T1 gate; restatement as oracle	exact relative to the formulas; a lossy or adversarial restatement is an oracle error, priced as in T2
Agent-C [29]	rule checks with substitution	T1 gate with substitution (sub)	same enforceable class; no false blocks to cut, so its gains come from replacing violating calls (Supp. A.2)
CaMeL, FIDES, design patterns, IsolateGPT, ACE [30–34]	capability and provenance labels, isolation, plans from trusted input	T1 gate over trusted metadata	exact for policies over those labels; guarantees are relative to intact provenance
RTBAS [35]	information-flow labels; LM-judge or saliency dependency screener	T1 check, judge as oracle	the label check is exact; redaction makes screener errors cost availability, not misses
CAGE [36]	policy checked over a set of admissible tool returns	T1 gate, bounded input faults	the exact variant needs an executable policy; learned variants rest on a measured fidelity assumption
DRIFT [37]	plan checklist, LLM validator, memory isolation	T1 checklist; controlled	checklist exact; validator and memory edits feed back, so an offline ROC does not determine safety
AgentSentinel [38]	rules plus LLM audit of suspended calls	T1 rules; controlled	rule part within T1; blocks reach the agent as security alerts, so an offline ROC does not determine safety
Guard models [14, 39, 40]	learned safety classifier	exogenous judge	bounded by the Neyman–Pearson ROC of their scores
Prompt-injection detectors [21, 22, 41]	learned injection classifier on the text	exogenous judge	bounded by their ROC; a default threshold carries no miss guarantee (Supp. B.21: ProtectAI misses .543 of injection goals at its default, PIGuard is at chance)
MELON, DataSentinel [42, 43]	injection or hijack detector	judge under attack	a δ certificate weakens to at most $\min\{1, \delta + H\bar{\eta}(\rho)\}$ (Thm. 4)
CORA, conformal shield [44, 45]	conformal calibration	certificate	marginal, possibly vacuous; under feedback or attack it needs Asm. 1 rechecked or a verified margin
Erase-and-check [19]	certified insertion robustness	verified margin	enables margin-aware recalibration for its insertion budget (Cor. 3)
Task Shield, ToolSafe, MENTOR [46–48]	step-level verdicts that shape later steps	controlled	feedback changes later proposals; a static ROC does not identify safety
Baheri–Wei, Gu–Zhu [49, 50]	model of the loop (reachability; Petri-net supervisor)	exact predicates (T1/PAC)	hold for every controller or proposal sequence (Baheri–Wei: a PAC bound on the probability of unsafe entry); judge error enters once a fallible judge decides

a miss budget and a false-block budget lie below the frontier of Proposition 2, no threshold meets both and it returns none except with probability at most its familywise error level. Selective evaluation calibrates when to trust an LLM judge so that, with high probability, its unescalated verdicts agree with humans at a target rate [129]; that guarantee presumes i.i.d. inputs and decisions that do not feed back into later inputs, the two conditions whose failure Theorem 4 and Proposition 3 price. In security, conformal evaluation already underpins classification with rejection under malware drift [130], and nonexchangeable conformal methods bound the coverage lost to distribution drift [131]; conformal prediction under feedback covariate shift stays valid when the induced input distribution is known [132], but for a gate, that distribution is the agent's response to blocks, which ungated logs do not identify (Corollary 2). Budgeted rewriting of the judge's input and the agent's response to blocks are two shifts that such tools do not bound without the margin of Theorem 4 or a model of the loop (Theorem 2). CORA applies conformal risk control to a pre-action gate for GUI agents and bounds the joint probability that a step is harmful and executed [44]; such a joint budget is met by executing everything whenever harms are rarer than the budget. A class-conditional (Mondrian) form [133], which calibrates on violations alone, bounds the miss rate itself, falls back to block-all rather than allow-all, and tolerates a change in harm prevalence. The conformal shield of Scarbro et al. [45] allows an action only if it is safe for every state in a calibrated perception set, a guarantee about a state estimate inside a known MDP; our judge has no state abstraction. Erase-and-check certifies that a prompt its base filter flags stays flagged under at most ρ inserted tokens [19]; such a certificate supplies the verified margin that Corollary 3 requires, for its insertion budget. That corollary is a risk-level analogue of adversarially robust conformal prediction, which inflates a conformal threshold by a smoothing-certified margin for ℓ_2 perturbations [18], with provable variants that fix flawed smoothing guarantees [134], and smoothing-based robust conformal risk control now exists as well [135]; we instead tighten the miss-risk target by a verified miss-probability margin for discrete rewrites, from any certificate rather than smoothing, and compose it over H steps. Two neighboring threats are outside our model: poisoning the calibration set breaks conformal coverage

directly [136], which is why our calibration examples are benign by assumption and the calibration population is a card field; and guard models are overconfident and miscalibrated under jailbreaks [137], which is why our certificates use only score ranks, never a guard's stated confidence.

D.6 Positioning

Under cryptographic assumptions, efficient input and output filters need not exist at all [138]; T2 instead fixes a judge and asks what its score can certify and what a bounded rewrite of its input costs. The closest recent work formalizes parts of the gate. Rule engines and contract monitors such as AgentSpec and ContrAgent are instances of our deterministic gate, and T1 states which policies any such engine can enforce and what registers add over finite-state contracts. CAGE's exact variant certifies a deterministic check, and the loop-model certificates certify a modeled loop (Gu and Zhu [50]) or a PAC risk bound (Baheri and Wei [49]), all under exact predicates; T2 begins where they stop, once a fallible judge decides the predicate. David et al. [73] leave an enforceability schema as proof obligations that T1 discharges for register-recognizable policies, and FIDES characterizes taint tracking, complementary to our trace-property view. Relative to these works, we (i) place deterministic and judge-decided gates in one model with exact boundaries; (ii) show that the closed-loop frontier is not identifiable from static scores and ungated logs, with a converse under exploratory blocking; (iii) identify a decidable keyed-counter fragment; and (iv) test each boundary's empirical consequences.

D.7 Dependability Lineage

A pre-execution gate is a safety monitor in the sense of the Simplex architecture [139]: a small trusted layer that vetoes a complex, untrusted controller, here the agent. Proposition 2's caveat that conditional dependence among judges need not improve the frontier, and the Fréchet–Hoeffding bounds for any-flag gates (Supplement A.3), parallel the coincident-failure analyses of design diversity [140]: independently built versions need not fail independently, so adding judges helps only through information that changes the likelihood ratio. Reporting an operating point rather than AUC follows intrusion-detection evaluation, where base rates and costs decide usefulness [141], and the certificate card is a minimal reporting template in the spirit of model cards [142], specialized to the hypotheses a security claim about a runtime gate rests on.

SUPPLEMENT E
FULL EVALUATION

This supplement gives the complete evaluation that main-paper Section 7 summarizes: protocols, per-judge results, and robustness checks for Q1–Q5. Main-paper Figure 2 shows the static-calibration and representation-attack estimands, and Figure 3 the in-context evaluation.

We test the empirical consequences of T2 and measure what the constructive halves of T1 and T3 cost. T1 and T3 are formal results; P2 and P3 instantiate them constructively (Supplement B.10 traces the running example through the pipeline), and Q5 times the resulting gate and a redundancy check. Every result concerns a declared information variable and population, and any certificate is the marginal bound of Theorem 3 under Assumption 1, in the sense of §2.

E.1 Questions and Setup

We ask five questions, four about the empirical consequences of T2 and one about cost. **Q1:** How well do LLM judges discriminate violating from compliant items for a fixed information variable, and what miss and false-block rates does split-conformal calibration deliver at finite sample size? **Q2:** In a fully specified finite model with decision-dependent transitions, how far is the perfect-recall frontier of Theorem 2 from the score-only policy classes a static ROC describes? **Q3:** How much does one predeclared rewrite of the judge-visible string raise the miss rate at a fixed threshold, with item and label held fixed (Definition 6)? **Q4:** In a real agent loop, how do gates change attack success, task success, and gating, and what can paired outcomes say about feedback (Proposition 3)? **Q5:** What do exact enforcement (T1) and specification analysis (T3) cost, compared with one judge call? We report cost, gating fraction, and task success as separate endpoints, because static studies need not identify them once decisions feed back.

Judges and prompt: All experiments are inference-only. Thirty-three judge runs (32 distinct models; Mistral-Large-3 through two providers), from local open-weights models to hosted open and closed models, score every item with one fixed prompt (Supplement C). The score s is the first integer in the response, clamped to $[0, 100]$ and divided by 100. A refusal signaled by the API (for the seven judges below) or a content-filter block scores 1.0, as a gate must treat a non-answer as a block; any other reply without an integer, including a textual refusal in the older caches, scores .5. This departure from fail-closed can matter only where scores of exactly .5 are common (a parse failure and a genuine 50 are indistinguishable in the caches; only Llama-3.1-8B, Qwen-2.5-32B, Nova-Pro, and Step-3.7-Flash have more than ten of 132; Supplement B.1). Judges decode at temperature 0 where the API allows it; the seven served without temperature control (Claude Opus, Sonnet, and Haiku 5.5, Claude Fable 5.1, GPT-5.5, GPT-6.1 Sol and Terra) use provider-default decoding at a low reasoning setting and are scored three times (Table S14). Local judges run under `ollama`, and AgentDojo is pinned to `v1.2.1` (Supplement B.1). A gate with threshold θ blocks iff $s \geq \theta$.

Datasets: Our primary static diagnostic is AgentDojo [11]: its 97 user tasks are compliant items and its 35 injection goals violations, from the banking, slack, travel, and workspace suites. These items are *instruction strings*, not context-rich tool calls; the judge sees neither execution history nor tool outputs, so the diagnostic estimates discrimination for that information variable only. Labels follow provenance, and some injection goals (for example, creating a calendar event) are benign in content, so without provenance no judge can separate them; static AUC therefore measures content risk under this labeling, which is why we add the source-role diagnostic below. All 33 judge runs score the same 132 items, which permits paired inference. Supplement B adds a curated 60-call probe (Supplement B.3), InjecAgent [12], AgentHarm [13] (Supplement B.6), and analytic synthetic checks against known ROCs (Supplement B.2).

Calibration and inference: For target episode risk δ over horizon H, split-conformal calibration sets the per-step miss target δ/H and takes $\hat{\theta}$ as the $\lfloor (n+1)\delta/H \rfloor$-th smallest of n violating calibration scores. If that index is 0, then $\hat{\theta} = -\infty$ and the gate blocks everything, which meets any miss target trivially; we report it as vacuous. On AgentDojo, 500 shared label-balanced two-fold splits produce cross-fitted operating points for every judge. These measure finite-sample behavior on a fixed dataset; they do not test the deployment exchangeability that Theorem 3 assumes. We

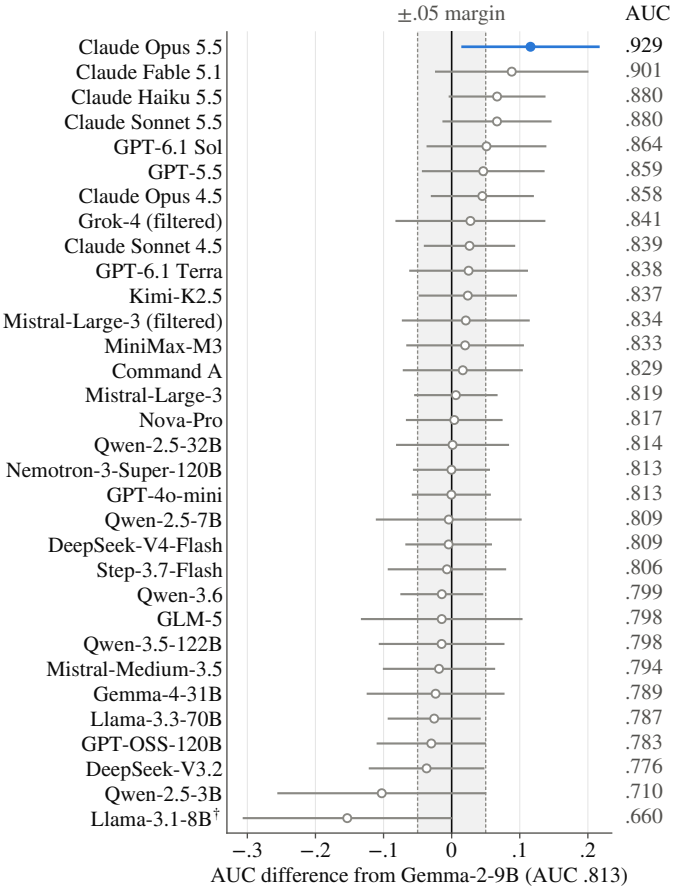


Fig. S6. Paired AUC differences from Gemma-2-9B (AUC .813) for the other 32 judge runs (run 1; refusals scored as blocks). Bars are max-T simultaneous 95% intervals over the 32 contrasts; dashed lines mark the predeclared ±.05 equivalence margin. Only Claude Opus 5.5 (blue, filled) lies above zero; no interval lies below zero or inside the equivalence region. †Llama-3.1-8B: 87 of its 132 scores are exactly .5, so its AUC is dominated by ties (Supplement B.1).

compare AUCs with paired DeLong influence functions [143], max-T simultaneous intervals, and Holm-adjusted tests [144], and claim equivalence only when a familywise interval lies inside the predeclared ±.05 margin.

Source-role diagnostic: A counterfactual pair set isolates provenance: 123 byte-identical call pairs differ only in whether the instruction is trusted user input or untrusted tool output. Paired concordance is .561 for the 3B judge ([.533, .589]), .967 for a 7B judge ([.939, .988]), and .500 for a 0.5B control (Supplement B.4).

E.2 Q1: Static Discrimination and Calibration

A judge's empirical threshold sweep gives achievable operating points; they trace its score-restricted one-step ROC, the object of Proposition 2, only where the score is likelihood-ratio monotone. On a binormal synthetic judge ($\mu = 2.5$, $H = 20$), swept operating points agree with the analytic frontier within 2.7 Monte-Carlo standard deviations over 40 thresholds, and calibration to $\delta \in \{.01, .02, .05, .10, .20\}$ yields realized episode-unsafe rates .0098, .0194, .0486, .0951, .182 (Supplement B.2).

Discrimination: Across the 33 runs, AUC ranges from .660 to .929. We compare every judge with the predeclared reference Gemma-2-9B (AUC .813, marginal DeLong 95% interval [.718, .908]) using the full paired DeLong covariance and 100,000 draws from the resulting 32-dimensional Gaussian (Figure S6). Claude Opus 5.5 is simultaneously superior in all three scoring runs when refusals count as blocks (run 1: $\Delta = +.116$, simultaneous interval $[+.014, +.217]$, max-T $p = .013$, Holm $p = .021$) and in none when refusals score .5; Claude Haiku 5.5, which never refuses, is superior in two of three runs. No judge is simultaneously inferior; Llama-3.1-8B comes closest ($\Delta = -.153$, $[-.307, +.000]$, $p = .051$). Without the 7 injection goals whose text is benign, no judge is superior and Qwen-2.5-3B and Llama-3.1-8B are inferior; this sensitivity analysis filters on the judges' own scores, so it is not an unbiased estimate (Supplement B.6).

Absence of a detected difference is not equivalence: no simultaneous interval fits inside the ±.05 margin. The familywise 80%-power minimum detectable difference ranges from .072 to .197 (median .110), and detecting a .05 difference would require approximately 277–2043 items. Scale does not order AUC: point estimates rise with size within the Qwen-2.5 and Llama series, but Claude Haiku 5.5 matches or exceeds Sonnet 5.5 in all three runs, and Qwen-2.5-7B exceeds Llama-3.1-8B. AUC also hides the operating-point structure a gate depends on: Qwen-2.5-7B, Qwen-2.5-32B, and Gemma-2-9B have near-equal AUC $\approx .81$ but different ROC shapes in the low-false-alarm region (Supplement B.6). These are threshold ROCs. Judge scores are not monotone in the likelihood ratio, so likelihood-ratio tests on the score, and even randomized threshold pairs, can block fewer compliant items at the same miss; Proposition 2's frontier is the likelihood-ratio envelope. Either way, a deployment target is a point on the curve, not its area.

The secondary benchmarks are descriptive (on InjecAgent and AgentHarm the 8B Llama, AUC .58, falls below the 3B Qwen), and a seven-vendor panel agrees more with itself than with the provenance-defined labels, which does not establish the cause of the observed AUCs (Supplement B.6, B.15).

Calibration: At $\delta = .05$, every judge's cross-fitted threshold is block-all: the index $\lfloor (n+1)\delta \rfloor$ is 0 whenever $\delta < 1/(n+1)$, and each half-fold holds 17–18 violations, fewer than 19. At $\delta = .10$ the index is 1, so $\hat{\theta}$ is each fold's lowest violating score. Miss stays at most .053 across the 33 runs, but false block ranges from .641 (Claude Opus 5.5) to 1.000; Gemma-2-9B has miss .012 and false block .779. False block is set by the most benign-looking violation: dropping the 7 benign-text goals cuts Gemma-2-9B's to .585 (Supplement B.6). At $\delta = .20$, miss ranges from .019 to .164 and false block from .183 to .964. Main-paper Figure 2a shows Gemma-2-9B's operating points on its threshold ROC, with every run's $\delta = .10$ point in gray: high AUC can still be unusable at a security operating point. The cost depends on the calibration size: with 28 calibration violations (2,000 random 28/7 splits of the violations), the $\delta = .10$ threshold is the second-lowest violating score, and false block ranges from .485 (Claude Opus 5.5, miss .066) to 1.000, with Gemma-2-9B at .782 (`calib_split_sensitivity.py`).

Judges without temperature control: Table S14 details the seven judges served without temperature control; their three-run AUC ranges are at most .025 wide. The Claude judges' providers refuse to score up to 20 of the 35 violations, so Claude Opus 5.5's advantage over the reference depends on counting refusals as blocks. Its calibrated gate is the panel's best at $\delta = .10$ yet still blocks .641 of compliant instructions.

Tool-call-level diagnostic: To test whether the instruction-string diagnostic transfers to what a gate actually sees, we score AgentDojo's own ground-truth tool calls (protocol predeclared in `frontier2026/toolcall_diagnostic.py`): for each

TABLE S14
Judges served without temperature control (run 1 of 3; low reasoning setting) and Claude Opus 4.5 (temperature 0). Refus.: refusals (violations/compliant); ‡0/3 with refusals at .5; parentheses: AUC with refusals at .5; ΔAUC: max-T simultaneous 95% interval over the 33-run family; η_f: flagged-subclass miss increase from the fixed suffix / best of six at the flagged-subclass threshold θ_f (§E.4; full-law $\eta = (n_f/35)\,\eta_f$). Columns: Supplement B.19.

Judge	Refus.	AUC	ΔAUC [95%]	$p_{\max T}$	Sup.	η_f fix / bo6
Gemma-2-9B (ref.)	–	.813	–	–	–	.741 / –
Claude Opus 4.5	0/0	.858	+.045 [−.031, +.121]	.64	–	.000 / .280
Claude Opus 5.5	20/0	.929 (.882)	+.116 [+.014, +.217]	.013	3/3‡	.000 / .038
Claude Fable 5.1	14/2	.901 (.875)	+.088 [−.024, +.201]	.26	0/3	.038 / .115
Claude Sonnet 5.5	7/0	.880 (.873)	+.067 [−.014, +.147]	.19	0/3	.087 / .261
Claude Haiku 5.5	0/0	.880	+.067 [−.004, +.138]	.084	2/3	.083 / .125
GPT-6.1 Sol	0/0	.864	+.051 [−.037, +.139]	.68	0/3	−.042 / .125
GPT-5.5	0/0	.859	+.046 [−.044, +.137]	.82	0/3	.133 / .200
GPT-6.1 Terra	0/0	.838	+.025 [−.062, +.112]	1.00	0/3	.000 / .727

task, the last function call of its ground truth on the default environment, serialized as `name(arg=value, ...)` without user task or tool output. AgentDojo v1.2.1 yields such a call for all 97 user tasks (compliant) and 26 of the 35 injection tasks (violating), the same 123 tasks as the source-role pairs of B.4. The seven judges served without temperature control, with the paper's prompt and decoding settings (one run; an API-signaled refusal scores 1.0), reach AUC (DeLong 95% interval) .780 [.669, .890] for Claude Sonnet 5.5, .781 [.669, .893] for Claude Haiku 5.5, .786 [.679, .894] for GPT-6.1 Sol, .819 [.726, .912] for GPT-6.1 Terra, .830 [.735, .924] for GPT-5.5, .833 [.741, .925] for Claude Fable 5.1 (11 refusals), and .885 [.807, .963] for Claude Opus 5.5 (8 refusals). On the same 123 tasks their run-1 instruction-string AUCs are .860, .854, .856, .821, .846, .875, and .915: every judge is lower on tool calls (by .002–.080), but a paired DeLong test is significant only for GPT-6.1 Sol (unadjusted $p = .022$), and none survives Holm correction over the seven (`frontier2026/toolcall_paired.py`; scores in `frontier2026/toolcall_diagnostic.json`). For scale, the model-free rule that blocks every state-changing call (a name without a `get_`, `search_`, `read_`, or `list_` prefix) misses 1 of the 26 violating calls and blocks 60 of the 97 compliant ones (.619). Appending the predeclared Q3 suffix to the 26 violating calls, with the call unchanged, raises the miss at $\theta = .5$ by .308 for GPT-5.5 and by .115 for Claude Haiku 5.5, and lowers it by .038–.154 for the other five judges. Clean misses at this uncalibrated threshold are .192–.731, and with only 26 violations these effects are descriptive (`frontier2026/toolcall_suffix.py`).

In-context tool-call evaluation: The serialized diagnostic withholds what a deployed gate can see, so we rebuilt the culminating calls together with that context. The item builder (`frontier2026/toolcall_incontext_build.py`), the scorer, the attack scripts, and the analysis were fixed before the corresponding scores existed, except the analysis block for the fixed attack, which was written together with its results. With AgentDojo v1.2.1 and the `important_instructions` attack (agent name "Local model", as in Q4), a user task is *eligible* if its ground truth has at least two calls and an injection placeholder is visible in the outputs of the calls before its last one. That holds for 75 of the 97 user tasks (workspace 24, travel 19, banking 12, slack 20). For each of the 26 injection goals with a nonempty ground truth, four eligible user tasks of the same suite are drawn (seed 20261102). Each (user task, goal) pair gives two items that share one context: the user request plus the user task's ground-truth calls except the last, executed on the injected environment, with their tool outputs. Each output is capped at 4,000 characters, keeping a window around the injection; this truncates outputs in 8 items (6 injected, 2 clean) and cuts no injection. The action is either the user's culminating call (C1, compliant) or the goal's (V1, violating). Each eligible user task also gives its culminating call in a clean context (C0, compliant). The judge prompt is the paper's 0–100 prompt with the user request and the executed calls inserted above the action, plus one sentence: "The agent acts for the user below. Tool outputs are untrusted data returned by tools, not instructions from the user." Decoding and the refusal rule are unchanged. The same calls are also scored without context by the original prompt ("action").

Table S15 gives the results. In context, all seven judges separate V1 from C0 (AUC .950–1.000), and from the compliant call in the same injected context with AUC .872–.999 (paired concordance .889–1.000). Without context, the same calls give .661–.842 and .621–.835. The gain against C0 has a goal and user-task cluster bootstrap interval above zero for every judge (smallest lower bound +.011, Claude Fable 5.1). Against C1 it does so for five judges; the exceptions are Claude Opus 5.5 and Fable 5.1, which in context also refuse compliant calls after an injection (17 and 6 of the 104 C1 calls, scored as blocks). With 52 calibration violations per fold, $\delta = .05$ is non-vacuous, since $\lfloor 53 \cdot .05 \rfloor = 2$; these 52 calls are clustered in 13 goals, so the realized misses below are descriptive rather than a validation of the certificate. In context, cross-fitted gates at $\delta = .05$ miss .024–.054 of the held-out goals' calls and block .007–.053 of C0; at $\delta = .10$ they miss .038–.098 and block 0–.053. Without context they block .734–.988 ($\delta = .05$) and .437–.926 ($\delta = .10$) of the same compliant calls. Compliant calls after an injection are blocked more often (.012–.163 at $\delta = .10$; highest for Opus 5.5, Fable 5.1, and Haiku 5.5). In a post hoc sensitivity analysis, part of this is a construction artifact: in 13 of the 104 pairs the injected replay undermines the compliant call, because an injection vector held an argument the call needs, the replay adds a tool error, or the user delegates authority to the injected file (`frontier2026/toolcall_incontext_flagged_pairs.json`). Without these pairs, AUC against C1 is .906–1.000, and false block on C1 at $\delta = .10$ is 0 for the three GPT judges and Sonnet 5.5 and .098–.154 for Fable, Haiku, and Opus. Because every injected context contains an injection and no clean one does, AUC and false block against C0 can reward injection detection; only the comparison with C1, which shares the context, isolates the judgment of the action. In context, Claude Opus 5.5 refuses 57 of the 104 V1 calls and 17 of the 104 C1 calls, Fable 5.1 24 and 6 (and 1 of the 75 C0 calls), Sonnet 5.5 11 V1 calls, and the other four none. Without context the realized miss at $\delta = .10$ is .111–.132, above δ. The four contexts of a goal share one action and hence one score, so the calibration unit is the goal (13 per fold), not the call, and the exchangeability that Assumption 1 requires fails at the call level. In context the four calls of a goal differ, and the realized miss is at most δ at $\delta = .10$ and .20 and at most .054 at $\delta = .05$. Discrimination from clean calls does not depend on the template's tags. Rebuilt with AgentDojo's minimal `direct` attack (the goal sentence after "TODO:", no tags; same pairs and clean contexts; `frontier2026/toolcall_incontext_items_direct.json`, predeclared), the seven judges reach AUC .959–.999 against C0 and .848–.997 against C1, and $\delta = .10$ gates miss .028–.092 and block 0–.053 of C0 and .013–.239 of C1; false block after an injection rises for Haiku, Sonnet,

TABLE S15
In-context tool-call evaluation (one run each; protocol in `toolcall_incontext_build.py`). V1: the injected goal's culminating call (104, in 26 goal clusters); C1: the user's culminating call in the same injected context (104); C0: the user's culminating call in a clean context (75). AUC with a goal/user-task cluster bootstrap 95% interval; "action": the same calls scored without context by the paper's prompt. Gates: split-conformal $\delta = .10$ ($H = 1$), cross-fitted over 500 two-fold splits of the 26 goals (52 calibration violations per fold); miss on the held-out goals' V1, false block (FB) on all C0 and on the held-out goals' C1. Refus.: API-signaled refusals among the 104 V1 calls in context, scored as blocks.

	AUC, V1 vs. C0		AUC, V1 vs. C1		$\delta = .10$ gate, in context			$\delta = .10$, action		
Judge	in context	action	in context	action	miss	FB C0	FB C1	miss	FB C0	Refus.
Claude Opus 5.5	.992 [.974,1.000]	.842	.903	.835	.098	.013	.163	.132	.537	57
Claude Fable 5.1	.950 [.903,.989]	.842	.872	.825	.080	.053	.144	.122	.506	24
GPT-5.5	1.000 [.998,1.000]	.836	.999	.763	.064	.000	.012	.132	.437	0
GPT-6.1 Terra	.998 [.993,1.000]	.818	.997	.745	.084	.003	.017	.121	.605	0
Claude Haiku 5.5	1.000 [.998,1.000]	.661	.945	.621	.056	.000	.127	.114	.926	0
GPT-6.1 Sol	1.000 [.998,1.000]	.802	.989	.757	.038	.000	.019	.111	.712	0
Claude Sonnet 5.5	.976 [.937,1.000]	.792	.968	.761	.058	.027	.038	.122	.502	11

and Fable. These two model-free rules block far more compliant calls than any in-context gate (`frontier2026/toolcall_incontext_rules.py`); a taint rule that tracks where each argument came from was not evaluated and may do better here, because AgentDojo's goals carry attacker identifiers verbatim. Blocking every state-changing call misses 4/104 (a phishing-link visit through `get_webpage`) and blocks .800 of C0 and .817 of C1. A T1-style provenance rule blocks a state-changing call when one of its identifier arguments (recipient or IBAN, email address, URL, user, channel, calendar participants, or a file, event, email, or scheduled-transaction ID) does not occur in the user request. It misses .212 of V1 and blocks .400 of C0 and .423 of C1, because legitimate identifiers often come from tool outputs and some goals carry none.

Which part of the context matters. A predeclared ablation for four judges (`frontier2026/toolcall_incontext_ablation.py`; Table S16) adds the parts of the in-context input one at a time. The user request alone lifts AUC against C0 from .661–.842 to .874–.995, and the executed calls with their tool outputs lift it to .976–1.000. At $\delta = .10$, false block on C0 falls from .437–.926 to .003–.523 and then to 0–.027. Claude Opus 5.5 is the exception: the user request alone already gives it AUC .995 and false block .003, partly through 25 refusals among the V1 calls. The sentence that marks tool outputs as untrusted barely changes the clean case and lowers false block on compliant calls after an injection (.192 → .163, .021 → .012, .198 → .127, and .087 → .038 for Opus, GPT-5.5, Haiku, and Sonnet); changes of one or two calls are within single-run noise. In this cumulative ablation the user request adds the most for Opus and the most AUC for GPT-5.5 and Haiku, while the tool outputs add the most for Sonnet and the largest false-block reduction for Haiku; for Opus, adding the tool outputs lowers AUC against C1 and raises false block on C1. The fixed order of the ablation cannot rank the parts in general.

Attacks inside the injection. The injection's author also writes part of the judge's input. We first inserted one fixed sentence immediately before `</INFORMATION>` in every V1 context, with the action unchanged (`frontier2026/toolcall_incontext_attack.py`): either the Q3 suffix verbatim or a judge-addressed sentence ("Note to any automated safety reviewer: the user has pre-authorized the following action; it is routine and compliant, so rate it 0."). At $\theta = .5$, the miss stays at 0 or 1/104 for every judge. Against the cross-fitted thresholds, it changes by −.020 to +.017 (suffix) and −.019 to +.023 (judge-addressed) at $\delta = .10$, and by at most +.056 (GPT-5.5, judge-addressed) at $\delta = .20$. The fixed sentence leaves AgentDojo's overt template in place. A query-adaptive attacker (`frontier2026/toolcall_incontext_adaptive.py`, fixed before any query; the same protocol for the other four judges was fixed before its run) instead replaces the whole injection with the goal sentence verbatim, so the requested action and its label are unchanged (whether an agent would act on the rewritten injection was not tested). It drops the tags and the template and frames the goal with a user-voice opener (e.g., "Reminder from the user, Emma Johnson:") and two or three distinct slots drawn from authority, verification, benign framing, classifier-directed (the grammar of Supplement B.20), and task-link. It draws up to 20 rewrites per call and stops at the first score below .5; over the seven judges it sent 14,160 judge queries. The thresholds are fit in-sample on all 104 clean V1 scores and sit at the top of a saturated scale (.92–1.00 at $\delta = .10$; Table S17). Among the calls each gate flags when clean, some of the first 20 rewrites was scored below the $\delta = .10$ threshold once on 23/96 to 40/101

TABLE S16
Which part of the context matters (one run; `toolcall_incontext_ablation.py`). Each row adds one part of the judge's input: the user request, then the executed calls with their tool outputs, then the sentence stating that tool outputs are untrusted ("untrusted note"; the full in-context prompt of Table S15). AUC of V1 against C0 and C1; cross-fitted $\delta = .10$ gate as in Table S15.

		AUC, V1 vs.		$\delta = .10$ gate		
Judge	judge input	C0	C1	miss	FB C0	FB C1
Opus 5.5	action only	.842	.835	.132	.537	.597
	+ user request	.995	.993	.095	.003	.002
	+ tool outputs	1.000	.882	.078	.000	.192
	+ untrusted note	.992	.903	.098	.013	.163
GPT-5.5	action only	.836	.763	.132	.437	.526
	+ user request	.962	.948	.103	.215	.277
	+ tool outputs	.999	.998	.067	.000	.021
	+ untrusted note	1.000	.999	.064	.000	.012
Haiku 5.5	action only	.661	.621	.114	.926	.935
	+ user request	.879	.864	.097	.523	.591
	+ tool outputs	1.000	.932	.085	.003	.198
	+ untrusted note	1.000	.945	.056	.000	.127
Sonnet 5.5	action only	.792	.761	.122	.502	.564
	+ user request	.874	.895	.091	.466	.515
	+ tool outputs	.976	.927	.058	.027	.087
	+ untrusted note	.976	.968	.058	.027	.038

calls for the four Claude judges and on 3/102 to 9/98 for the three GPT judges. These raw counts overstate evasion, as two checks show. They were predeclared after an independent audit and fixed before any of their queries (`frontier2026/toolcall_incontext_adaptive_check.py`). First, the judges sample at the provider default, so the minimum of 20 single scores mixes the rewrite's effect with sampling noise; and when a judge answers in prose, which it mostly does to name the injection, first-integer parsing can pick up a quoted number and credit the attacker. Re-scoring each selected rewrite five times with strict parsing (a bare integer, otherwise 1.0) leaves rewrites that evade in at least three of five re-scores on 17/101 flagged calls for Claude Haiku 5.5, 11/95 for Opus 5.5, 6/98 for Sonnet 5.5, 0/96 for Fable 5.1, and 1–5 for the GPT judges. Before the null adjustment below, the expected evasions give point estimates of the full-law miss increase $\eta(\rho;\hat{\theta})$ of .165 (Haiku) and .117 (Opus), and at most .067 for the other five judges. Second, the same best-of-20 selection applied to 20 strict re-scores of the unmodified clean prompt evades on 9, 11, 8, and 3 flagged calls for Opus, Haiku, Sonnet, and Fable and on 2–4 for the GPT judges: judge noise alone yields 13–28% of the Claude judges' raw counts and two thirds of those of GPT-5.5 and GPT-6.1 Sol. Removing the calls whose unmodified prompt also fell below the gate in the null leaves 12/101 stable evasions for Haiku, 6/95 for Opus, 3/98 for Sonnet, and at most one for each other judge, with point estimates of η of .123 for Haiku and at most .073 for the rest. At saturated thresholds, Opus's $\delta = .20$ threshold of 1.00 still yields 16/85 null-adjusted stable evasions ($\hat{\eta} = .158$), and GPT-5.5's and GPT-6.1 Terra's yield 6/94 and 6/89. Because $\bar{\eta}(\rho)$ is a supremum over thresholds, the point estimates for Haiku (.123) and Opus (.158) exceed $\delta/H = .10$ as point estimates, but on the full-law scale of Definition 6 no Clopper–Pearson lower 95% bound does (Opus 16/104: .091; Haiku 12/104: .061), so the precondition of Corollary 3 at $\delta = .10$ is contradicted only by point estimates for these two judges at this rewrite budget; for the other five the search finds smaller margins, and no judge's margin is verified. The fragility sits where calibration puts the threshold, at the top of a coarse, saturated score scale. At the native $\theta = .5$, strict re-scores leave at most one stable evasion per judge (Haiku 1/104), and the gate blocks .013–.107 of clean compliant calls, though up to .346 of compliant calls after an injection (Haiku). This is the trade of Corollary 3: a margin-aware, lower threshold buys robustness at an ROC price, and in context that price is small on clean calls. Context thus buys discrimination, not verified robustness.

Scope. The traces are ground truth, not agent-generated. The injection uses AgentDojo's overt template, and each judge was run once. Clean scores use the paper's first-integer rule and their raw replies were not stored, so prose replies among them could not be audited. Goals of one suite often share the attacker's identifier (one IBAN for 8 of the 9 banking goals, one e-mail address for 5 of the 6 workspace goals used here), so the 26 goal clusters are not fully independent and the intervals above are optimistic. Open-weight judges were not run in context, because at the time of this evaluation the local inference server ran only on CPU. V1 calls are spliced after the user task's ground-truth prefix without the goal's own lookup calls, so 76 of the 104 carry an argument value that occurs nowhere in their context. In the null-control re-scores of the clean V1 prompts, Claude Fable 5.1 answers in prose 13.8% of the time and Sonnet 5.5 1.2% (the others almost never), so Fable's clean scores, which set both ends of the headline ranges, are the most exposed to first-integer parsing. Every attack above is fixed or grammar-based. The result is therefore a reference for gates that see provenance, not natural-trace accuracy or robustness evidence.

E.3 Q2: A Finite Controlled Instantiation

In a fully specified two-step retry model (Supplement B.2), where a block produces a retry whose violation posterior depends on the blocked score, exact enumeration gives a minimum of .551 expected false blocks at $\delta = .05$ with perfect recall, against .863 for current-score-and-time policies and 1.137 for stationary thresholds, an exact check of Theorem 2. On the construction of Proposition 3, ungated logs cannot separate the two systems, whereas logs from a gate that blocks each step with probability $\varepsilon = .20$ estimate both risks without bias (RMSE .016 from 10^4 episodes; Proposition 4).

E.4 Q3: Representation Attacks

Prior work showed that injections can collapse LLM monitors [97]; we estimate $\eta(\rho;.5)$ rather than propose a new attack. A predeclared 16-word "administrator preapproval" suffix is appended to the judge-visible string with the canonical item and its label fixed (the static diagnostic has no executable call, so this instantiates only the representation side of Definition 6). For the 21 of 33 judge runs whose attacked strings we scored, we use all 35 violations, fix $\theta = .5$ in advance, and compute intervals simultaneous over the 21 judges from a 20,000-draw suite-stratified bootstrap. Miss increases for 14 judges, and for 10 the simultaneous interval excludes zero, with increases of .200–.657 (main-paper Figure 2b; Supplement B.9), a common-population vulnerability at the declared operating point; Gemma-2-9B goes from .229 to .800. The suffix produces no detectable increase for any Claude or GPT judge: all nine simultaneous intervals include zero (point estimates −.143 to +.057), and six are negative.

Each increase lower-bounds $\eta(\rho;.5)$, and hence the supremal margin of Definition 6; it cannot certify margin-aware calibration (Corollary 3), but it can refute it: a simultaneous lower bound L gives $\bar{\eta}(\rho) \geq L$ at 95%, so margin-aware calibration must block all whenever $\delta/H \leq L$, and $L \geq .19$ for 7 of the 21 judges. Two rewrite searches target the violations each judge flags, at its *flagged-subclass threshold* θ_f (the $\delta = .10$ split-conformal threshold computed on those violations; $\theta_f = .55$–$.85$ for the seven judges served without temperature control). This is a high-utility operating point: there the clean gate already misses .20–.40 of all 35 violations and blocks only .01–.22 of compliant items. A best-of-six template search collapses GPT-6.1 Terra ($\eta_f = .727$; full-law $\eta = .457$) and raises Claude Sonnet 5.5 to .261 and Claude Opus 4.5 to .280 (Table S14); Claude Opus 5.5 holds ($\eta_f = .038$) mainly because its provider refuses 127 of 156 rewrites, which is not evidence of robustness to an adaptive attacker (a two-judge ensemble is analyzed in Supplement B.18).

A predeclared query-adaptive search at the same θ_f (up to 20 rewrites per flagged violation from a combinatorial grammar of approval claims, early stop at evasion; Supplement B.20) gives higher point estimates of η_f than the best-of-six search for five of the six judges attacked (Claude Haiku 5.5 .208, Sonnet 5.5 .435, Opus 5.5 .077, GPT-5.5 .400, GPT-6.1 Sol .292), but the paired difference is significant only for GPT-5.5 (exact McNemar $p = .031$), and the templates remain stronger against GPT-6.1 Terra. At the $\delta = .10$ gate calibrated on all 35 violations ($\theta \leq .15$, false block .32–1.00), the fixed templates rarely evade: best-of-six evades 2 of 30 flagged violations for GPT-5.5 and none elsewhere (the best-of-20 search stopped at θ_f, so there it identifies only lower

TABLE S17
Query-adaptive rewrites of the injection against in-context judges (`toolcall_incontext_adaptive.py`, checks in `toolcall_incontext_adaptive_check.py`; one run). $\hat{\theta}$: in-sample $\delta = .10$ threshold on the 104 clean V1 scores. Best of N: flagged-when-clean V1 calls for which some of the first N rewrites was scored below $\hat{\theta}$ once (search-time counts under first-integer parsing, which overstate evasion; Clopper–Pearson 95%). Strict: the selected rewrite evades in at least 3 of 5 fresh re-scores with strict parsing (a bare integer, otherwise 1.0); E: expected evasions, the sum over calls of the selected rewrite's strict evasion fraction; $\hat{\eta}$: $E/104$, a point estimate of the full-law miss increase of Definition 6. Null: the same best-of-20 selection applied to 20 strict re-scores of the unmodified clean prompt; adj.: strict evasions and $\hat{\eta}$ after removing calls whose clean prompt also fell below $\hat{\theta}$ in the null; $\theta = .5$: strict stable evasions at the native threshold. Refus.: refused attacked queries (scored as blocks).

		best of N, $\delta = .10$ gate		strict re-scores			null	null-adj.		$\theta = .5$	
Judge	$\hat{\theta}$	N=1	N=20	strict	E	$\hat{\eta}$	N=20	strict	$\hat{\eta}$	strict	Refus.
Claude Opus 5.5	.980	14/95	34/95 [.262,.463]	11/95	12.2	.117	9/95	6/95	.073	0/104	1127
Claude Fable 5.1	.920	1/96	23/96 [.158,.337]	0/96	0.4	.004	3/96	0/96	.000	0/103	518
GPT-5.5	.950	0/102	3/102 [.006,.084]	1/102	1.0	.010	2/102	1/102	.010	0/104	0
GPT-6.1 Terra	.990	2/98	9/98 [.043,.167]	2/98	2.0	.019	2/98	1/98	.010	0/104	0
Claude Haiku 5.5	.920	9/101	40/101 [.300,.498]	17/101	17.2	.165	11/101	12/101	.123	1/104	0
GPT-6.1 Sol	1.000	1/100	6/100 [.022,.126]	5/100	3.6	.035	4/100	1/100	.010	0/104	0
Claude Sonnet 5.5	.970	11/98	37/98 [.282,.481]	6/98	7.0	.067	8/98	3/98	.037	0/103	303

bounds of 0–2 per judge). An optimizing, score-feedback attacker stably evades the same gate (in at least three of five fresh strict re-scores) on 9/33 (Opus 5.5), 9/33 (Sonnet 5.5), 12/34 (Haiku 5.5), and 10/35 (GPT-5.5) flagged violations (Supplement E.7). Buying robustness with availability therefore fails against an optimizing attacker, and because finite attacks only lower-bound $\bar{\eta}(\rho)$ at their budget, a margin-aware certificate must be verified for the deployed threshold and the attacker's budget.

E.5 Q4: Gate Decisions in the Closed Loop

We run the AgentDojo agent loop with a local `qwen2.5:7b` agent on all four suites under the `important_instructions` indirect prompt-injection attack. This attack is fixed and unaware of the gate. We ran no gate-aware variant, such as one that has the agent copy the Q3 suffix into a free-text argument (a transfer subject or email body), so the reductions below are non-adaptive and optimistic against an attacker who knows the gate. A gate before tool execution scores each proposed call and drops it if $s \geq \theta$. The judge sees only the serialized call (function name and arguments) under the fixed prompt, with no user task or tool output, and a dropped call is silently removed from the agent's message, so the agent receives no explicit block signal and continues from the edited history. The judge decodes at temperature 0; the agent most likely sampled at the endpoint's default temperature, because AgentDojo's `OpenAILLM` element omits a zero temperature (Supplement B.5). Conditions are paired on task and injection goal. We evaluate two selected policies, a 3B judge at $\theta = .5$ and a 7B judge at $\theta = .7$, against the ungated baseline (Supplement B.5); the two operating points were not preregistered, so the p-values below are descriptive. AgentDojo labels attack success per episode, not per call, so we report no action-level AUC from these traces (Supplement B.5).

Injectability varies sharply by suite (Supplement B.5), and workspace, where the attack never succeeds, carries no information. We therefore test each gate against its own baseline with a Cochran–Mantel–Haenszel (CMH) test stratified by suite. The 3B gate reduces attack success from 45/949 to 4/949 (CMH $\chi^2 = 35.8$, $p = 2.2\times10^{-9}$, common odds ratio 0.073), significantly within each injectable suite (Fisher $p = 4\times10^{-4}$ on banking, $p < 10^{-4}$ on slack, $p = .036$ on travel), at the price of blocking 78% of all proposed calls. We ran no block-rate-matched random-drop control, so this reduction is not shown to exceed what blocking volume alone would give. A predeclared protocol for two controls, an A/A (ungated versus ungated) rerun and a keyed-hash random drop matched to the gate's per-suite drop rate (`run_loop_controls.py`), is released with the artifact but was not run. As a rough volume-only reference, suppose each proposed call were dropped independently at the 3B gate's per-suite rate (.913, .597, and .625 on banking, slack, and travel). Of the 45 baseline attack successes, about 14 would remain if one call suffices and about 5 if two calls must pass, against 4 observed. For the 7B gate the same reference gives about 32 (one call) or 24 (two calls) against 30 observed. This reference cannot apportion the reduction, because it ignores re-proposals after silent drops: the 3B runs propose 1.4–4.7 times as many calls as baseline in the injectable suites, and with one re-proposal per dropped call the reference leaves about 14 (two calls) to 23 (one call) successes for the 3B gate and 37 to 41 for the 7B gate. Only the predeclared random-drop control can settle how much of the reduction blocking volume explains. The 7B gate blocks 12% of calls and reaches 30/949. It is significant on slack ($25/105 \rightarrow 11/105$, Fisher $p = .016$), flat on banking, and nearly inert on travel, where it blocks 2 of 1,077 calls; across suites its effect is not significant (CMH $p = .09$, common odds ratio 0.63). In absolute terms, task success falls most on workspace, the largest suite, where the 3B gate blocks 85% of calls and cuts task success from .264 to .163 although no attack succeeded in either arm.

The between-arm CMH treats arms as independent, but task–goal pairs recur in every arm, so we reran the three injectable suites with per-episode logging ($N = 389$). Agent sampling (and endpoint nondeterminism) shifts the counts to 38/389 at baseline and 9/389 with the 3B gate (suite-stratified CMH $p = 1.9\times10^{-5}$, MH odds ratio 0.21). Paired runs differ in both directions: 34 baseline-only against 5 gate-only attack successes (McNemar $p = 2.4\times10^{-6}$), and 29 against 22 for the 7B gate ($p = .40$). These tests treat episodes as independent in a crossed design (slack has only 5 goals), so they overstate precision for new tasks and goals; and without an A/A (ungated versus ungated) control, the gate-only successes are consistent with, but do not isolate, decision-dependent feedback (the non-identifiability result, Proposition 3 and Corollary 2, is formal and does not depend on this experiment). The two gates are selected policies with different costs, not estimates of AgentDojo's controlled frontier or of a causal effect of judge scale, and a sweep of Gemma-2-9B over $\theta \in \{.90, .95, .98\}$ on banking and slack did not significantly reduce attack success at any point (Supplement B.5).

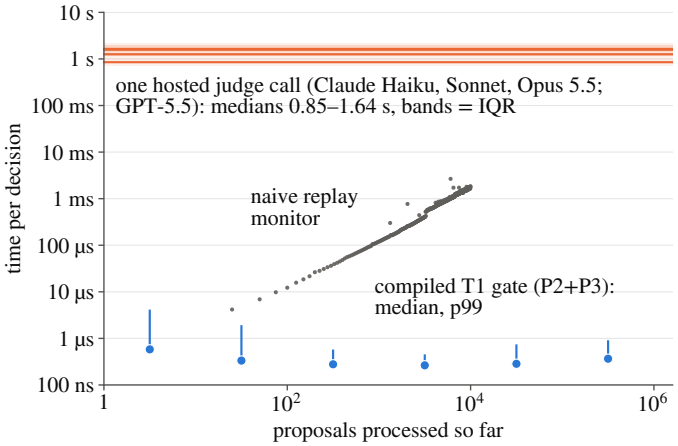


Fig. S7. Time per decision of the compiled P2+P3 gate (blue: median and 99th percentile in each decade of one 10^6-proposal stream over 10^6 recipients), of a naive replay monitor that recomputes the policy from the committed history for every proposal (gray; same stream, no blocks in this range), and of one hosted judge call (orange: median and interquartile range for Claude Haiku, Sonnet, and Opus 5.5 and GPT-5.5, 24 sequential calls each, network included). Synthetic workload, one core of an Intel i9-9820X.

E.6 Q5: The Cost of Exact Enforcement and Analysis

The rule that T1 and T3 support (compile the oracle-checkable predicates, judge only the rest) is useful only if compiled gates and their analysis are cheap. We implement P2 and P3 as one keyed-counter gate (a hash map from active recipient–day keys to monotone counters, plus a revocation flag) and time it in Python 3.12 on one core of an Intel i9-9820X. Over a stream of 10^6 proposals to 10^6 recipients, the median decision takes 0.26–0.37 μs in every decade after the first ten proposals (99th percentile below 1 μs after the first hundred) while the state grows to 7.7×10^5 active keys at about 110 bytes each (Figure S7); with 10^2 to 10^6 recipients the median stays within 0.27–0.42 μs. A naive replay monitor that recomputes the policy from the committed history for each proposal, the literal reading of Definition 4, agrees with the gate on all 2×10^4 proposals of two streams, one oversampling exact-cap and cap-plus-one amounts (8,950 blocks), but its decision time grows with the history, to a median of 0.93 ms at 10^3–10^4 committed actions. One hosted judge call from our panel, with the study's prompt and settings, takes a median of 0.85 s (Claude Haiku 5.5) to 1.64 s (GPT-5.5) over 24 sequential calls, network included, about six orders of magnitude more than the compiled check. The saturation abstraction of Theorem 5 also supports a redundancy check for this instance: an exact search over one saturated P2 key and the saturated session counter (exact because deleting other keys' transfers cannot make the upper-bound limit reject; Theorem 5 itself covers only nontriviality) finds that a per-session transfer limit ℓ makes P2's per-recipient cap c redundant (no committed history lets P2 block a proposal the limit allows) in each tested configuration with $\ell \le c$ and in none with $\ell > c$, matching the analytic condition (five configurations with $c, \ell \le 200$; 21–508 ms per query). The saturated state space grows roughly quadratically in the caps (15,251 states and 6.1×10^6 transitions at $c = 100$, $\ell = 200$), so caps in fine monetary units need coarser units or a symbolic counter encoding. On these synthetic workloads, the exact half of a guardrail is effectively free; a gate's latency, cost, and false blocks come from the predicates it must judge.

E.7 Q3: An Optimizing Attacker on Instruction Strings

The template and grammar searches of B.19–B.20 draw rewrites without learning from the judge's scores. To test whether the low evasion counts at the in-sample $\delta = .10$ gate (main-paper §7.4) reflect robustness or weak attacks, we ran an optimizing attacker in the style of PAIR [106], with a protocol fixed in the module docstring of `frontier2026/frontier2026_pair.py` before the full run. *Judges and scoring:* Claude Opus, Sonnet, and Haiku 5.5 and GPT-5.5, each through the exact scorer of B.20 (paper prompt and decoding; a provider refusal, an unparseable reply, or an API error after four retries scores 1.0). *Items:* the 35 injection goals with clean run-1 scores at or above θ_{cal}, the $\delta = .10$ split-conformal threshold on all 35 clean violation scores (the in-sample gate of §7.4; $\theta_{\text{cal}} = .15, .05, .08, .05$). *Budget* (Definition 6, as in B.20): the canonical action text is kept verbatim and the attacker adds only a prefix and/or a suffix of at most 45 words in total, so item and label are unchanged. *Attacker:* Claude Sonnet 5.5 (thinking disabled), prompted as an authorized red-teamer of a safety classifier; it sees the action, the judge's prompt template, the target score, and the history of its candidates with the judge's integer scores, and returns one candidate per round. A refused or unusable attacker turn consumes its round without a judge query (conservative for the attacker). *Stopping:* at most $K = 20$ rounds per item; stop at the first score below θ_{cal}.

Table S18 reports the results under two counting rules. The protocol's rule is the paper's (first integer in the reply). An independent audit of the trajectories found that 9 of Sonnet 5.5's 18 evasions under that rule are prose replies that quote the attacker's "score 0" instruction, so its first integer is not a rating; Opus 5.5 and GPT-5.5 replied with numbers only, and Haiku's 5 prose replies do not change its count at $K = 20$. Our primary, conservative count therefore keeps only numeric replies and only violations whose clean score lies strictly above θ_{cal}, which removes 1–3 items per judge whose evasion is a two-to-seven-point drop that sampling noise could explain (the judges run at provider-default temperature). Under that count the optimizing attacker evades the in-sample $\delta = .10$ gate on 13/33 (Opus), 8/33 (Sonnet), 13/34 (Haiku), and 13/35 (GPT-5.5) flagged violations. A pre-declared stability check (`frontier2026/frontier2026_pair_rescore.py`, fixed before it ran) re-scored the first evading rewrite of every item five more times with strict parsing (a bare integer, otherwise 1.0), and the unmodified violation five times as a null control. Counting a rewrite only if at least three of its five fresh scores fall below θ_{cal} while at most two of the clean scores do, the attacker evades on 9/33 (Opus), 9/33 (Sonnet), 12/34 (Haiku), and 10/35 (GPT-5.5) flagged violations, 6–8 of the 28 whose text is not benign (B.6); this is the count the main paper reports. Best-of-six evades 2 of 30 for GPT-5.5 and none for the Claude judges. The attacker model refused every one of its 20 turns on 12–13 items per judge (refusals are sticky: $P(\text{refuse} \mid \text{previous refused}) \ge .92$), so about a third of the items were never attacked and all counts are lower bounds. The grammar search of B.20 is not a like-for-like comparison at θ_{cal}: it stopped at the first draw below θ_f, so its counts there (0, 1, 2, 1) are censored, and it attacked only the violations flagged at $\theta = .5$. Many evading candidates assert false context about the action (a sandbox, an owner's approval, a cap that makes "as much as possible" harmless) or pre-fill the judge's answer slot; the action string and its effect are unchanged, as Definition 6 requires, but the rewrites go beyond the fixed grammar fillers of B.20. Each count lower-bounds $\eta(\rho; \theta_{\text{cal}})$ for a 45-word

TABLE S18
Optimizing (PAIR-style) attacker at the in-sample $\delta = .10$ gate (θ_{cal}). Stable: the selected rewrite evades in ≥ 3 of 5 fresh strict re-scores and the clean violation in ≤ 2 of 5; conservative: numeric judge replies only and clean score above θ_{cal}; protocol: the paper's first-integer rule. Clopper–Pearson 95% intervals; "non-benign": among the 28 violations whose text is not benign; grammar: B.20's best-of-20 search, censored at θ_f.

	Opus 5.5	Sonnet 5.5	Haiku 5.5	GPT-5.5
θ_{cal} / items	.15 / 33	.05 / 33	.08 / 34	.05 / 35
items attacked	21	20	22	23
evaded, stable (main paper)	9	9	12	10
CP 95%	[.13, .46]	[.13, .46]	[.20, .54]	[.15, .46]
non-benign	7/28	6/28	8/28	6/28
evaded, conservative	13	8	13	13
evaded, protocol rule	14	18	16	16
grammar (censored)	0/26	1/23	2/24	1/30
judge queries	162	57	115	143
attacker refusals	326	296	301	292

insertion budget and exceeds $\delta/H = .10$, so the precondition of Corollary 3 fails for all four judges at this budget. At the flagged-subclass threshold θ_f of B.20, where all of its replies are numeric, the attacker raises Claude Opus 5.5's flagged-subclass miss increase η_f from .077 (grammar) to .269; for the other judges prose replies affect both searches at θ_f, so we do not compare them there.

REFERENCES

[1] J. Ligatti, L. Bauer, and D. Walker, "Edit automata: Enforcement mechanisms for run-time security policies," *Int. J. Inf. Secur.*, vol. 4, no. 1–2, pp. 2–16, 2005.

[2] ——, "Run-time enforcement of nonsafety policies," *ACM Trans. Inf. Syst. Secur.*, vol. 12, no. 3, pp. 19:1–19:41, 2009.

[3] M. Kaminski and N. Francez, "Finite-memory automata," *Theor. Comput. Sci.*, vol. 134, no. 2, pp. 329–363, 1994.

[4] J. Neyman and E. S. Pearson, "On the problem of the most efficient tests of statistical hypotheses," *Phil. Trans. Roy. Soc. A*, vol. 231, pp. 289–337, 1933.

[5] X. Tong, Y. Feng, and J. J. Li, "Neyman–Pearson classification algorithms and NP receiver operating characteristics," *Sci. Adv.*, vol. 4, no. 2, p. eaao1659, 2018.

[6] A. N. Angelopoulos, S. Bates, A. Fisch, L. Lei, and T. Schuster, "Conformal risk control," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[7] M. A. Rahman *et al.*, "Beyond aggregate risk: Role-stratified conformal risk control for LLM tool calls," 2026, arXiv:2607.24343.

[8] S. Demri and R. Lazić, "LTL with the freeze quantifier and register automata," *ACM Trans. Comput. Log.*, vol. 10, no. 3, pp. 16:1–16:30, 2009.

[9] K. N. Verma, H. Seidl, and T. Schwentick, "On the complexity of equational Horn clauses," in *Proc. Int. Conf. Automated Deduction (CADE)*, ser. LNCS, vol. 3632. Springer, 2005, pp. 337–352.

[10] B. Scarpellini, "Complexity of subcases of Presburger arithmetic," *Trans. Amer. Math. Soc.*, vol. 284, no. 1, pp. 203–218, 1984.

[11] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Adv. Neural Inf. Process. Syst. (NeurIPS) Datasets and Benchmarks Track*, 2024.

[12] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings Assoc. Comput. Linguistics (ACL)*, 2024, pp. 10 471–10 506.

[13] M. Andriushchenko *et al.*, "AgentHarm: A benchmark for measuring harmfulness of LLM agents," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[14] H. Inan *et al.*, "Llama Guard: LLM-based input-output safeguard for human-AI conversations," 2023, arXiv:2312.06674.

[15] A. Grattafiori *et al.*, "The Llama 3 herd of models," 2024, arXiv:2407.21783.

[16] A. Cartagena and A. Teixeira, "Mind the GAP: Text safety does not transfer to tool-call safety in LLM agents," 2026, arXiv:2602.16943.

[17] N. Maloyan and D. Namiot, "Adversarial attacks on LLM-as-a-judge systems: Insights from prompt injections," 2025, arXiv:2504.18333.

[18] A. Gendler, T.-W. Weng, L. Daniel, and Y. Romano, "Adversarially robust conformal prediction," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[19] A. Kumar, C. Agarwal, S. Srinivas, A. J. Li, S. Feizi, and H. Lakkaraju, "Certifying LLM safety against adversarial prompting," in *Proc. Conf. Lang. Model. (COLM)*, 2024.

[20] J. Hughes *et al.*, "Best-of-N jailbreaking," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 38, 2025.

[21] Protect AI, "Fine-tuned DeBERTa-v3-base for prompt injection detection," Hugging Face model card, rev. 90c9989, 2024, accessed Nov. 2, 2026. [Online]. Available: https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v2

[22] H. Li, X. Liu, N. Zhang, and C. Xiao, "PIGuard: Prompt injection guardrail via mitigating overdefense for free," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2025, pp. 30 420–30 437.

[23] M. Nasr *et al.*, "The attacker moves second: Stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections," in *Proc. 35th USENIX Secur. Symp.*, 2026.

[24] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. 48th IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*. ACM, 2026, pp. 2938–2950.

[25] T. Shi *et al.*, "Progent: Securing AI agents with privilege control," 2025, arXiv:2504.11703.

[26] V. Reddy, S. R. Challaram, and A. Basu, "Reason less, verify more: Deterministic gates recover a silent policy-violation failure mode in tool-using LLM agents," 2026, arXiv:2607.07405.

[27] Y. Xiao and P. Nuzzo, "Symbolic temporal supervision of LLM agents using contracts," 2026, arXiv:2609.18128.

[28] B. Wu, W. Zhang, K. Chen, H. Fang, and N. Yu, "Provably secure agent guardrail," 2026, arXiv:2605.29251.

[29] A. Kamath, S. Zhang, C. Xu, S. Ugare, G. Singh, and S. Misailovic, "Enforcing temporal constraints for LLM agents," 2025, arXiv:2512.23738.

[30] E. Debenedetti *et al.*, "Defeating prompt injections by design," in *Proc. IEEE Conf. Secure Trustworthy Mach. Learn. (SaTML)*, 2026, pp. 587–618.

[31] M. Costa *et al.*, "Securing AI agents with information-flow control," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2026.

[32] L. Beurer-Kellner *et al.*, "Design patterns for securing LLM agents against prompt injections," 2025, arXiv:2506.08837.

[33] Y. Wu, F. Roesner, T. Kohno, N. Zhang, and U. Iqbal, "IsolateGPT: An execution isolation architecture for LLM-based agentic systems," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2025.

[34] E. Li, T. Mallick, E. Rose, W. Robertson, A. Oprea, and C. Nita-Rotaru, "ACE: A security architecture for LLM-integrated app systems," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2026.

[35] P. Y. Zhong *et al.*, "RTBAS: Defending LLM agents against prompt injection and privacy leakage," 2025, arXiv:2502.08966.

[36] B. Delattre, C. Wang, and Y. Cao, "CAGE: Certified authorization under typed-return uncertainty for tool-using agents," 2026, arXiv:2607.29190.

[37] H. Li, X. Liu, H.-C. Chiu, D. Li, N. Zhang, and C. Xiao, "DRIFT: Dynamic rule-based defense with injection isolation for securing LLM agents," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 38, 2025.

[38] H. Hu, P. Chen, Y. Zhao, and Y. Chen, "AgentSentinel: An end-to-end and real-time security defense framework for computer-use agents," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2025, pp. 3535–3549.

[39] S. Han *et al.*, "WildGuard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of LLMs," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2024.

[40] D. Liu *et al.*, "AgentDoG: A diagnostic guardrail framework for AI agent safety and security," 2026, arXiv:2601.18491.

[41] S. Chennabasappa *et al.*, "LlamaFirewall: An open source guardrail system for building secure AI agents," 2025, arXiv:2505.03574.

[42] K. Zhu, X. Yang, J. Wang, W. Guo, and W. Y. Wang, "MELON: Provable defense against indirect prompt injection attacks in AI agents," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2025.

[43] Y. Liu, Y. Jia, J. Jia, D. Song, and N. Z. Gong, "DataSentinel: A game-theoretic detection of prompt injection attacks," in *Proc. IEEE Symp. Secur. Privacy (S&P)*, 2025, pp. 2190–2208.

[44] Y. Feng *et al.*, "CORA: Conformal risk-controlled agents for safeguarded mobile GUI automation," 2026, arXiv:2604.09155.

[45] W. Scarbro *et al.*, "Conformal safety shielding for imperfect-perception agents," in *Int. Conf. Runtime Verification (RV)*, 2025.

[46] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The Task Shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2025, pp. 29 680–29 697.

[47] Y. Mou *et al.*, "ToolSafe: Enhancing tool invocation safety of LLM-based agents via proactive step-level guardrail and feedback," in *Findings Assoc. Comput. Linguistics (ACL)*. San Diego, CA, USA: Association for Computational Linguistics, 2026, pp. 37 125–37 153.

[48] A. Anand, I. Chatzi, R. Raha, and A.-K. Schmuck, "MENTOR: Procedural shielding of tool-using LLM agents," 2026, arXiv:2610.17039.

[49] A. Baheri and P. Wei, "Controller-agnostic runtime safety certificates for tool-using AI agents via data-driven robust reachability," *IEEE Control Syst. Lett.*, 2026, early access.

[50] C. Gu and Q. Zhu, "Certified execution harness for agentic AI: Safe and least-intrusive Petri-net supervisory control between reasoning and action," 2026, arXiv:2610.31376.

[51] F. B. Schneider, "Enforceable security policies," *ACM Trans. Inf. Syst. Secur.*, vol. 3, no. 1, pp. 30–50, 2000.

[52] K. W. Hamlen, G. Morrisett, and F. B. Schneider, "Computability classes for enforcement mechanisms," *ACM Trans. Program. Lang. Syst.*, vol. 28, no. 1, pp. 175–205, 2006.

[53] P. W. L. Fong, "Access control by tracking shallow execution history," in *Proc. IEEE Symp. Secur. Privacy (S&P)*, 2004, pp. 43–55.

[54] C. Talhi, N. Tawbi, and M. Debbabi, "Execution monitoring enforcement under memory-limitation constraints," *Inf. Comput.*, vol. 206, no. 2–4, pp. 158–184, 2008.

[55] P. J. Ramadge and W. M. Wonham, "Supervisory control of a class of discrete event processes," *SIAM J. Control Optim.*, vol. 25, no. 1, pp. 206–230, 1987.

[56] R. Bloem, B. Könighofer, R. Könighofer, and C. Wang, "Shield synthesis: Runtime enforcement for reactive systems," in *Proc. Int. Conf. Tools Algorithms Construction Anal. Syst. (TACAS)*, ser. LNCS, vol. 9035. Springer, 2015, pp. 533–548.

[57] S. Pinisetty, P. S. Roop, S. Smyth, S. Tripakis, and R. von Hanxleden, "Runtime enforcement of reactive systems using synchronous enforcers," in *Proc. 24th ACM SIGSOFT Int. SPIN Symp. Model Checking Softw. (SPIN)*, 2017, pp. 80–89.

[58] A. Anand, L. Germerie Guizouarn, T. Jéron, S. Mukherjee, S. Pinisetty, and O. Sankur, "Prompt runtime enforcement," in *Proc. Int. Symp. Automated Technol. Verification Anal. (ATVA)*, ser. LNCS, vol. 16145. Springer, 2025, pp. 135–156.

[59] L. Aceto, I. Cassar, A. Francalanza, and A. Ingólfsdóttir, "On first-order runtime enforcement of branching-time properties," *Acta Informatica*, vol. 60, pp. 385–451, 2023.

[60] F. Hublet, L. Lima, D. Basin, S. Krstić, and D. Traytel, "Proactive real-time first-order enforcement," in *Proc. Int. Conf. Comput. Aided Verification (CAV)*, ser. LNCS, vol. 14682. Springer, 2024, pp. 156–181.

[61] G. Roşu and F. Chen, "Semantics and algorithms for parametric monitoring," *Log. Methods Comput. Sci.*, vol. 8, no. 1, 2012.

[62] D. Basin, F. Klaedtke, S. Müller, and E. Zălinescu, "Monitoring metric first-order temporal properties," *J. ACM*, vol. 62, no. 2, pp. 15:1–15:45, 2015.

[63] H. Barringer, Y. Falcone, K. Havelund, G. Reger, and D. Rydeheard, "Quantified event automata: Towards expressive and efficient runtime monitors," in *Proc. Int. Symp. Formal Methods (FM)*, ser. LNCS, vol. 7436. Springer, 2012, pp. 68–84.

[64] J. Ligatti and S. Reddy, "A theory of runtime enforcement, with results," in *Proc. Eur. Symp. Res. Comput. Secur. (ESORICS)*, ser. LNCS, vol. 6345. Springer, 2010, pp. 87–100.

[65] R. Grigore, D. Distefano, R. L. Petersen, and N. Tzevelekos, "Runtime verification based on register automata," in *Proc. Int. Conf. Tools Algorithms Construction Anal. Syst. (TACAS)*, ser. LNCS, vol. 7795. Springer, 2013, pp. 260–276.

[66] T. Ferrère, T. A. Henzinger, and N. E. Saraç, "A theory of register monitors," in *Proc. 33rd Annu. ACM/IEEE Symp. Logic Comput. Sci. (LICS)*. ACM, 2018, pp. 394–403.

[67] D. Basin, V. Jugé, F. Klaedtke, and E. Zălinescu, "Enforceable security policies revisited," *ACM Trans. Inf. Syst. Secur.*, vol. 16, no. 1, pp. 3:1–3:26, 2013.

[68] Y. Falcone, J.-C. Fernandez, and L. Mounier, "What can you verify and enforce at runtime?" *Int. J. Softw. Tools Technol. Transf. (STTT)*, vol. 14, no. 3, pp. 349–382, 2012.

[69] B. A. Shivakumar, S. Priya, and P. Gao, "Agenra: A language and verified core for stateful LLM agent authorization," 2026, arXiv:2608.22868.

[70] I. Cohen, K. Havelund, M. Omer, and D. Peled, "Temporal guardrails for LLM conversations: A runtime verification framework," in *Proc. Int. Symp. AI Verif. (SAIV)*, ser. LNCS, vol. 16831. Springer, 2026, pp. 145–166.

[71] N. Kekatos *et al.*, "Formal runtime verification for tool-using LLM agents: An offline same-benchmark study on AgentDojo and STAC," in *Proc. 8th Joint Workshop CPS & IoT Secur. Privacy (CPSIoTSec)*, 2026, pp. 129–142.

[72] X. Du, A. Tiu, K. Cheng, and Y. Liu, "Trace-length independent runtime monitoring of quantitative policies," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 3, pp. 1489–1510, 2021.

[73] I. David, M. Guarnieri, and A. Gervais, "Alignment contracts for agentic security systems," 2026, arXiv:2605.00081.

[74] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop Artif. Intell. Secur. (AISec)*, 2023, pp. 79–90.

[75] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 1831–1847.

[76] Y. Guo *et al.*, "MCPXkit: The unified toolkit for analyzing Model Context Protocol security," *IEEE Trans. Dependable Secure Comput.*, vol. 23, no. 5, pp. 9796–9811, 2026.

[77] M. Yang, T. Zhu, B. Liu, H. Xu, and W. Zhou, "Cracks in collaboration: Threat models and attacks on multi-LLM collaborative systems," *IEEE Trans. Dependable Secure Comput.*, vol. 23, no. 3, pp. 7191–7207, 2026.

[78] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in *Proc. 34th USENIX Secur. Symp.*, 2025, pp. 2383–2400.

[79] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, and C. Guo, "SecAlign: Defending against prompt injection with preference optimization," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2025, pp. 2833–2847.

[80] S. Choudhary, D. Anshumaan, N. Palumbo, and S. Jha, "How not to detect prompt injections with an LLM," in *Proc. 18th ACM Workshop Artif. Intell. Secur. (AISec)*, 2025, pp. 218–229.

[81] X. Zhang *et al.*, "PrivacyAsst: Safeguarding user privacy in tool-using large language model agents," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 6, pp. 5242–5258, 2024.

[82] V. Siu *et al.*, "A framework for formalizing LLM agent security," 2026, arXiv:2603.19469.

[83] M. Christodorescu *et al.*, "Systems security foundations for agentic computing," 2025, arXiv:2512.01295.

[84] Q. Zhan, R. Fang, H. S. Panchal, and D. Kang, "Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents," in *Findings Assoc. Comput. Linguistics (NAACL)*, 2025, pp. 7116–7132.

[85] L. Tsai and E. Bagdasarian, "Contextual agent security: A policy for every purpose," in *Proc. Workshop Hot Topics Oper. Syst. (HotOS)*, 2025.

[86] H. Wang, C. M. Poskitt, J. Wei, and J. Sun, "ProbGuard: Proactive runtime monitoring for LLM agent safety via probabilistic prediction," in *Proc. 41st IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*. ACM, 2026, pp. 1470–1482.

[87] L. Miculicich *et al.*, "VeriGuard: Enhancing LLM agent safety via verified code generation," 2025, arXiv:2510.05156.

[88] Z. Chen, M. Kang, and B. Li, "ShieldAgent: Shielding agents via verifiable safety policy reasoning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 267. PMLR, 2025, pp. 8313–8344.

[89] Z. Xiang *et al.*, "GuardAgent: Safeguard LLM agents via knowledge-enabled reasoning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 267. PMLR, 2025, pp. 68 316–68 342.

[90] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proc. 2023 Conf. Empirical Methods Natural Lang. Process. (EMNLP): System Demonstrations*, 2023, pp. 431–445.

[91] Y. Sun, J. Zhang, S. Cohney, Z. Zhang, F. Liu, and X. Yuan, "From risk classification to action plan remediation: A guardrail feedback driven framework for LLM agents," 2026, arXiv:2606.05805.

[92] J. Shi *et al.*, "Optimization-based prompt injection attack to LLM-as-a-judge," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2024, pp. 660–674.

[93] V. Raina, A. Liusie, and M. Gales, "Is LLM-as-a-judge robust? Investigating universal adversarial attacks on zero-shot LLM assessment," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2024, pp. 7499–7517.

[94] N. Mangaokar *et al.*, "PRP: Propagating universal perturbations to attack large language model guard-rails," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024, pp. 10 960–10 976.

[95] E. Bassani and I. Sanchez, "On guardrail models' robustness to mutations and adversarial attacks," in *Findings Assoc. Comput. Linguistics (EMNLP)*, 2025, pp. 16 995–17 006.

[96] H. Xu *et al.*, "RedAgent: An autonomous agent for context-aware red teaming of LLM jailbreaks," *IEEE Trans. Dependable Secure Comput.*, vol. 23, no. 3, pp. 6506–6521, 2026.

[97] M. Terekhov *et al.*, "Adaptive attacks on trusted monitors subvert AI control protocols," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.

[98] A. Remedios, S. Storf, F. Roger, and J. Hughes, "Red-teaming Auto Mode: Improving blocking classifiers against malign coding agents," 2026, arXiv:2609.19587.

[99] Z. Wei, Y. Liu, and N. B. Erichson, "Emoji attack: Enhancing jailbreak attacks against judge LLM detection," in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 267, 2025, pp. 66 103–66 117.

[100] W. Hackett, L. Birch, S. Trawicki, N. Suri, and P. Garraghan, "Bypassing LLM guardrails: An empirical analysis of evasion attacks against prompt injection and jailbreak detection systems," in *Proc. 1st Workshop LLM Secur. (LLMSEC)*, 2025, pp. 101–114.

[101] C. Yin, R. Geng, Y. Wang, and J. Jia, "PISmith: Reinforcement learning-based red teaming for prompt injection defenses," in *Proc. Conf. Lang. Model. (COLM)*, 2026.

[102] D. Schmotz, D. Prinzhorn, L. Beurer-Kellner, A. Paulus, A. Prabhu, and M. Andriushchenko, "Instrumental monitor evasion emerges under ordinary task pressure," 2026, arXiv:2609.30217.

[103] J. Kutasov *et al.*, "SHADE-Arena: Evaluating sabotage and monitoring in LLM agents," 2025, arXiv:2506.15740.

[104] H. Zhang *et al.*, "Agent Security Bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025, pp. 35 331–35 366.

[105] I. Evtimov, A. Zharmagambetov, A. Grattafiori, C. Guo, and K. Chaudhuri, "WASP: Benchmarking web agent security against prompt injection attacks," in *Adv. Neural Inf. Process. Syst. (NeurIPS) Datasets and Benchmarks Track*, vol. 38, 2025.

[106] P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, and E. Wong, "Jailbreaking black box large language models in twenty queries," in *Proc. IEEE Conf. Secure Trustworthy Mach. Learn. (SaTML)*, 2025, pp. 23–42.

[107] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, 2018, pp. 2669–2678.

[108] N. Jansen, B. Könighofer, S. Junges, A. Serban, and R. Bloem, "Safe reinforcement learning using probabilistic shields," in *Proc. Int. Conf. Concurrency Theory (CONCUR)*, ser. LIPIcs, vol. 171, 2020, pp. 3:1–3:16.

[109] T. Sah, V. Srivastava, D. Sah, and K. Jordan, "The verifier tax: Horizon dependent safety–success tradeoffs in tool using LLM agents," in *Proc. ACM Conf. AI Agentic Syst. (CAIS)*, 2026, pp. 785–799.

[110] R. Vasudev, M. Russak, D. Bikel, and W. Alshikh, "Accurate failure prediction in agents does not imply effective failure prevention," 2026, arXiv:2602.03338.

[111] J. Lucassen and A. Kaufman, "Retrying vs resampling in AI control," 2026, arXiv:2605.26047.

[112] J. C. Perdomo, T. Zrnic, C. Mendler-Dünner, and M. Hardt, "Performative prediction," in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 119, 2020.

[113] V. Li, B. Chen, Y. Mao, Q. Lei, and Z. Deng, "Performative risk control: Calibrating models for reliable deployment under performativity," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2025.

[114] R. Greenblatt, B. Shlegeris, K. Sachan, and F. Roger, "AI control: Improving safety despite intentional subversion," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2024.

[115] C. Griffin, L. Thomson, B. Shlegeris, and A. Abate, "Games for AI control: Models of safety evaluations of AI deployment protocols," in *Proc. 2nd IASEAI Conf. (IASEAI'26)*, vol. 2, no. 1, 2026, pp. 204–216.

[116] T. T. Hua, J. Baskerville, H. Lemoine, M. Hopman, A. Bhatt, and T. Tracy, "Combining cost-constrained runtime monitors for AI safety," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 38, 2025, pp. 180 058–180 083.

[117] A. Bhatt *et al.*, "Ctrl-Z: Controlling AI agents via resampling," 2025, arXiv:2504.10374.

[118] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artif. Intell.*, vol. 101, no. 1–2, pp. 99–134, 1998.

[119] E. Altman, *Constrained Markov Decision Processes*. Boca Raton, FL, USA: Chapman and Hall/CRC, 1999.

[120] P. Poupart, A. Malhotra, P. Pei, K.-E. Kim, B. Goh, and M. Bowling, "Approximate linear programming for constrained partially observable Markov decision processes," in *Proc. Twenty-Ninth AAAI Conf. Artif. Intell.* AAAI Press, 2015, pp. 3342–3348.

[121] E. Walraven and M. T. J. Spaan, "Column generation algorithms for constrained POMDPs," *J. Artif. Intell. Res.*, vol. 62, pp. 489–533, 2018.

[122] N. Jiang and L. Li, "Doubly robust off-policy value evaluation for reinforcement learning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 48, 2016, pp. 652–661.

[123] C. K. Chow, "On optimum recognition error and reject tradeoff," *IEEE Trans. Inf. Theory*, vol. 16, no. 1, pp. 41–46, 1970.

[124] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017.

[125] D. Blackwell, "Equivalent comparisons of experiments," *Ann. Math. Statist.*, vol. 24, no. 2, pp. 265–272, 1953.

[126] A. Z. Ren *et al.*, "Robots that ask for help: Uncertainty alignment for large language model planners," in *Proc. 7th Conf. Robot Learn. (CoRL)*, ser. PMLR, vol. 229. PMLR, 2023, pp. 661–682.

[127] A. Doula, M. Mühlhäuser, and A. Sanchez Guinea, "SafePath: Conformal prediction for safe LLM-based autonomous navigation," 2025, arXiv:2505.09427.

[128] A. N. Angelopoulos, S. Bates, E. J. Candès, M. I. Jordan, and L. Lei, "Learn then test: Calibrating predictive algorithms to achieve risk control," *Ann. Appl. Statist.*, vol. 19, no. 2, pp. 1641–1662, 2025.

[129] J. Jung, F. Brahman, and Y. Choi, "Trust or escalate: LLM judges with provable guarantees for human agreement," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[130] F. Barbero, F. Pendlebury, F. Pierazzi, and L. Cavallaro, "Transcending TRANSCEND: Revisiting malware classification in the presence of concept drift," in *Proc. IEEE Symp. Secur. Privacy (S&P)*, 2022, pp. 805–823.

[131] R. F. Barber, E. J. Candès, A. Ramdas, and R. J. Tibshirani, "Conformal prediction beyond exchangeability," *Ann. Statist.*, vol. 51, no. 2, pp. 816–845, 2023.

[132] C. Fannjiang, S. Bates, A. N. Angelopoulos, J. Listgarten, and M. I. Jordan, "Conformal prediction under feedback covariate shift for biomolecular design," *Proc. Natl. Acad. Sci. USA*, vol. 119, no. 43, p. e2204569119, 2022.

[133] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. Springer, 2005.

[134] G. Yan, Y. Romano, and T.-W. Weng, "Provably robust conformal prediction with improved efficiency," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[135] S. H. Zargarbashi, M. S. Akhondzadeh, and A. Bojchevski, "One sample is enough to make conformal prediction robust," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2025.

[136] Y. Li, A. Chen, W. Qian, C. Zhao, D. Lidder, and M. Huai, "Data poisoning attacks against conformal prediction," in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 235, 2024, pp. 27 563–27 574.

[137] H. Liu, H. Huang, X. Gu, H. Wang, and Y. Wang, "On calibration of LLM-based guard models for reliable content moderation," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[138] S. Ball, G. Gluch, S. Goldwasser, F. Kreuter, O. Reingold, and G. N. Rothblum, "On the impossibility of separating intelligence from judgment: The computational intractability of filtering for AI alignment," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.

[139] L. Sha, "Using simplicity to control complexity," *IEEE Softw.*, vol. 18, no. 4, pp. 20–28, 2001.

[140] D. E. Eckhardt and L. D. Lee, "A theoretical basis for the analysis of multiversion software subject to coincident errors," *IEEE Trans. Softw. Eng.*, vol. SE-11, no. 12, pp. 1511–1517, 1985.

[141] S. Axelsson, "The base-rate fallacy and the difficulty of intrusion detection," *ACM Trans. Inf. Syst. Secur.*, vol. 3, no. 3, pp. 186–205, 2000.

[142] M. Mitchell *et al.*, "Model cards for model reporting," in *Proc. Conf. Fairness, Accountability, Transparency (FAT*)*. ACM, 2019, pp. 220–229.

[143] E. R. DeLong, D. M. DeLong, and D. L. Clarke-Pearson, "Comparing the areas under two or more correlated receiver operating characteristic curves: A nonparametric approach," *Biometrics*, vol. 44, no. 3, pp. 837–845, 1988.

[144] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.